

ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ РЕСПУБЛИКИ ХАКАСИЯ
«ХАКАССКИЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

ОТЧЕТ

по преддипломной практике

специальности 09.02.07 Информационные системы и программирование
Квалификация Программист

Студент гр. ИС(ПРО)-41

подпись

Мальцева А.А.

Фамилия И.О.

Руководитель
практики
от предприятия

оценка

16.05.2026

дата

подпись

Козлов А.И.

Фамилия И.О.

М.П.

Руководитель
практики
от ГБПОУ РХ
ХПК

оценка

16.05.2026

дата

подпись

Якасова Н.В.

Фамилия И.О.

Абакан 2026 г.

СОДЕЖРАНИЕ

Введение	3
1 Общая часть	6
1.1 Общие сведения о предприятии.....	6
1.2 Описание организационной структуры предприятия	7
1.3 Описание существующей информационной системы	8
2 Анализ и проектирование	12
2.1 Постановка задачи	12
2.2 Проектирование модульной структуры	16
2.3 Проектирование структуры базы данных приложения	20
3 Разработка и интеграция приложения.....	25
3.1 Выбор средств разработки приложения.....	25
3.2 Разработка приложения	26
3.3 Разработка пользовательского интерфейса.....	34
4 Отладка и оценка качества приложения	40
4.1 Разработка тестовых наборов и тестовых сценариев.....	40
4.2 Отладка программного продукта с использованием специализированных программных средств	46
4.3 Оценка качества систем приложения	54
Заключение.....	57
Список использованных источников.....	59
Глоссарий	61
Список аббревиатур	63
Приложения	64
Приложение А Техническое задание.....	65
Приложение Б Описание структуры БД.....	71
Приложение В Программный код.....	77
Приложение Г Руководство оператора	130
Приложение Д Программный продукт на диске	134

ВВЕДЕНИЕ

В условиях цифровой трансформации современных предприятий и организаций эффективное управление внутренними процессами, включая техническое обслуживание и ремонт оборудования, становится ключевым фактором обеспечения непрерывности бизнес-процессов. Особенно остро эта задача стоит в медицинских учреждениях, где парк техники значителен, а своевременное устранение неисправностей напрямую влияет на производительность труда сотрудников и качество оказываемых пациентам услуг.

Для государственного бюджетного учреждения здравоохранения Республики Хакасия «Абаканская межрайонная клиническая больница», обладающего развитой IT-инфраструктурой и большим количеством клиентских машин, отсутствие автоматизированной системы учета заявок приводит к потере времени, некорректного распределению задач и отсутствию статистики поломок. Решение данной проблемы требует создания специализированного программного инструмента, интегрированного в существующую локальную сеть учреждения.

Внедрение разработанного приложения обеспечит сокращение времени простоя оборудования за счет оперативного оповещения специалистов, исключит потерю обращений благодаря централизованному реестру и предоставит руководству отдела ИТ объективные данные для анализа нагрузки и планирования закупок запасных частей. Практическая значимость работы заключается в создании готового к внедрению программного продукта, адаптированного под специфику работы.

Объект исследования: информационная система учёта и обработки заявок неисправностей техники в ГБУЗ РХ «АМКБ».

Предмет исследования: разработка приложения для учета заявок неисправностей техники в ГБУЗ РХ «АМКБ».

Цель работы — автоматизация процесса учета заявок неисправностей техники в ГБУЗ РХ «АМКБ».

Задачи:

- проанализировать структуру предприятия;
- провести анализ и проектирование предметной области;
- разработать и интегрировать приложение;
- провести отладку и оценку качества приложения.

Отчет по преддипломной практике состоит из: введение, общая часть, анализ и проектирование, разработка и интеграция приложения, отладка и оценка качества

приложения, заключение, список использованных источников, глоссарий, список аббревиатур, приложения.

Раздел «Введение» содержит обоснование актуальности работы, обусловленной необходимостью автоматизации учета заявок на ремонт техники в ГБУЗ РХ «АМКБ», формулировку цели работы – разработка программного обеспечения для учета заявок по неисправностям техники, а также перечень решаемых задач: анализ структуры предприятия, анализ и проектирование предметной области, разработка и интеграция приложения, проведение отладки и оценка качества приложения. Определены объект исследования и предмет исследования.

Раздел «Общая часть» содержит описание проектируемой предметной области и предприятия характеристику организационной структуры отдела информационных технологий и защиты информации с перечислением должностей и их функциональных задач. Приведено описание существующей ИТ-инфраструктуры, используемого программного обеспечения и детальный анализ проблем текущего неавтоматизированного процесса приема обращений.

Раздел «Анализ и проектирование» содержит информацию о проектировании функциональной архитектуры программного средства. Включает описание функциональных задач, решаемых системой, с использованием диаграмм: DFD-диаграммы, IDEF0-диаграмм 0 и 1 уровней, диаграммы вариантов использования, диаграммы деятельности и диаграммы последовательности. Описана многоуровневая архитектура приложения с размещением уровня бизнес-логики на стороне серверного фреймворка Django. Проектирование базы данных выполнено с помощью ER-диаграммы, описывающей 19 взаимосвязанных сущностей, с указанием типов данных, первичных и внешних ключей, а также правил ссылочной целостности.

Раздел «Реализация программного модуля» содержит информацию о разработке объектов базы данных в выбранной СУБД PostgreSQL, включая описание таблиц, индексов, ограничений целостности и связей, реализованных через ORM-модели Django. Представлена реализация функциональной архитектуры программного средства: разработаны веб-формы пользовательского интерфейса для ролей «Сотрудник», «Техник», «Администратор», «Руководитель», приведены фрагменты программного кода на Python.

Раздел «Отладка и оценка качества приложения» содержит информацию о разработанных тест-кейсах с использованием встроенного фреймворка тестирования Django на базе библиотеки unittest. Описаны тестовые сценарии, охватывающие аутентификацию пользователей с разными ролями, навигацию по разделам, мягкое и полное удаление заявок, сортировку и фильтрацию списков. Представлены результаты

автоматизированного тестирования с подтверждением успешного прохождения всех сценариев, что гарантирует соответствие приложения техническому заданию и требованиям стабильности.

Список использованных источников оформлен в соответствии с ГОСТ и состоит из официальных документов по технологиям, охватывающих вопросы проектирования ИС, стандартов ЕСКД, методологий IDEF/UML, а также документацию по Django, PostgreSQL и средствам визуализации данных.

Заключение содержит итоги проведенной работы, подтверждающие достижение поставленной цели, решение всех задач практики и готовность разработанного программного продукта к практическому внедрению в инфраструктуру ГБУЗ РХ «АМКБ».

Пояснительная записка содержит следующие приложения: техническое задание, описание структуры БД, программный код, руководство оператора и информацию о расположении программного продукта на носителе.

Пояснительная записка состоит из 63 стр., 23 рисунка, 7 таблиц.

1 ОБЩАЯ ЧАСТЬ

1.1 Общие сведения о предприятии

Государственное бюджетное учреждение здравоохранения Республики Хакасия «Абаканская межрайонная клиническая больница» представляет собой крупное многопрофильное медицинское учреждение, которое играет важную роль в системе здравоохранения региона. Больница оказывает специализированную и высокотехнологичную помощь населению, выполняя функции межрайонного центра. Учреждение обслуживает не только жителей города Абакана, но и пациентов из близлежащих районов, которые направляются сюда для получения квалифицированного лечения. В структуре больницы работает большое количество различных отделений, каждое из которых оснащено необходимым оборудованием для диагностики, лечения и обеспечения жизнедеятельности пациентов.

Для нормальной работы требуется бесперебойное функционирование самой разной техники, начиная от офисных компьютеров и принтеров и заканчивая сложными медицинскими аппаратами и системами связи. Преддипломная практика проходит в отделе информационных технологий и защиты информации под руководством начальника отдела информационных технологий и защиты информации [9].

Главная задача начальника отдела является организация работы информационного отделения, решая разнообразные технологические и кадровые вопросы осуществляя стратегическое планирование. Начальник отдела получает запросы на цифровое обслуживание, определяет схему и приказы исполнителям, контролируя результат выполнения работы. В подчинении у начальника отдела информационных технологий и защиты информации находятся специалисты, занимающиеся следующими направлениями:

- связь и телефония;
- инженерно-программная разработка;
- инженер программист;
- администрирование IP-телефонии;
- защита информации;
- сетевое администрирование;
- сопровождение медицинской информационной системы «ПроМед».

Связисты работают с телефонией, принимают информацию из штаба, передают и записывают её. Ведущий инженер-программист руководит разработкой программного обеспечения, а именно проектирует архитектуру приложений, распределяет задачи между

исполнителями и контролирует сроки реализации проектов. На инженер-программисте лежит ответственность за архитектуру, лежащую в основе разработанной программы, обучения новых разработчиков и распределения работы между ними. Инженер-программист занимается конструированием системы с элементами искусственного интеллекта, разработкой чертежей и схем, разработкой комплексных расчетов. Специалисты по защите информации обеспечивают конфиденциальность и сохранность данных на этапе разработки продукта, выполняя защиту от внешних угроз.

1.2 Описание организационной структуры предприятия

Обеспечение работоспособности технической инфраструктуры возложено на отдел информационных технологий и защиты информации. В штате отдела находятся специалисты разного профиля, включая связистов, инженеров-программистов, специалистов по защите информации и администраторов сети. В их задачи входит поддержка работы семисот клиентских машин и тринадцати серверов, администрирование локальной сети, обеспечение безопасности данных и обслуживание программного обеспечения, такого как медицинские информационные системы и бухгалтерские программы. В условиях современного медицинского учреждения практически любое оборудование тем или иным образом связано с общей инфраструктурой, и при возникновении неисправностей сотрудники обращаются именно в этот отдел за помощью или координацией ремонтных работ. Техники фиксируют проблему, определяют исполнителя и контролируют процесс устранения неисправности, независимо от того, требует ли проблема замены компьютерной мыши или ремонта сложного диагностического прибора.

В настоящее время процесс взаимодействия между сотрудниками больницы и техническими специалистами не имеет четкой организационной структуры и не автоматизирован. Когда у работника любого отделения возникает проблема с техникой, он вынужден искать контакты ответственного специалиста самостоятельно. Чаще всего обращение происходит по телефону, через личные сообщения в мессенджерах или устно при личной встрече. Такой способ подачи информации о неисправностях является крайне неорганизованным и зависит от человеческого фактора. Технический специалист, принимая звонок во время выполнения другой задачи, может не записать детали обращения или забыть перезвонить заявителю. При устных просьбах информация часто передается неточно, что приводит к тому, что мастер приезжает на место без нужных инструментов или запасных частей. В условиях высокой загруженности отдела, когда одновременно

поступает множество сигналов о проблемах, некоторые заявки просто теряются в потоке информации и остаются без внимания на неопределенный срок.

Важной проблемой является отсутствие прозрачности в распределении задач между специалистами. В отделе работают люди разной квалификации, и некоторые проблемы требуют участия узкопрофильных экспертов, не работающих в данной организации.

При текущей схеме подачи заявок через телефон или лично сложно оперативно найти свободного специалиста нужного профиля и передать ему задачу. Информация может передаваться из уст в уста, искажаясь по пути, что увеличивает время простоя оборудования. В медицинской сфере время простоя техники может критически влиять на качество оказания помощи пациентам, поэтому задержки в ремонте недопустимы. Когда ремонт выполняется без фиксации в системе, сложно понять, какие детали были потрачены, и своевременно пополнить складские запасы, что в будущем может привести к невозможности проведения ремонта из-за отсутствия комплектующих. Невозможность анализа исторических данных о частоте поломок конкретного оборудования препятствует переходу от быстрого ремонта к планово-предупредительному обслуживанию, что экономически менее выгодно для учреждения.

Кроме того, разность информации усложняет процедуру контроля гарантийных обязательств производителей техники, так как документальное подтверждение даты и характера поломки часто отсутствует. Это приводит к необоснованным финансовым затратам больницы на ремонт устройств, которые должны обслуживаться за счет поставщика. Систематизация процесса учета позволит не только оптимизировать логистику запасных частей, но и создать прозрачную систему мотивации для технических специалистов на основе реальных показателей выполненной работы.

На основе анализа выявленных недостатков текущего процесса и сформулированных требований заказчика, было разработано техническое задание, полный текст которого приведён в приложении А.

1.3 Описание существующей информационной системы

ГБУЗ РХ «АМКБ» лучше всего подходит для разработки приложения, так как имеет сложную организационную структуру, обладает развитой IT-инфраструктурой, использует современные системы учета и отчетности, а также имеет значительный парк медицинского оборудования, требующий систематического учета и контроля.

В организации существует всего 700 клиентских машин и 13 серверов, без учета филиалов организации. Клиентские машины и сервер работают на одной операционной

системе – Astra Linux. Также характеристики машин на которых работают клиенты и сервера имеют одинаковый минимум:

- от 8 Гб оперативной памяти;
- от 120 Гб основной памяти HDD или SSD диска;
- процессор Intel Pentium 7400.

Организованные серверы имеют важную роль в защите и использовании информации в организации. Он обеспечивает точность выдачи препаратов, предотвращает их нецелевое использование и формирует отчетность для фармацевтического надзора. Данные в реальном времени помогают медицинскому персоналу оперативно отслеживать наличие лекарств и минимизировать риски их нехватки. Сетевая инфраструктура предприятия построена на базе локальной сети, в которую объединены все клиентские машины и серверы организации. Топология сети реализована по комбинированному типу «звезда–шина», что обеспечивает надёжное соединение между всеми рабочими станциями и централизованными ресурсами.

Общий доменный сервер служит центральным узлом для управления учетными записями сотрудников, разграничения прав доступа и обеспечения сетевой безопасности. На нем хранятся политики паролей, групповые политики и настройки доступа к корпоративным ресурсам, что позволяет поддерживать стабильность работы всей ИТ-системы и защищать данные от несанкционированного доступа.

Сервер централизованного хранения медицинских изображений предназначен для архивации, обработки и быстрого доступа к диагностическим снимкам. Он интегрирован с радиологическим оборудованием, что позволяет врачам оперативно просматривать и анализировать изображения, ставить диагнозы и консультироваться с коллегами без необходимости физического переноса данных.

Сервер радиологической службы поддерживает работу диагностического оборудования, обеспечивает передачу данных между аппаратами и рабочими станциями врачей, а также ведет базу исследований. Это позволяет радиологам оперативно обрабатывать запросы, вести историю болезней пациентов и формировать заключения в едином формате.

Сервер видеонаблюдения обеспечивает безопасность учреждения, записывая и храня данные с камер, установленных в ключевых зонах. Он позволяет контролировать обстановку в реальном времени, расследовать инциденты и предотвращать нарушения внутренних регламентов безопасности, что особенно важно для соблюдения режима в закрытых помещениях, таких как лаборатории или хранилища лекарств.

Сервер телефонии и почты объединяет корпоративную связь, включая IP-телефонию и электронную почту. Он обеспечивает стабильную коммуникацию между сотрудниками, автоматическую маршрутизацию звонков, запись разговоров и защиту переписки от спама и кибер атак.

Сервер расчета зарплаты и отдела кадров автоматизирует процессы начисления заработной платы, учета рабочего времени, налоговых отчислений и кадрового документооборота. Он интегрирован с бухгалтерскими системами, что минимизирует ошибки при расчетах, ускоряет формирование отчетности и упрощает кадровые процедуры, такие как оформление отпусков или больничных.

Файловый сервер документооборота обеспечивает централизованное хранение, управление и обмен электронными документами в учреждении. На нём размещаются внутренние приказы, медицинские карты, отчёты, договоры и другие важные файлы в структурированном виде. Сервер поддерживает систему версионности, разграничение прав доступа для разных и резервное копирование данных. Это позволяет сотрудникам быстро находить нужные документы, совместно работать над ними и исключает потерю информации. Интеграция с электронной подписью обеспечивает юридическую значимость документов [12].

В организации для учёта и управления медицинскими данными используется специализированное программное обеспечение, которое включает следующие программы:

- электронная медицинская карта «МедЭлемент»;
- лабораторная информационная система «МедЛаб»;
- система записи на прием «ЕМИАС»;
- учет в стационаре «МедОС»;
- интеграция с диагностическим оборудованием «МедПАКС»;
- финансовый учет «1С:Бухгалтерия».

Отсутствие единой системы учета заявок создает серьезные трудности не только для сотрудников, ожидающих ремонта, но и для самих технических специалистов. Без централизованного реестра невозможно отследить статус выполнения работ, понять, какая заявка находится в очереди, а какая уже решается. Это приводит к хаосу в планировании рабочего времени инженеров и связистов. Кроме того, при таком подходе теряется история обслуживания оборудования. Если какой-то аппарат ломается регулярно, об этом нет никакой статистики, так как все обращения разрознены и нигде не фиксируются документально. Это мешает руководству отдела принимать обоснованные решения о замене устаревшей техники или проведении профилактических работ. Также возникает

проблема с учетом гарантийных обязательств. Без записи даты обращения и истории ремонтов сложно доказать производителю наличие гарантийного случая, что может привести к лишним финансовым расходам больницы на ремонт оборудования, которое должно обслуживаться бесплатно [13].

Вывод: в ходе анализа предметной области ГБУЗ РХ «АМКБ» систематизированы сведения об организационной структуре учреждения, функциях отдела информационных технологий и защиты информации, а также проведён обзор существующей ИТ-инфраструктуры и программного обеспечения. Выявлено, что разветвлённая локальная сеть и серверное оборудование на базе Astra Linux создают надёжную технологическую базу для внедрения корпоративных информационных систем. Анализ текущего бизнес-процесса приёма и обработки заявок на ремонт техники показал отсутствие его автоматизации и зависимость от ручных каналов коммуникации, что обуславливает ряд критических недостатков:

- потеря или дублирование обращений из-за отсутствия единого реестра и человеческого фактора;
- невозможность отслеживания статусов заявок;
- отсутствие истории обслуживания оборудования, что препятствует формированию статистики поломок, контролю гарантийных обязательств;
- неэффективное управление складскими запасами запчастей и непрозрачность распределения задач между сотрудниками разной квалификации.

На основании проведённого исследования обоснована объективная необходимость разработки автоматизированной системы учёта заявок по неисправностям техники. Внедрение данного программного решения позволит стандартизировать процесс регистрации обращений, обеспечить сквозной контроль выполнения работ, исключить потерю данных и сформировать аналитическую основу для оптимизации расходов на техническое обслуживание. Сформулированные требования и выявленные ограничения предметной области формируют необходимую базу для перехода к следующему этапу работы - анализу и проектированию архитектуры разрабатываемого программного средства [1].

2 АНАЛИЗ И ПРОЕКТИРОВАНИЕ

2.1 Постановка задачи

После определения основной проблемы, была разработана DFD диаграмма Йордана-Де Марко «принятие и обработка заявки на ремонт техники». Просмотреть диаграмму можно в соответствии с рисунком 2.1.

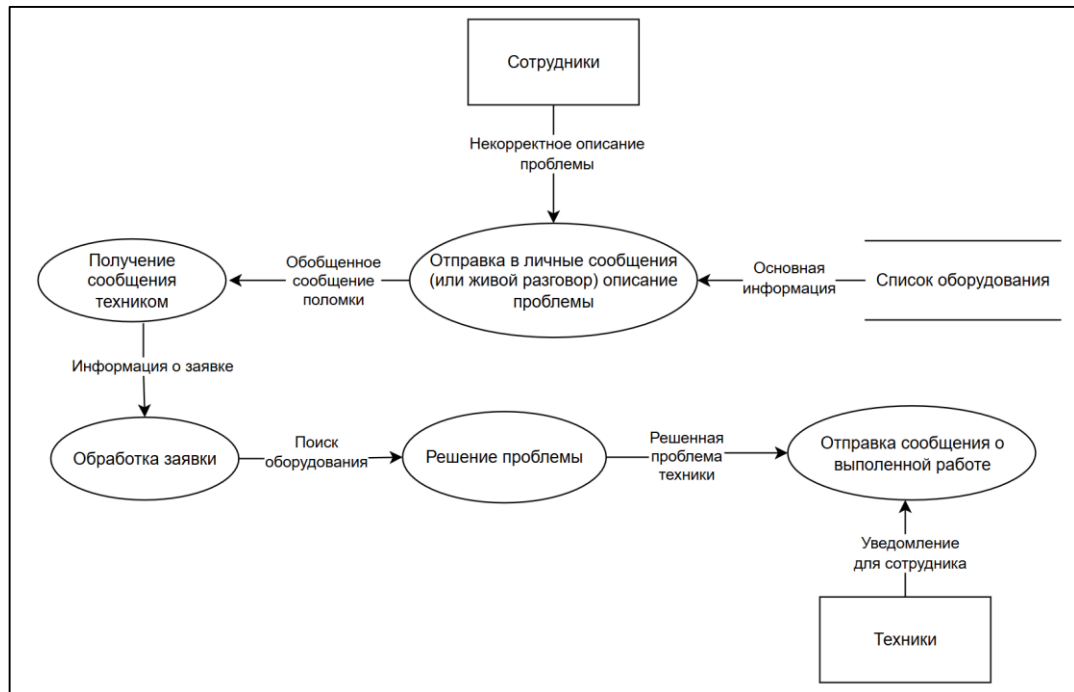


Рисунок 2.1 - DFD диаграмма Йордана-Де Марко до автоматизации процесса «принятие и обработка заявки на ремонт техники»

Представленная DFD-диаграмма демонстрирует существующий процесс обработки заявок на ремонт техники до внедрения автоматизированной системы. Процесс инициируется сотрудниками, которые направляют описание неисправности через личные сообщения или при личном общении с техническими специалистами, при этом часто возникает проблема некорректного или неполного описания проблемы. Техники получают обобщённые сообщения о поломках, обрабатывают заявки, осуществляют поиск информации об оборудовании в списках техники, после чего приступают к решению проблемы. По завершении работ технический специалист отправляет сотруднику уведомление о выполненной работе. Таким образом выявлено сразу несколько проблем:

- отсутствие стандарта для формы подачи заявки;
- ручная передача информации без фиксирования заявки;

- отсутствие контроля статусов и состояний оборудования и заявки в данный момент времени;
- отсутствие хранилища по заявкам, на случай составления картины частых поломок оборудования или уже замененных частей ранее;
- ручной поиск информации по оборудованию.

Для повышения эффективности и производительности была разработана DFD диаграмма Йордана-Де Марко, показывающая, как будет выглядеть система при организации автоматизации процесса подачи заявки на ремонт техники. DFD диаграмма Йордана-Де Марко показана в соответствии с рисунком 2.2.

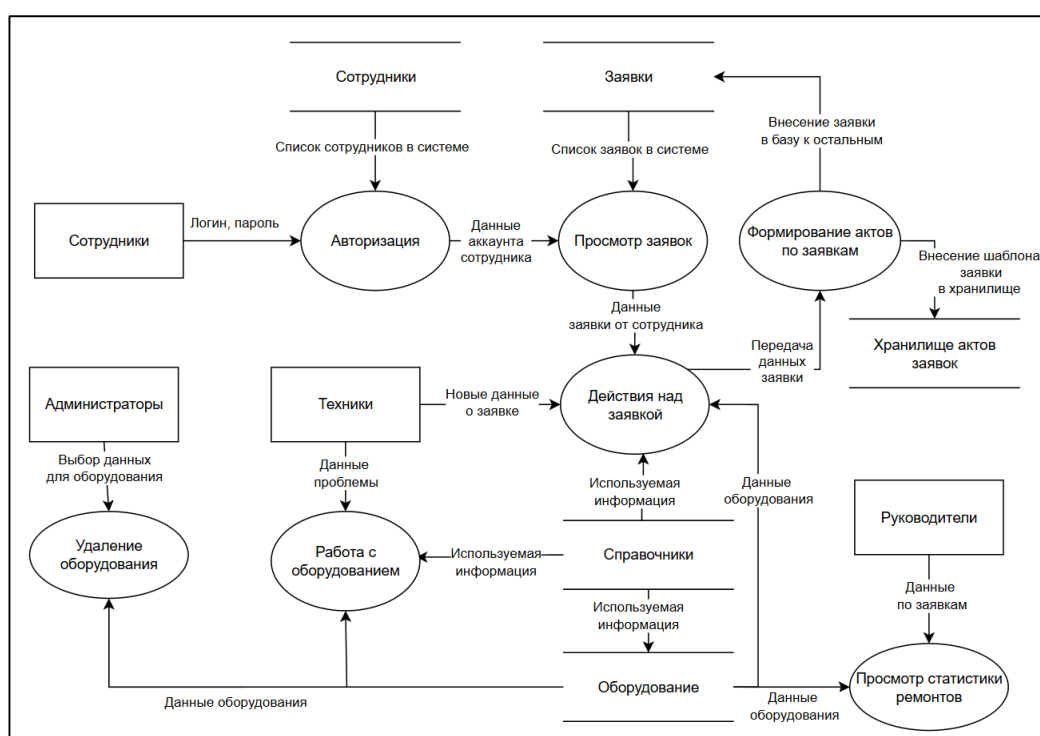


Рисунок 2.2 – DFD диаграмма Йордана-Де Марко после автоматизации процесса «принятие и обработка заявки на ремонт техники»

DFD диаграмма отражает архитектуру автоматизированной системы учёта заявок после внедрения программного решения. Взаимодействие начинается с процесса «Авторизация», куда сотрудники передают логин и пароль, после чего система извлекает данные аккаунта из хранилища «Сотрудники» и предоставляет доступ к просмотру списка заявок. Сформированные обращения передаются в процесс «Действия над заявкой», где техники направляют новые данные, работают с оборудованием, используя информацию из справочников и базы техники, а также инициируют «Формирование актов по заявкам». Готовые шаблоны актов и данные обращений автоматически сохраняются в «Хранилище

актов заявок» и обновляются в базе «Заявки». Администраторы управляют реестром техники через процессы «Работа с оборудованием» и «Удаление оборудования», корректируя справочную информацию. Руководители взаимодействуют с модулем «Просмотр статистики ремонтов», куда передаются данные по заявкам и состоянию оборудования для формирования аналитических отчетов. Диаграмма демонстрирует четкое разделение ролей, централизацию данных и автоматический обмен потоками информации между функциональными блоками, что исключает ручной ввод, минимизирует риски потери данных и обеспечивает прозрачный контроль на всех этапах жизненного цикла обращения [8, 17].

Выделяя основную задачу, решение которой в первую очередь требуется решить: учет и обработка заявок на ремонт техники. Была разработана диаграмма IDEF0-0, показывающая входящие элементы, элементы управления, механизм и что будет получено на выходе. Посмотреть диаграмму можно в соответствии с рисунком 2.3.



Рисунок 2.3 – IDEF0-0 учет и обработка заявки на ремонт техники

Представленная диаграмма IDEF0-0 отражает функциональную модель процесса «Учет и обработка заявок на ремонт техники» в едином контексте. Процесс регламентируется политиками информационной безопасности и стандартом ГОСТ-Р-2.601-2019 ЕСКД, которые задают требования к документированию, защите данных и порядку формирования технической документации. На вход системы поступают справочные сведения об оборудовании, персональные данные сотрудников-заявителей, а также актуальная информация о наличии запасных частей на складе. Функционирование процесса обеспечивается за счет взаимодействия технических специалистов с внедренным

программным решением автоматизации учета. В результате работы системы формируется зарегистрированная заявка с уникальным идентификатором, а также генерируются печатные формы актов о создании обращения и о завершении ремонтных работ, что обеспечивает полный документальный цикл фиксации, контроля и закрытия заявки.

Также была разработана диаграмма IDEF0-1. Посмотреть диаграмму можно в соответствии с рисунком 2.4.



Рисунок 2.4 – IDEF0-1 учет и обработка заявки на ремонт техники

Представленная диаграмма IDEF0-1 детализирует процесс учета и обработки заявок на ремонт техники, декомпозируя его на четыре последовательных функциональных блока. Процесс начинается с модуля «Заполнение формы заявки», куда в качестве входных потоков поступают сведения об оборудовании, персональные данные сотрудника-заявителя и актуальная информация о наличии комплектующих со склада, при этом этап регулируется политиками информационной безопасности. Результатом работы блока становится заполненная форма заявки, которая передается в модуль «Проверка склада запасных частей» для анализа доступности необходимых ресурсов и инструментов. На выходе из данного этапа формируется заявка с данными сотрудника, содержащая верифицированную информацию и готовая к передаче исполнителю.

Далее документ поступает в блок «Обработка заявки», где технические специалисты, опираясь на программное решение автоматизации учета, выполняют непосредственные ремонтные или наладочные работы в строгом соответствии со стандартом ГОСТ-Р-2.601-2019 ЕСКД. Сформированная составленная заявка направляется на финальный этап — «Подтверждение и печать заявки», где система регистрирует обращение в базе данных, присваивает ему уникальный номер и генерирует печатные формы актов о создании обращения и об окончании ремонта. Таким образом, диаграмма наглядно демонстрирует сквозной поток данных, где каждый функциональный блок

использует результаты предыдущего, обеспечивая прозрачность, контроль статусов и полное документальное сопровождение жизненного цикла заявки от регистрации до закрытия [2].

2.2 Проектирование модульной структуры

При проектировании модульной структуры, на основе предыдущих диаграмм была создана диаграмма вариантов использования. Посмотреть диаграмму вариантов использования можно в соответствии с рисунком 2.5.

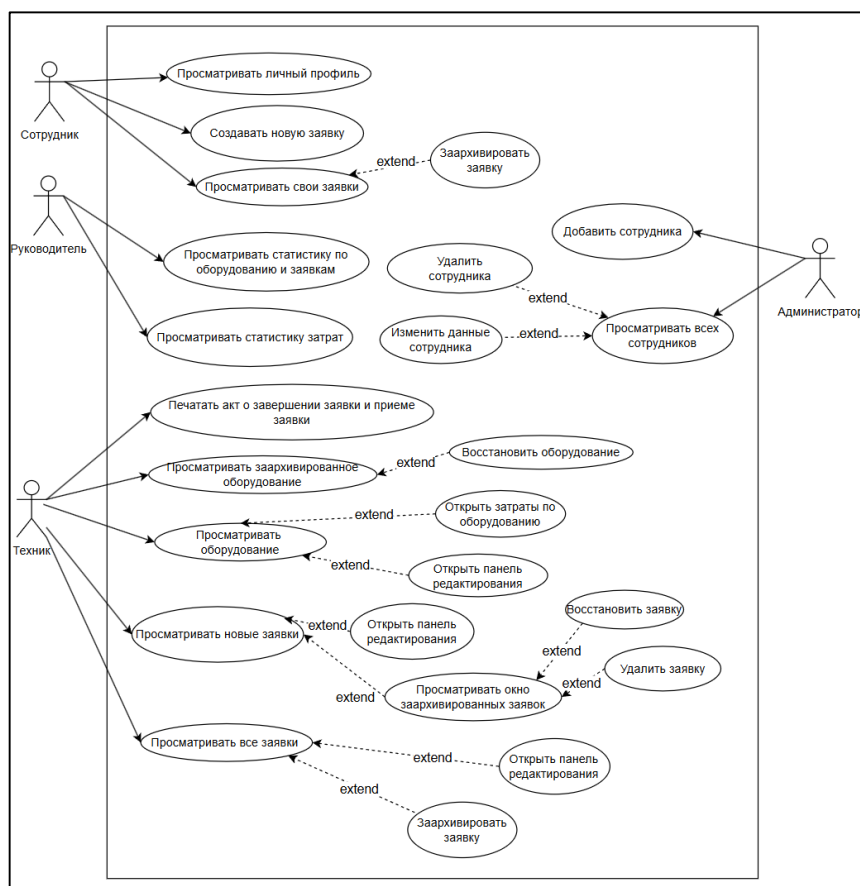


Рисунок 2.5 – Диаграмма вариантов использования

Представленная диаграмма вариантов использования визуализирует функциональную архитектуру системы и определяет сценарии взаимодействия пяти ключевых ролей с разрабатываемым приложением. В рамках границ системы выделены акторы: сотрудник, руководитель, техник и администратор, каждый из которых наделён набором функций. Сотрудник инициирует обращения и отслеживает их выполнение, руководитель получает аналитические отчёты по статистике отказов техники и финансовым затратам, а техник управляет полным жизненным циклом заявок:

редактирование, архивацию и генерацию печатных актов. Администратор обеспечивает учётную политику и управление кадровыми данными. Связи отражают обязательные вложенные сценарии и опциональные расширения функционала, такие как восстановление удалённых записей, корректировка данных оборудования и работа с архивом, что обеспечивает гибкость системы, прозрачность бизнес-процессов и надёжное разграничение прав доступа на всех этапах эксплуатации [5,14].

Далее рассмотрен процесс создания и обработки заявки при помощи диаграммы деятельности, представленной в соответствии с рисунком 2.6.

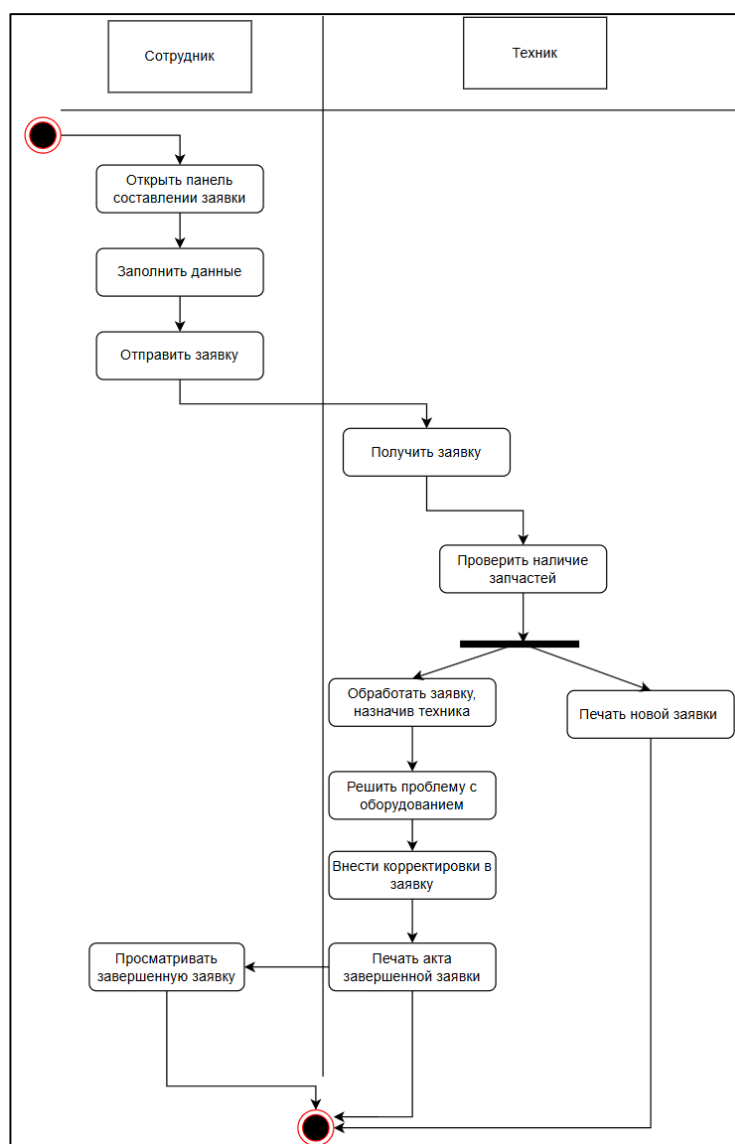


Рисунок 2.6 – Диаграмма деятельности «создание и обработка заявки»

Диаграмма деятельности описывает процесс «создание и обработка заявки», который начинается с дорожки «Сотрудник». Сотрудник открывает панель составления заявки, заполняет необходимые данные и отправляет заявку. Поток управления переходит

к «Технику», получает заявку и проверяет наличие запчастей. После проверки происходит разветвление на два параллельных процесса: одна ветвь предполагает обработку заявки, назначение техника, решение проблемы с оборудованием, внесение корректировок в заявку и печать акта о завершении заявки; вторая ветвь ведет к печати новой заявки и сразу завершает процесс. В случае успешного выполнения работ, после печати акта, поток возвращается к Сотруднику, который просматривает завершённую заявку, после чего процесс полностью завершается [20].

Также была составлена диаграмма состояний, показывающая жизненный цикл заявки на ремонт оборудования с момента создания, до полного удаления из системы учета заявок. Посмотреть диаграмму состояний можно в соответствии с рисунком 2.7.

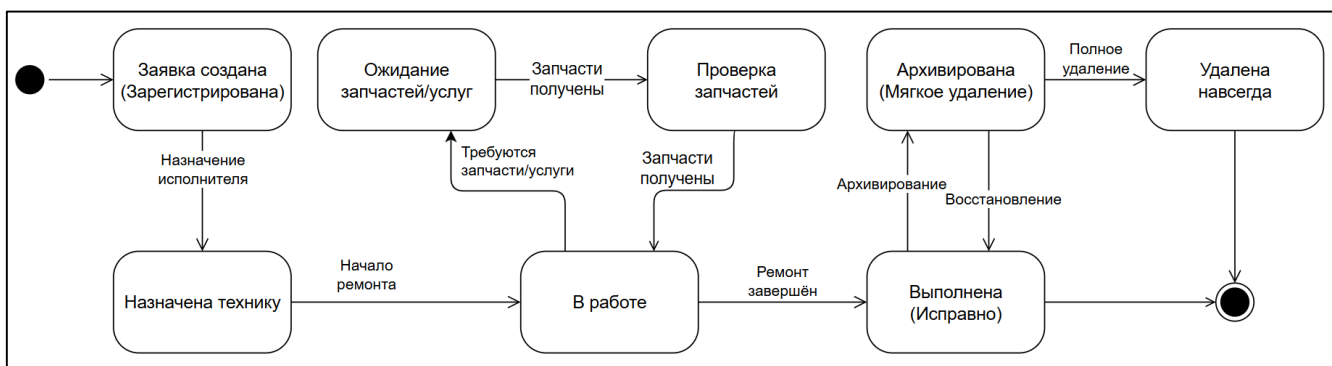


Рисунок 2.7 – Диаграмма состояний жизненного цикла заявки на ремонт техники

Диаграмма состояний жизненного цикла заявки на ремонт техники отражает последовательное изменение статуса обращения от момента регистрации до полного удаления из системы. Процесс начинается с состояния «Заявка создана (Зарегистрирована)», после чего происходит назначение исполнителя и переход в состояние «Назначена технику». При начале работ заявка переходит в статус «В работе», где может потребоваться ожидание или получение запчастей, услуг - при их наличии процесс продолжается, иначе возвращается к ожиданию [3].

По завершении ремонта заявка переходит в состояние «Выполнена (Исправно)», откуда возможна архивация («Мягкое удаление») или восстановление. Архивированная заявка может быть либо полностью удалена навсегда, либо восстановлена для повторной обработки.

Диаграмма также предусматривает прямое удаление заявки без прохождения через этап выполнения, что соответствует сценариям ошибочного создания или отмены обращения. Все переходы между состояниями строго регламентированы бизнес-правилами системы и обеспечивают полный контроль над жизненным циклом каждой заявки.

Для более детального показа как именно проходит процесс печати акта, была составлена диаграмма последовательности, просмотреть диаграмму можно в соответствии с рисунком 2.8.

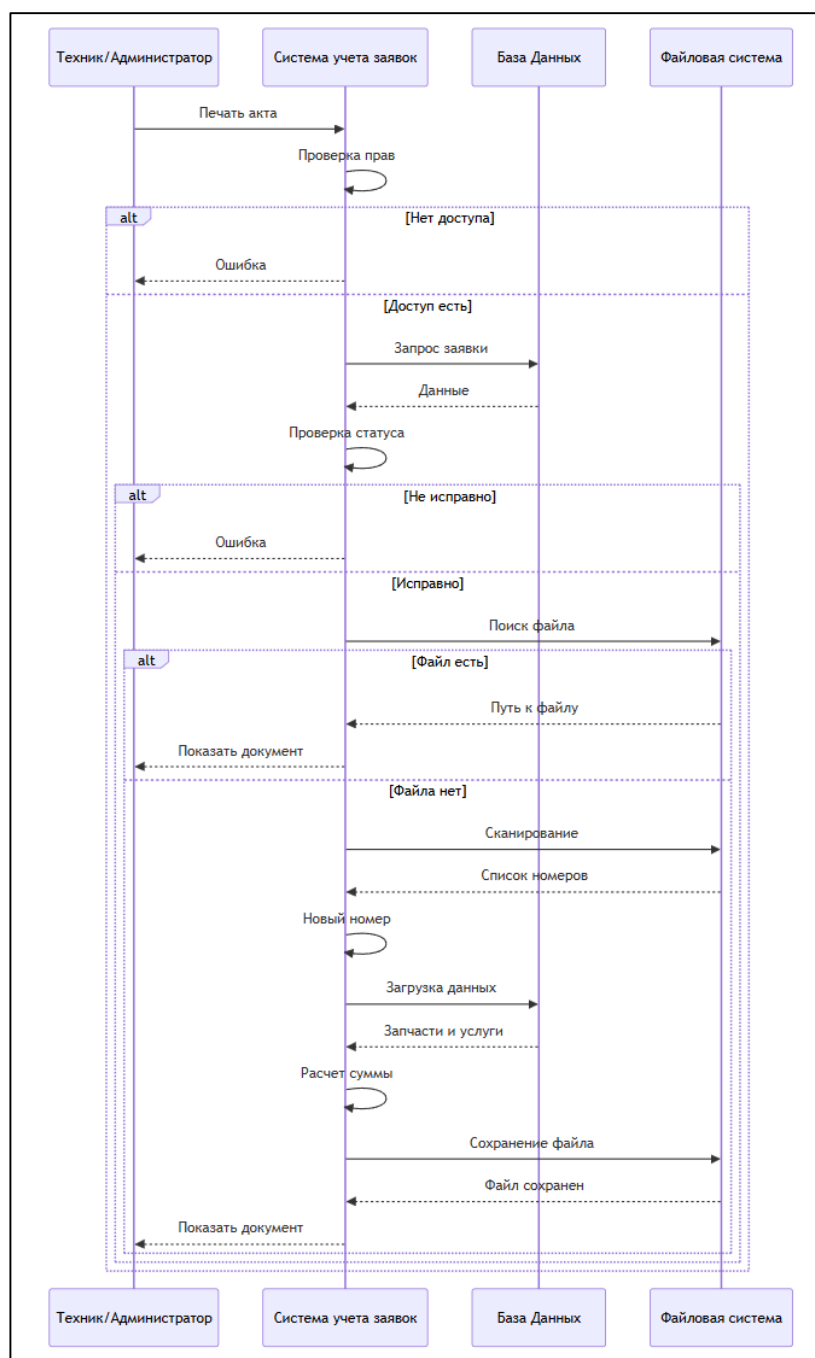


Рисунок 2.8 – Диаграмма последовательности «печать акта выполнения заявки»

Также для наглядного просмотра работы автоматизированного процесса была составлена диаграмма последовательности «создание и обработка заявки», отображающая работу автоматизированного процесса создания и обработки заявки. Посмотреть диаграмму последовательности можно в соответствии с рисунком 2.9.

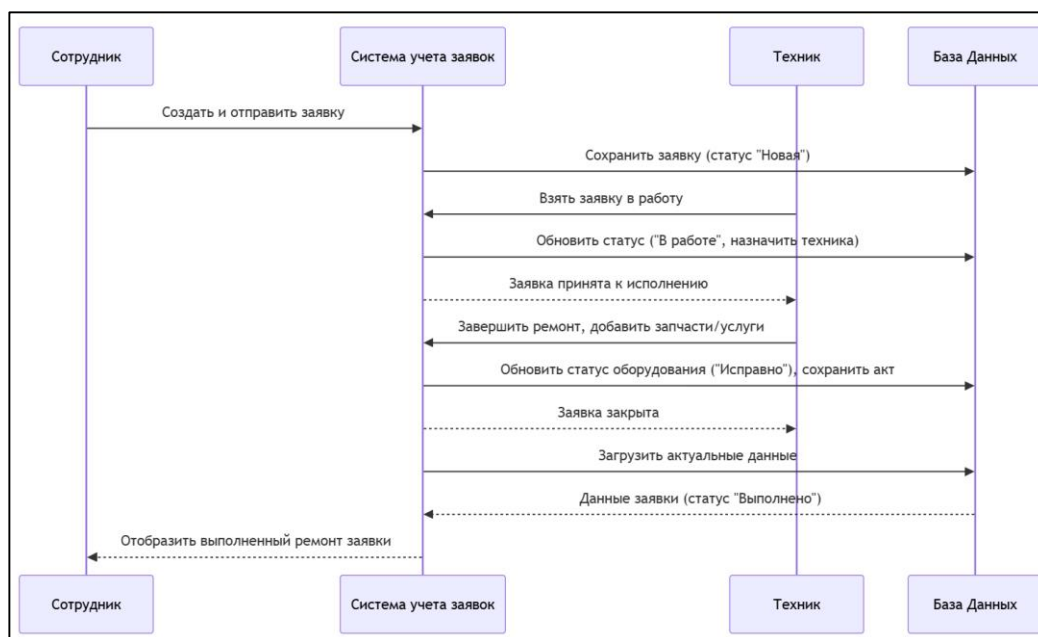


Рисунок 2.9 – Диаграмма последовательности «создание обработка заявки»

2.3 Проектирование структуры базы данных приложения

В ходе проектирования структуры базы данных приложения была разработана ER-диаграмма, состоящая из 19 сущностей:

Должности имеют атрибуты: «ID» и «Наименование». «ID» — первичный ключ. Должность может быть у одного или нескольких сотрудников, при этом обязательно есть.

Отделы имеют атрибуты: «ID» и «Наименование». «ID» — первичный ключ. Отдел может включать как одного, так и нескольких сотрудников, при этом обязательно привязан к сотрудникам.

Роли имеют атрибуты: «ID» и «Наименование». «ID» — первичный ключ. Роль может быть, как у одного, так и у нескольких сотрудников и обязательно есть.

Корпуса имеют атрибуты «ID» и «Наименование». «ID» — первичный ключ. Корпус может включать как один, так и несколько кабинетов, при этом обязательно присутствует.

Кабинеты имеют атрибуты: «ID» — первичный ключ, «Номер» и «Корпус» — внешний ключ. Кабинету соответствует только один корпус и обязательно соответствует. В кабинете может быть закреплено одно оборудование, несколько или вообще не быть оборудования. В кабинете может быть несколько сотрудников.

Сотрудники имеют атрибуты: «ID» — первичный ключ, «Номер телефона» «Почта», «Пароль», «Фамилия», «Имя», «Отчество», «Должность» — внешний ключ, «Отдел» — внешний ключ, «Кабинет» — внешний ключ, «Роль» — внешний ключ. Сотруднику

соответствует только одна должность, один отдел, один кабинет и одна роль, при этом обязательно соответствует. Сотрудник может быть, как заказчиком, так и назначенным исполнителем в одной или нескольких заявках. Посмотреть первую часть диаграммы можно в соответствии с рисунком 2.10 [7].

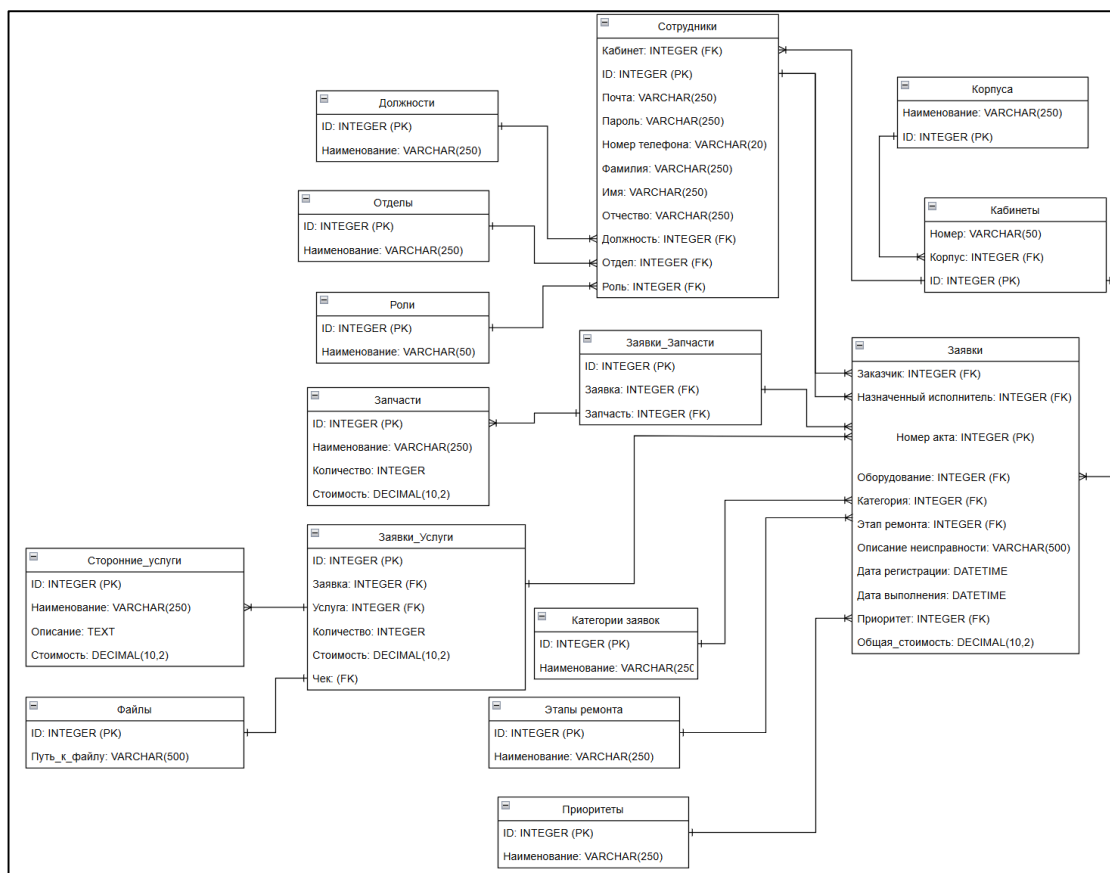


Рисунок 2.10 – ER-диаграмма часть с сотрудниками и параметрами заявки

Производители имеют атрибуты: «ID» и «Наименование». «ID» — первичный ключ. Производитель может быть, как у одной, так и у нескольких моделей оборудования и обязательно есть.

Модели оборудования имеют атрибуты: «ID» — первичный ключ, «Наименование» и «Производитель» — внешний ключ. Модели соответствует только один производитель и обязательно соответствует. Модель может быть, как у одного, так и у нескольких оборудований.

Типы оборудования имеют атрибуты: «ID» и «Наименование». «ID» — первичный ключ. Тип оборудования может быть, как у одного, так и у нескольких оборудований и обязательно есть.

Статусы оборудования имеют атрибуты: «ID» и «Наименование». «ID» — первичный ключ. Статус может быть, как у одного, так и у нескольких оборудования и обязательно есть.

Фотографии имеют атрибуты: «ID» и «Наименование». «ID» — первичный ключ. Фотография может быть, как у одного, так и у нескольких оборудования и обязательно есть.

Оборудование имеет атрибуты: «Закрепленный кабинет» — внешний ключ, «Инвентарный номер» — первичный ключ, «Модель» — внешний ключ, «Тип» — внешний ключ, «Статус» — внешний ключ, «Фото» — внешний ключ, «Комплектация», «Гарантия» — внешний ключ, «Дата приобретения». Оборудованию соответствует только один кабинет, одна модель, один тип, один статус, одно фото и обязательно соответствует. Оборудование может быть в одной или нескольких заявках.

Гарантии имеют атрибуты: «ID» — первичный ключ, «Дата начала», «Дата окончания». Гарантия есть у каждого оборудования и у каждого она своя.

Категории заявок имеют атрибуты: «ID» и «Наименование». «ID» — первичный ключ. Категория может быть, как у одной, так и у нескольких заявок и обязательно есть.

Этапы ремонта имеют атрибуты: «ID» и «Наименование». «ID» — первичный ключ. Этап ремонта может быть, как у одной, так и у нескольких заявок и обязательно есть.

Приоритеты имеют атрибуты: «ID» и «Наименование». «ID» — первичный ключ. Приоритет может быть, как у одной, так и у нескольких заявок и обязательно есть.

Запчасти имеют атрибуты: «ID» — первичный ключ, «Наименование», «Количество», «Стоимость». Запчасть может, быть как в одной, так и в нескольких заявках.

Заявки_Запчасти имеют атрибуты: «ID» — первичный ключ, «Заявка» — внешний ключ, «Запчасть» — внешний ключ. Связывает заявки и запчасти в соотношении многие-ко-многим.

Заявки имеют атрибуты: «Заказчик» — внешний ключ, «Назначенный исполнитель» — внешний ключ, «Номер акта» — первичный ключ, «Оборудование» — внешний ключ, «Категория» — внешний ключ, «Этап ремонта» — внешний ключ, «Описание неисправности», «Дата регистрации», «Дата выполнения», «Приоритет» — внешний ключ. Заявке соответствует только один заказчик, только один исполнитель, только одно оборудование, только одна категория, только один этап ремонта, только один приоритет и обязательно соответствует. Посмотреть часть, содержащую сущности оборудования, кабинетов и заявки созданной ER-диаграммы, можно в соответствии с рисунком 2.11.

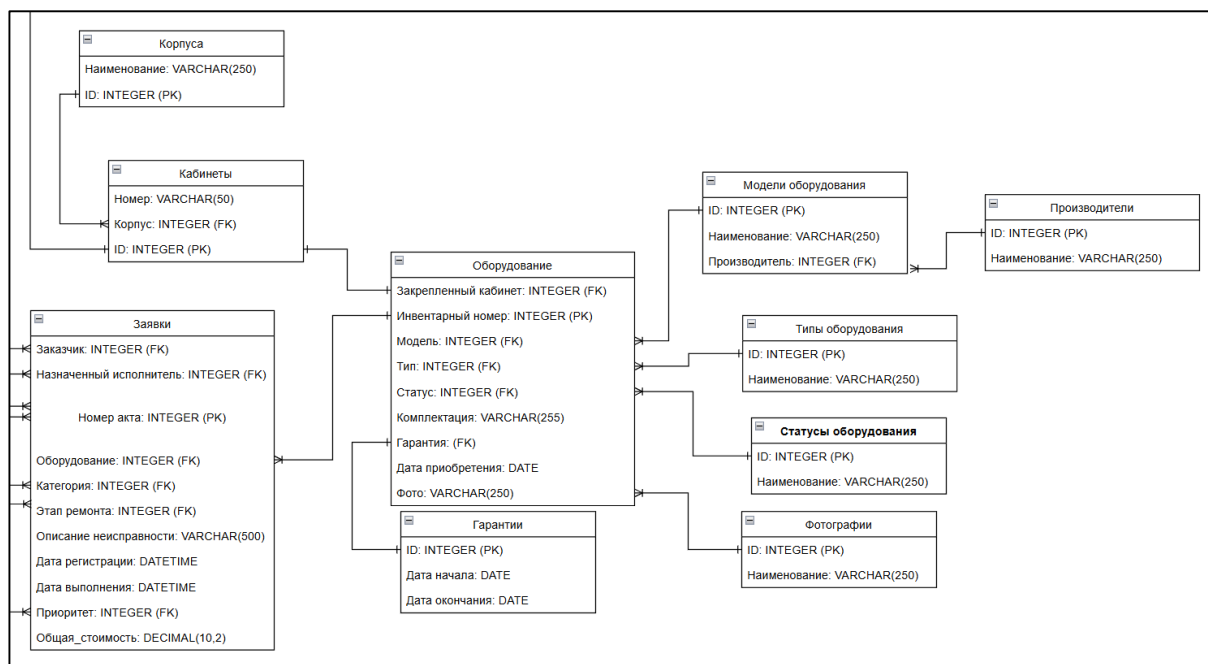


Рисунок 2.11 – ER-диаграмма часть с оборудованием и заявкой

Представленная ER-диаграмма отражает логическую структуру базы данных и обеспечивает целостность информационных потоков системы учёта заявок на ремонт оборудования, поддерживая корректное взаимодействие между всеми ролями пользователей.

Особое внимание в данной структуре уделено детальным взаимосвязям между оборудованием и заявками, где каждая единица техники имеет полную неизменяемую историю всех обслуживаний, подкреплённую актуальными данными о моделях, производителях, типах и гарантийных обязательствах. Структура первичных и внешних ключей спроектирована таким образом, что исключает возможность появления пустых записей и поддерживает корректное каскадное обновление данных. Это значительно упрощает техническое администрирование системы в долгосрочной перспективе развития проекта.

Достаточность данной диаграммы для нормального функционирования подтверждается наличием всех необходимых атрибутов для реализации сложной бизнес-логики. Хранение файлов и фотографий в отдельной таблице с привязкой к конкретным объектам системы обеспечивает эффективное управление медиа контентом без перегрузки основных таблиц заявлений и оборудования лишними данными. Посмотреть структуру базы данных можно в приложении Б.

Для полного представления, была сформирована физическая ER-диаграмма, используемая приложением для функционирования в полной мере и реализации всех

возможностей, заказанных заказчиком. Просмотреть физическую ER-диаграмму можно в соответствии с рисунком 2.12.

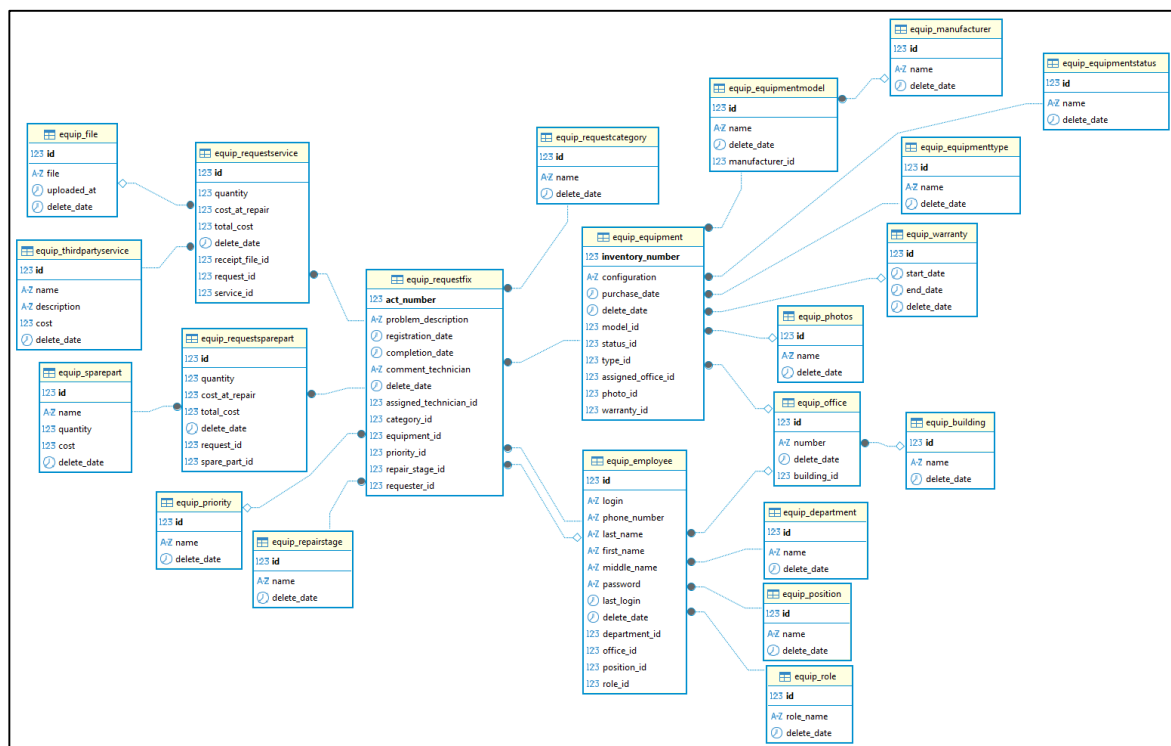


Рисунок 2.12 – Физическая ER-диаграмма

Вывод: в ходе анализа и проектирования была детально изучена предметная область учета заявок на ремонт техники в ГБУЗ РХ «АМКБ». Из DFD-диаграмм выявлены ключевые проблемы существующего процесса. Разработанная модель автоматизированного процесса демонстрирует переход к централизованному учету с четким разграничением ролей. Функциональное моделирование методом IDEF0 детализировало входные и выходные потоки данных. Проектирование модульной структуры с применением UML-нотации обеспечило формализацию сценариев взаимодействия акторов, описание алгоритмов обработки заявок и моделирование жизненного цикла. Спроектированная ER-диаграмма базы данных из 19 сущностей с корректно определенными первичными и внешними ключами, обеспечивает целостность данных, поддержку сложных связей между оборудованием и заявками, а также возможность формирования аналитической отчетности. Созданная проектная документация формирует надежную методологическую базу для перехода к практической реализации программного средства.

3 РАЗРАБОТКА И ИНТЕГРАЦИЯ ПРИЛОЖЕНИЯ

3.1 Выбор средств разработки приложения

Для разработки веб-приложения была выбрана интегрированная среда разработки PyCharm Community Edition, поскольку она предоставляет специализированную поддержку фреймворка Django, включая подсветку синтаксиса шаблонов и возможность запуска сервера отладки непосредственно из интерфейса, что критически важно для оперативного тестирования функционала учета заявок и оборудования без необходимости использования сторонних плагинов. Бесплатная версия полностью покрывает потребности проекта, так как не требует инструментов для командной разработки уровня Enterprise позволяя сосредоточиться на написании бизнес-логики обработки заявок и редактирования оборудования [10].

Основой серверной части выступает фреймворк Django, который был выбран благодаря своей архитектуре MTV и наличию встроенной административной панели, что позволяет администраторам системы управлять сотрудниками и справочниками без написания дополнительного кода. Мощная система ORM фреймворка упрощает работу с базой данных, позволяя описывать сложные связи между заявками оборудованием и запчастями на уровне моделей Python, что обеспечивает безопасность запросов и защиту от SQL-инъекций. Механизмы аутентификации и авторизации идеально подходят для реализации ролевой модели доступа сотрудников техников и руководителей что видно в функциях проверки прав доступа в `views.py` [16].

В качестве системы управления базами данных используется PostgreSQL обладающая высокой надежностью и поддержкой сложных SQL-запросов необходимых для формирования статистических отчетов по затратам и поломкам оборудования. Эта СУБД отлично интегрируется с Django через ORM и поддерживает транзакции, что гарантирует целостность данных при одновременном создании заявок и списании запчастей со склада. Открытость и отсутствие лицензионных отчислений делают ее оптимальным выбором для бюджетных проектов, а масштабируемость PostgreSQL позволит системе расти вместе с увеличением количества регистрируемого оборудования и пользователей обеспечивая стабильную работу при высокой нагрузке на таблицу заявок.

Для обеспечения связи между приложением на Python и базой данных PostgreSQL применяется библиотека `psycopg-binary` являющаяся адаптером с предустановленными бинарными файлами, избавляя разработчика от необходимости компилировать драйвер вручную и устраняет проблемы совместимости зависимостей в разных операционных

системах, а использование бинарной версии ускоряет процесс развертывания проекта на серверах и локальных машинах разработчиков, обеспечивая стабильное и быстрое соединение для выполнения запросов через ORM Django, что особенно важно при генерации печатных форм актов, где требуется быстрая выборка связанных данных об оборудовании и услугах [19].

Для визуального управления базой данных и проверки целостности связей используется клиент DBeaver который поддерживает работу с PostgreSQL и позволяет просматривать таблицы в виде ER-диаграмм, что удобно для анализа структуры данных и связей между таблицами заявок, использованных услуг. Инструмент предоставляет удобный интерфейс для выполнения ручных SQL-запросов при отладке сложной логики подсчета затрат и позволяет экспортировать данные в различные форматы для внешней отчетности. Бесплатная версия сообщества обладает достаточным функционалом для администрирования базы данных проекта, позволяя разработчикам и администраторам контролировать состояние данных без написания консольных команд [15].

3.2 Разработка приложения

В процессе разработки серверной части приложения был выделен набор ключевых функций, реализующих основную бизнес-логику системы учёта заявок. Для детального рассмотрения в данном подразделе отобраны две функции: создание заявки и ее печать. Полный код приложения по учету заявок представлен в приложении В.

Функция создания заявки является центральным элементом бизнес-логики приложения, поскольку именно с неё начинается жизненный цикл любого обращения о неисправности. Функция обрабатывает POST-запрос от авторизованного сотрудника, выполняет валидацию обязательных полей, таких как: описание проблемы, выбранное оборудование, генерирует уникальный номер акта путём автоматического инкремента последнего значения в базе данных и создаёт запись в модели RequestFix с привязкой к заказчику, оборудованию, категории и приоритету. Особое внимание уделено защите от дублирования: с помощью флага `form_open` в сессии предотвращается повторная отправка формы при обновлении страницы. Данная функция критически важна, так как обеспечивает стандартизированный, защищённый и отслеживаемый процесс регистрации обращений, исключая потерю заявок, характерную для ручного приёма через телефон или мессенджеры, и формирует основу для всей последующей обработки, статистики и отчётности системы. Просмотреть код функции можно в листинге 1 [6].

Листинг 1 – Create_request_customer

#Функция обрабатывает процесс создания новой заявки на ремонт оборудования авторизованными пользователями с проверкой их прав доступа.

#Метод валидирует входные данные формы, генерирует уникальный номер акта на основе последней записи и сохраняет обращение в базе данных.

#Дополнительно реализуется механизм защиты от повторной отправки формы через сессионный флаг для обеспечения целостности транзакций.

```
def create_request_customer(request):
    #Проверка наличия идентификатора пользователя в сессии для
    #подтверждения авторизации
    if 'user_id' not in request.session:
        #Вывод сообщения об ошибке через встроенную систему
        #уведомлений Django
        messages.error(request, 'Необходимо авторизоваться')
        #Перенаправление неавторизованного пользователя на
        #страницу входа в систему
        return redirect('login_view')
    #Проверка соответствия роли пользователя списку разрешённых
    #для создания заявок
    if request.session.get('user_role') not in ['Сотрудник',
        'Администратор', 'Руководитель', 'Техник']:
        #Вывод сообщения о запрете доступа при отсутствии
        #необходимой роли
        messages.error(request, 'Нет доступа')
        #Перенаправление пользователя на страницу входа при
        #нарушении политики доступа
        return redirect('login_view')
    try:
        #Получение объекта активного сотрудника из базы данных
        #по идентификатору из сессии, исключая удалённые записи
        one_employee = Employee.objects.get(id=request.session['user_id'],
        delete_date__isnull=True)
        #Получение списка оборудования, закреплённого за
        #кабинетом сотрудника, с оптимизацией запросов через select_related
        equipment_list = Equipment.objects.filter(assigned_office=one_employee.office,
        delete_date__isnull=True).select_related('model', 'type',
        'status')
        #Сброс сессионного флага защиты формы при получении
        #GET-запроса для подготовки к новому созданию
        if request.method == 'GET':
            request.session['form_open'] = False
        #Обработка HTTP-запроса методом POST для получения
        #данных формы создания заявки
        if request.method == 'POST':
            #Проверка сессионного флага для предотвращения
            #повторной отправки формы при обновлении страницы
            if request.session.get('form_open', False):
```

Продолжение листинга 1

```
#Уведомление пользователя о невозможности
повторной отправки без перезагрузки
messages.error(request, 'Форма уже открыта.
Обновите страницу.')
#Перенаправление на страницу создания заявки
для сброса состояния формы
return redirect('create_request_customer')
try:
    #Установка сессионного флага в состояние
активной обработки формы
    request.session['form_open'] = True
    #Извлечение инвентарного номера выбранного
оборудования из POST-данных
    equipment_inv_num =
request.POST.get('equipment')
    #Извлечение и очистка текстового описания
неисправности от лишних пробелов
    description =
request.POST.get('problem_description', '').strip()
    #Извлечение идентификатора категории заявки из
данных формы
    category_id = request.POST.get('category')
    #Извлечение идентификатора приоритета
выполнения заявки из данных формы
    priority_id = request.POST.get('priority')
    #Валидация обязательных полей на предмет их
наличия и непустоты
    if not description or not equipment_inv_num:
        #Вывод сообщения об ошибке валидации
пользователю
        messages.error(request, 'Заполните
обязательные поля')
        #Сброс флага защиты формы при возникновении
ошибки валидации
        request.session['form_open'] = False
        #Перенаправление пользователя обратно на
страницу формы для коррекции данных
        return
    redirect('create_request_customer')
    #Получение объекта оборудования по
инвентарному номеру, исключая удалённые записи
    equipment =
Equipment.objects.get(inventory_number=equipment_inv_num,
delete_date__isnull=True)
    #Получение объекта категории заявки по
идентификатору или присвоение None при его отсутствии
    category =
RequestCategory.objects.get(id=category_id) if category_id else
None
    #Получение объекта приоритета заявки по
идентификатору или присвоение None при его отсутствии
```

Продолжение листинга 1

```
        priority =
Priority.objects.get(id=priority_id) if priority_id else None
        #Поиск последней созданной заявки в базе данных
для расчёта следующего номера акта
        last_req =
RequestFix.objects.filter(delete_date__isnull=True).order_by('-
act_number').first()
        #Генерация нового номера акта путём инкремента
последнего значения или установки 1 для первой записи
        new_act_number = (last_req.act_number + 1) if
last_req else 1
        #Инициализация нового экземпляра модели заявки
с заполнением извлечённых и сгенерированных данных
        req = RequestFix(
            act_number=new_act_number,
            problem_description=description,
            registration_date=timezone.now(),
            requester=one_employee,
            equipment=equipment,
            category=category,
            priority=priority,
            repair_stage=RepairStage.objects.first()
        )
        #Сохранение созданного экземпляра заявки в базе
данных через ORM
        req.save()
        #Сброс сессионного флага защиты формы после
успешного сохранения заявки
        request.session['form_open'] = False
        #Вывод сообщения об успешном создании обращения
пользователю
        messages.success(request, 'Заявка создана!')
        #Перенаправление пользователя на страницу
списка его личных заявок
        return redirect('show_customer')
    #Обработка исключений, возникших в процессе
сохранения или валидации данных
    except Exception as e:
        #Сброс сессионного флага защиты формы при
возникновении критической ошибки
        request.session['form_open'] = False
        #Вывод сообщения с текстом возникшего
исключения пользователю
        messages.error(request, f'Ошибка создания:
{str(e)}')
        #Перенаправление пользователя на страницу
формы для повторной попытки
        return redirect('create_request_customer')
    #Формирование контекста для передачи данных в HTML-
шаблон при GET-запросе или ошибке POST
    context = {
```

Продолжение листинга 1

```
        'employee': one_employee,
        'equipment_list': equipment_list,
        'categories':
RequestCategory.objects.filter(delete_date__isnull=True),
        'priorities':
Priority.objects.filter(delete_date__isnull=True),
        'user_name': request.session.get('user_name',
''),
    }
    #Рендеринг HTML-шаблона страницы создания заявки с
передачей сформированного контекста
    return render(request,
'create_request_customer.html', context)
    #Обработка исключений, возникших на уровне получения
сотрудника или оборудования
    except Exception as e:
        #Сброс сессионного флага защиты формы при возникновении
ошибки верхнего уровня
        request.session['form_open'] = False
        #Вывод сообщения об общей ошибке выполнения функции
пользователю
        messages.error(request, f'Ошибка: {str(e)}')
        #Перенаправление пользователя на страницу списка заявок
для предотвращения заикливания
        return redirect('show_customer')
```

Функция печати выполненной заявки реализует финальный этап документального сопровождения ремонтных работ - формирование юридически значимого акта о выполнении заявки. Функция проверяет, что заявка находится на этапе «Завершена», генерирует уникальный идентификатор файла на основе номера акта и даты выполнения, рендерит HTML-шаблон с подстановкой данных о заказчике, исполнителе, использованных запчастях и услугах с указанием их стоимости, а также сохраняет результат в защищённую директорию `media/request_forms/` для последующей архивации или печати. Механизм проверки на существующий файл предотвращает создание дубликатов документов. Эта функция является одной из наиболее значимых, поскольку обеспечивает соответствие процесса требованиям делопроизводства медицинского учреждения: каждый закрытый ремонт фиксируется в виде официального документа, который может быть использован для отчётности перед руководством, подтверждения гарантийных случаев или аудита расходов на техническое обслуживание. Просмотреть код функции можно в листинге 2.

Листинг 2 – Print_completed_request

```
#Функция формирует и сохраняет печатную форму акта о
выполненных ремонтных работах, проверяя статус заявки и права
доступа пользователя.
```

Продолжение листинга 2

#Метод анализирует файловую систему на наличие дубликатов документа, генерирует уникальный номер акта и рендерит HTML-шаблон с подстановкой данных.

#При успешном завершении файл сохраняется в защищённую директорию медиа-ресурсов, а пользователь получает возможность распечатать готовый документ.

```
def print_completed_request(request, request_id):
    #Проверка наличия идентификатора пользователя в сессии для
    подтверждения факта авторизации
    if 'user_id' not in request.session:
        #Вывод сообщения об ошибке доступа через встроенную
        систему уведомлений Django
        messages.error(request, 'Нет доступа')
        #Перенаправление неавторизованного пользователя на
        страницу входа в систему
        return redirect('login_view')
    #Извлечение роли текущего пользователя из данных сессии для
    последующей проверки прав
    user_role = request.session.get('user_role', '')
    #Верификация соответствия роли пользователя списку
    разрешённых для формирования актов
    if user_role not in ['Техник', 'Администратор']:
        #Вывод сообщения о запрете доступа при отсутствии
        необходимой роли в системе
        messages.error(request, 'Нет доступа')
        #Перенаправление пользователя на страницу входа при
        нарушении политики доступа
        return redirect('login_view')
    try:
        #Извлечение объекта заявки из базы данных по номеру
        акта с автоматической обработкой отсутствия записи
        req = get_object_or_404(RequestFix,
        act_number=request_id)
        #Проверка соответствия текущего этапа ремонта значению
        «Завершена» для допуска к печати
        if req.repair_stage.name != 'Завершена':
            #Уведомление пользователя о недоступности функции
            для незавершённых обращений
            messages.error(request, 'Печать доступна только
            для заявок с этапом ремонта "Завершена"')
            #Перенаправление на страницу списка заявок техника
            return redirect('show_technician')
        #Формирование абсолютного пути к директории хранения
        сформированных печатных форм актов
        forms_folder = os.path.join(settings.MEDIA_ROOT,
        'request_forms')
        #Создание целевой директории в файловой системе, если
        она ещё не существует
        os.makedirs(forms_folder, exist_ok=True)
```

Продолжение листинга 2

```
        #Преобразование даты выполнения заявки в строковый
        формат для использования в имени файла
        completion_date_str =
req.completion_date.strftime('%Y%m%d') if req.completion_date
else 'no_date'
        #Генерация уникального ключа для поиска существующего
        файла и предотвращения создания дубликатов
        duplicate_key =
f"completed_{req.act_number}_{completion_date_str}"
        #Инициализация переменной для хранения имени найденного
        существующего файла акта
        existing_file = None
        #Проверка физического существования директории
        хранения документов перед сканированием
        if os.path.exists(forms_folder):
            #Перебор всех файлов в целевой директории для
            поиска совпадения по ключу
            for filename in os.listdir(forms_folder):
                #Фильтрация файлов по префиксу и расширению
                HTML для выбора только актов выполнения
                if
filename.startswith('Акт_об_выполнении_Заявки_') and
filename.endswith('.html'):
                    #Сравнение содержимого имени файла с
                    сгенерированным уникальным ключом дубликата
                    if duplicate_key in filename:
                        #Сохранение имени найденного файла в
                        переменную для последующего использования
                        existing_file = filename
                        #Прерывание цикла поиска после первого
                        успешного совпадения для оптимизации
                        break
        #Проверка результата поиска на предмет нахождения ранее
        сгенерированного файла акта
        if existing_file:
            #Извлечение числового номера акта из имени
            существующего файла с помощью регулярного выражения
            match = re.search(r'_(\d+)\.html$',
existing_file)
            #Преобразование найденного номера в целое число или
            присвоение значения 1 при отсутствии совпадения
            act_number = int(match.group(1)) if match else 1
            #Формирование контекста для передачи данных в
            шаблон при использовании существующего файла
            context = {
                'request': req,
                'organization_name': 'ГБУЗ РХ «Абаканская
межрайонная клиническая больница»',
                'print_date':
timezone.now().strftime('%d.%m.%Y %H:%M'),
                'act_number': act_number,
```


Продолжение листинга 2

```
        'existing_file': True,
    }
    #Рендеринг HTML-шаблона страницы печати с передачей
контекста существующего документа
    return render(request,
'print_completed_request.html', context)
    #Инициализация переменной для хранения максимального
найденного номера акта при генерации нового
    max_act_number = 0
    #Повторная проверка существования директории для
сканирования файлов при создании нового акта
    if os.path.exists(forms_folder):
        #Перебор файлов в директории для определения
текущего максимального номера акта
        for filename in os.listdir(forms_folder):
            #Фильтрация файлов по префиксу и расширению для
выбора только актов выполнения
            if
filename.startswith('Акт_об_выполнении_Заявки_') and
filename.endswith('.html'):
                #Извлечение номера акта из имени файла с
помощью регулярного выражения
                match = re.search(r'_(\d+)\.html$',
filename)
                #Проверка успешности применения
регулярного выражения к имени файла
                if match:
                    #Преобразование извлечённой строки
номера в целое число
                    num = int(match.group(1))
                    #Сравнение текущего номера с
максимальным найденным значением
                    if num > max_act_number:
                        #Обновление переменной
максимального номера при нахождении большего значения
                        max_act_number = num
                    #Расчёт нового уникального номера акта путём инкремента
максимального найденного значения
                    new_act_number = max_act_number + 1
                    #Формирование нового контекста с уникальным номером
акта для генерации документа с нуля
                    context = {
                        'request': req,
                        'organization_name': 'ГБУЗ РХ «Абаканская
межрайонная клиническая больница»',
                        'act_number': new_act_number,
                        'existing_file': False,
                    }
                #Импортирование функции рендеринга шаблона в строку для
последующего сохранения в файл
                from django.template.loader import render_to_string
```

Продолжение листинга 2

```
        #Генерация HTML-кода акта на основе шаблона и
переданного контекста с данными заявки
        html_string =
render_to_string('print_completed_request.html', context)
        #Формирование уникального имени файла с новым номером,
номером заявки и датой выполнения
        filename =
f"Акт_об_выполнении_Заявки_{new_act_number}_{req.act_number}_{co
mpletion_date_str}.html"
        #Объединение пути к директории и сгенерированного имени
файла для получения полного пути сохранения
        file_path = os.path.join(forms_folder, filename)
        #Открытие файла в режиме записи с указанием кодировки
UTF-8 для корректного сохранения кириллицы
        with open(file_path, 'w', encoding='utf-8') as f:
            #Запись сгенерированной HTML-строки акта в
созданный файл на диске
            f.write(html_string)
        #Рендеринг шаблона страницы печати для отображения
только что созданного акта пользователю
        return render(request,
'print_completed_request.html', context)
        #Блок обработки исключений, возникающих при работе с базой
данных или файловой системой
        except Exception as e:
            #Вывод сообщения с текстом возникшего исключения
пользователю через систему уведомлений
            messages.error(request, f'Ошибка: {str(e)}')
            #Перенаправление пользователя на страницу списка заявок
в случае критической ошибки
            return redirect('show_technician')
```

3.3 Разработка пользовательского интерфейса

Пользовательский интерфейс был выполнен с использованием CSS, соблюдая интуитивно понятную расстановку и название элементов страниц. Цветовая палитра построена на градиентном фоне мятно-бирюзовых оттенков, что снижает зрительную нагрузку при длительной работе и соответствует стандартам медицинских информационных систем. Навигационные панели реализованы на базе Flexbox с автоматическим переносом элементов. Карточки, таблицы и формы оформлены с помощью мягких теней, скруглённых углов и контрастных заголовков, что обеспечивает визуальное разделение рабочих зон. Таблицы данных стилизованы чёткой сеткой с эффектом подсветки строки при наведении курсора, улучшая читаемость больших массивов информации. Система статусов и уведомлений реализована через цветовую индикацию с

акцентными левыми границами и бейджами, позволяя персоналу мгновенно идентифицировать критичность обращения. Код с сформированными стилями, использующиеся на всех страницах приложения представлен на листинге 3.

Листинг 3 – Стили страниц

```
<style>
  /* Основные стили и основа */
  body { font-family: 'Segoe UI', sans-serif; background:
linear-gradient(135deg, #e8f5e9 0%, #b2dfdb 100%); color: #263238;
padding: 20px; margin: 0; } /* Шрифт, градиент фона, сброс внешних
отступов */
  .container { max-width: 1400px; margin: 0 auto; } /*
Центрирование контента и ограничение ширины */

  /* Шапка */
  .header { background: white; padding: 20px; border-radius:
10px; margin-bottom: 20px; box-shadow: 0 2px 8px
rgba(0,0,0,0.1); } /* Белая карточка-шапка с мягкой тенью */
  h1 { color: #00695c; margin-top: 0; } /* Заголовок в
фирменном цвете, без верхнего отступа */

  /* Навигация */
  .nav-bar { margin-bottom: 20px; display: flex; gap: 10px;
flex-wrap: wrap; } /* Flex-контейнер меню с переносом кнопок */
  .nav-link { text-decoration: none; color: white;
background: #26a69a; padding: 8px 15px; border-radius: 5px; font-
weight: 600; } /* Базовая кнопка навигации */
  .nav-link:hover { background: #00897b; } /* Затемнение при
наведении */
  .nav-link.danger { background: #ef5350; } /* Кнопка
удаления/критических действий */

  /* Уведомления и карточки */
  .notification-box { background: white; padding: 15px;
border-radius: 8px; margin-bottom: 20px; box-shadow: 0 2px 5px
rgba(0,0,0,0.1); } /* Контейнер списка уведомлений (исправлен typo
order-radius → border-radius) */
  .notification-item { display: flex; align-items: center;
padding: 10px 15px; margin: 10px 0; border-radius: 5px; border-
left: 4px solid; } /* Строка уведомления с акцентной левой границей */
  .notification-count { background: white; color: #263238;
padding: 4px 12px; border-radius: 15px; font-weight: bold; margin-
right: 15px; min-width: 30px; text-align: center; } /* Бейдж-
счетчик */
  .notification-text { flex: 1; color: #263238; } /*
Растягивание текстового блока */
  .category-red { background: #F5C2C2; border-left-color:
#EB5A61; } /* Красная тема (критичные/ошибки) */
  .category-red .notification-count { background: #EB5A61;
color: white; } /* Инверсия цвета счетчика для красной темы */

```

Продолжение листинга 3

```
.category-purple { background: #f3e5f5; border-left-
color: #ab47bc; } /* Фиолетовая тема (информационные) */
.category-purple .notification-count { background:
#ab47bc; color: white; } /* Инверсия для фиолетовой */
.category-orange { background: #fff3e0; border-left-
color: #ff6f00; } /* Оранжевая тема (внимание/предупреждения) */
.category-orange .notification-count { background:
#ff6f00; color: white; } /* Инверсия для оранжевой */

/* Подсветка строк */
.row-red { background: #F5C2C2 !important; } /*
Принудительная покраска строки в красный */
.row-purple { background: #f3e5f5 !important; } /*
Принудительная покраска в фиолетовый */
.row-orange { background: #fff3e0 !important; } /*
Принудительная покраска в оранжевый */

/* Таблицы */
table { width: 100%; border-collapse: collapse;
background: white; border-radius: 8px; overflow: hidden; box-
shadow: 0 2px 5px rgba(0,0,0,0.1); } /* Карточный стиль таблицы,
скрытие углов */
th { background: #80cbc4; color: #004d40; padding: 12px;
text-align: left; font-weight: 600; } /* Заголовки столбцов в
мятном цвете */
td { padding: 10px; border-bottom: 1px solid #e0f2f1; } /*
Ячейки данных с разделителем */
tr:hover { background: #e0f7fa; } /* Легкая подсветка
строки при наведении курсора */

/* Кнопки и системные сообщения */
.btn-sm { padding: 4px 8px; font-size: 12px; text-
decoration: none; border-radius: 4px; color: white; background:
#EB5A61; border: none; cursor: pointer; } /* Компактная кнопка
действий */
.btn-sm:hover { background: #f57c00; } /* Эффект наведения
на кнопку */
.messages { color: #c62828; background: #ffebee; padding:
10px; margin-bottom: 15px; border-radius: 5px; } /* Блок flash-
сообщений Django (ошибки/успех) */
</style>
```

Посмотреть основные страницы, показывающие все использованные стили, можно в соответствии с рисунками 3.1 – 3.5.



ГБУЗ РХ АМКБ
Система учёта заявок

К заявкам

Новые заявки

Выйти

Редактирование заявки №2024012

Добро пожаловать, **Иванов Александр Александрович**

Заказчик: Волков Роман Романович

Дата регистрации: 10.04.2026 10:25

Основная информация

Описание проблемы *:

Компьютер зависает при работе с графическими приложениями. Возможна проблема с видеокартой.

Оборудование *:

10012 - Lenovo ThinkCentre

Исполнитель:

Иванов Александр

Статус оборудования:

Списано

Дата выполнения:

11 . 02 . 2026

Детали заявки

Категория:

Сетевая проблема

Приоритет:

Низкий

Этап ремонта:

Ожидание запчастей

Запчасти

Процессор Intel i5 (18000.00 Р)

18000.00

3

+ Добавить запчасть

Услуги

Установка ОС (2500.00 Р)

2500.00

1

Обзор... Файл... рач...

+ Добавить услугу

Сохранить изменения

Отмена

Рисунок 3.1 – Страница редактирования заявки техником

Печать

Заккрыть

ГБУЗ РХ «Абаканская межрайонная клиническая больница»

ЗАЯВКА №20260414

Дата регистрации:	10.04.2026
Оборудование:	Инв. № 10008, Siemens CT Scanner, МФУ
Заказчик:	Михайлов Алексей Алексеевич
Описание проблемы:	Операционная система загружается с ошибками, требуется переустановка.

Заказчик:

Принято в работу:

/ Михайлов.А.А.

/ Сидоров.С.С.

Документ сформирован автоматически системой учёта заявок

Рисунок 3.2 – Страница с формой печати новой заявки

Печать
Закреть

ГБУЗ РХ «Абаканская межрайонная клиническая больница»

АКТ ВЫПОЛНЕНИЯ ЗАЯВКИ №1

Дата подачи: 10.04.2026 | Дата выполнения: 15.01.2026

Оборудование:	Инв. № 10009, Dell PowerEdge, Медицинское оборудование, Dell
Заказчик:	Новиков Игорь Игоревич
Исполнитель:	Иванов Александр Александрович
Описание проблемы:	Wi-Fi адаптер не видит беспроводные сети. Требуется замена или настройка драйверов.

Использованные материалы и услуги:

Запчасти:

- Кабель HDMI — 3 шт. × 500.00 Р = 1500.00 Р
- Картридж Canon — 3 шт. × 1500.00 Р = 4500.00 Р

Услуги:

- Диагностика оборудования — 2 шт. × 1000.00 Р = 2000.00 Р

ИТОГО: 8000.00 Р

Заказчик:

_____ / Новиков.И.И.

Исполнитель:

_____ / Иванов.А.А.

Документ сформирован автоматически системой учёта заявок

Рисунок 3.3 – Страница с формой печати акта выполненной заявки

Значение статусов Красный — Критично (≥5 поломок или ≥10 заявок в месяц) Сиреневый — Внимание (3-4 поломки или 5-9 заявок в месяц) Оранжевый — Норма (<3 поломки или <5 заявок в месяц) Зелёный — Выполнено/Без проблем	
Затраты по типам оборудования	
Тип оборудования	Затраты (Р)
МФУ	20500.0 Р
Медицинское оборудование	23900.0 Р
Оргтехника	21400.0 Р
Сервер	14400.0 Р
Компьютер	58500.0 Р
Принтер	49700.0 Р
ИТОГО:	188400.0 Р

Рисунок 3.4 – Страница руководителя



ГБУЗ РХ АМКБ
Система учёта заявок

НазадОборудованиеВыйти

Добавление оборудования

Добро пожаловать, **Мальцева Анастасия Александровна**

Инвентарный номер (автоматически)*:

10018

Тип оборудования *:

-- Выберите тип --

Новый

Модель *:

-- Выберите модель --

Новая

Статус *:

-- Выберите статус --

Кабинет:

-- Не распределён --

Новый

Гарантия:

-- Без гарантии --

Новая

Фото оборудования:

Обзор... Файл не выбран.

Поддерживаемые форматы: JPG, PNG, GIF. Максимальный размер: 5MB

Комплектация:

Дата приобретения:

дд . мм . гggg

Создать оборудование

Новая модель оборудования

Название модели *:

Производитель:

-- Выберите или оставьте пустым --

ОтменаСоздать

Рисунок 3.5 – Страница создания оборудования

Вывод: в ходе разработки был реализовано полностью функциональное приложение, соответствующий техническому заданию. Использование фреймворка Django и СУБД PostgreSQL обеспечило безопасность данных и масштабируемость системы, а серверная бизнес-логика включает валидацию форм, автоматическую нумерацию актов и модуль автоматизированного документооборота. Интерфейс выполнен на CSS с ролевым разделением прав и адаптивной вёрсткой, а механизмы обработки исключений и оптимизации ORM-запросов гарантируют отказоустойчивость.

4 ОТЛАДКА И ОЦЕНКА КАЧЕСТВА ПРИЛОЖЕНИЯ

4.1 Разработка тестовых наборов и тестовых сценариев

Этап тестирования является неотъемлемой частью жизненного цикла разработки программного обеспечения, позволяющей убедиться в соответствии реализованного функционала первоначальным требованиям и спецификации. В рамках преддипломной практики были разработаны тест-кейсы, описывающие последовательность действий пользователя и ожидаемые результаты работы системы для различных сценариев использования. Тест-кейсы представлены в таблицах 1-6.

Таблица 1 – Тест-кейс №1

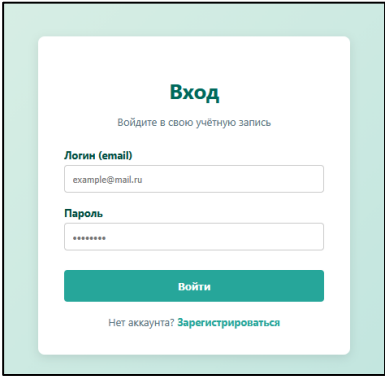
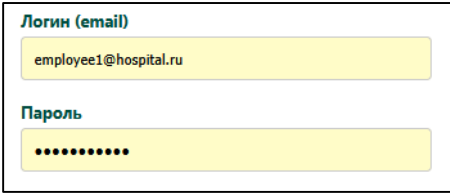
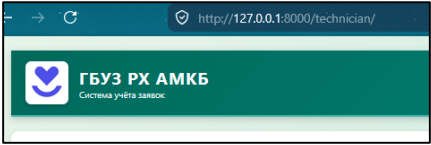
Номер	1
Название	Тестирование входа в аккаунт с ролью «Техник»
Предусловие	Запущен сервер разработки, база данных заполнена тестовыми данными
Шаг №1	Запустить программу через <code>python manage.py runserver</code> и перейти на страницу <code>http://127.0.0.1:8000/</code>
Результат шага №1	Отображение окна с наименованием «Вход в систему», поля для ввода с наименованиями «Логин» и «Пароль», кнопка «Войти» 
Шаг №2	Ввести логин <code>employee1@hospital.ru</code> и пароль <code>password123</code> с существующими данными в таблице, где пароль и логин будут соответствовать друг другу, нажать на кнопку «Войти» 
Результат шага №2	Перенаправление на страницу техника <code>http://127.0.0.1:8000/technician/</code> 
Поведение	Позитивное
Статус	Пройден

Таблица2 – Тест-кейс №2

Номер	2																											
Название	Тестирование навигации из техника на страницу новых заявок, далее на оборудование																											
Предусловие	Уже вошли под сотрудником с ролью «Техник»																											
Шаг №1	Кликнуть по кнопке на странице «Оборудование» Назад Новые заявки Профиль																											
Результат шага №1	Отображение страницы новых заявок со списком неотредактированных заявок Новые заявки Добро пожаловать, Иванов Александр Александрович Назад Новые заявки Все заявки Создать заявку Оборудование Сотрудник Выйти Статус новых заявок 3 Новые — нет исполнителя и нет даты выполнения 1 Требуют исполнителя — есть дата выполнения, но нет назначенного техника 4 В работе — есть назначенный техник, но нет даты завершения Сортировка: По дате (убыв.) <table><tr><th>№</th><th>Дата</th><th>Описание</th><th>Заказчик</th><th>Оборудование</th><th>Статус</th><th>Действия</th></tr><tr><td>2024022</td><td>27.04.2026</td><td>пухляпа</td><td>Смирнов Михаил</td><td>10005</td><td>Ожидает запчасти</td><td> Редактировать Печать </td></tr><tr><td>2024020</td><td>13.04.2026</td><td>неисправность конфигурации</td><td>Смирнов Михаил</td><td>10005</td><td>Ожидает запчасти</td><td> Редактировать Печать </td></tr></table>	№	Дата	Описание	Заказчик	Оборудование	Статус	Действия	2024022	27.04.2026	пухляпа	Смирнов Михаил	10005	Ожидает запчасти	Редактировать Печать	2024020	13.04.2026	неисправность конфигурации	Смирнов Михаил	10005	Ожидает запчасти	Редактировать Печать						
№	Дата	Описание	Заказчик	Оборудование	Статус	Действия																						
2024022	27.04.2026	пухляпа	Смирнов Михаил	10005	Ожидает запчасти	Редактировать Печать																						
2024020	13.04.2026	неисправность конфигурации	Смирнов Михаил	10005	Ожидает запчасти	Редактировать Печать																						
Шаг №2	Кликнуть по кнопке на странице «Оборудование» Создать заявку Оборудование Сотрудник																											
Результат шага №2	Отображение страницы оборудования с данными о всех оборудованях Оборудование Добро пожаловать, Иванов Александр Александрович Назад Добавить оборудование Удалённое Затраты Выйти Поиск: Название или номер Тип оборудования: Все типы Статус: Все статусы Сортировка: По инв. номеру (А-Я) Применить Сброс <table><tr><th>Фото</th><th>Инв. номер</th><th>Модель</th><th>Тип</th><th>Комплектация</th><th>Кабинет</th><th>Статус</th><th>Гарантия</th><th>Действия</th></tr><tr><td></td><td>10002</td><td>Lenovo ThinkCentre</td><td>МФУ</td><td>Конфигурация 2</td><td>103 (Главный корпус)</td><td>Списано</td><td>истекла 10.02.2024</td><td> Изменить Затраты История </td></tr><tr><td></td><td>10003</td><td>Canon i-SENSYS</td><td>Медицинское оборудование</td><td>Конфигурация 3</td><td>104 (Главный корпус)</td><td>В ремонте</td><td>истекла 01.03.2024</td><td> Изменить Затраты История </td></tr></table>	Фото	Инв. номер	Модель	Тип	Комплектация	Кабинет	Статус	Гарантия	Действия		10002	Lenovo ThinkCentre	МФУ	Конфигурация 2	103 (Главный корпус)	Списано	истекла 10.02.2024	Изменить Затраты История		10003	Canon i-SENSYS	Медицинское оборудование	Конфигурация 3	104 (Главный корпус)	В ремонте	истекла 01.03.2024	Изменить Затраты История
Фото	Инв. номер	Модель	Тип	Комплектация	Кабинет	Статус	Гарантия	Действия																				
	10002	Lenovo ThinkCentre	МФУ	Конфигурация 2	103 (Главный корпус)	Списано	истекла 10.02.2024	Изменить Затраты История																				
	10003	Canon i-SENSYS	Медицинское оборудование	Конфигурация 3	104 (Главный корпус)	В ремонте	истекла 01.03.2024	Изменить Затраты История																				
Поведение	Позитивное																											
Статус	Пройден																											

Таблица 3 – Тест-кейс №3

Номер	3																																																																																
Название	Тестирование удаления заявки																																																																																
Предусловие	Уже вошли под сотрудником с ролью «Техник», в системе существует минимум одна заявка																																																																																
Шаг №1	На странице техника нажать на кнопку «Удалить» рядом с заявкой (2024018) Приоритет Действия Средний Изменить Печать Удалить <table><tr><th></th><th>act_number</th><th>problem_description</th><th>registration_date</th><th>completion_date</th></tr><tr><td>1</td><td>2 024 002</td><td>Принтер зажевывает бумагу при печ</td><td>2026-04-10 17:25:35.743 +0700</td><td>[N]</td></tr><tr><td>2</td><td>2 024 003</td><td>Монитор мигает и периодически гас</td><td>2026-04-10 17:25:35.743 +0700</td><td>2025-11-22 17:25:35.742 +</td></tr><tr><td>3</td><td>2 024 004</td><td>Ноутбук сильно греется и шумит. Тр</td><td>2026-04-10 17:25:35.743 +0700</td><td>[N]</td></tr><tr><td>4</td><td>2 024 005</td><td>Клавиатура не реагирует на нажатия</td><td>2026-04-10 17:25:35.743 +0700</td><td>[N]</td></tr><tr><td>5</td><td>2 024 006</td><td>Сетевое подключение нестабильное,</td><td>2026-04-10 17:25:35.743 +0700</td><td>2025-12-27 17:25:35.742 +</td></tr><tr><td>6</td><td>2 024 007</td><td>Жесткий диск издает посторонние зв</td><td>2026-04-10 17:25:35.743 +0700</td><td>[N]</td></tr><tr><td>7</td><td>2 024 008</td><td>Операционная система загружается</td><td>2026-04-10 17:25:35.743 +0700</td><td>[N]</td></tr><tr><td>8</td><td>2 024 012</td><td>Компьютер зависает при работе с гр</td><td>2026-04-10 17:25:35.743 +0700</td><td>2026-02-11 17:25:35.742 +</td></tr><tr><td>9</td><td>2 024 014</td><td>МФУ не сканирует, только печатает. I</td><td>2026-04-10 17:25:35.743 +0700</td><td>2026-04-15 07:00:00.000 +</td></tr><tr><td>10</td><td>2 024 010</td><td>Блок бесперебойного питания не де</td><td>2026-04-10 17:25:35.743 +0700</td><td>2026-04-11 07:00:00.000 +</td></tr><tr><td>11</td><td>2 024 017</td><td>Тестирование от техника</td><td>2026-04-10 17:29:02.865 +0700</td><td>2026-04-10 17:28:00.000 +</td></tr><tr><td>12</td><td>2 024 013</td><td>Сканер не определяется системой. Т</td><td>2026-04-10 17:25:35.743 +0700</td><td>2026-04-11 07:00:00.000 +</td></tr><tr><td>13</td><td>2 024 009</td><td>Wi-Fi адаптер не видит беспроводны</td><td>2026-04-10 17:25:35.743 +0700</td><td>2026-01-15 17:25:00.000 +</td></tr><tr><td>14</td><td>2 024 018</td><td>Не работает подача красителя, в рем</td><td>2026-04-13 16:57:55.066 +0700</td><td>2026-04-24 16:57:00.000 +</td></tr><tr><td>15</td><td>2 024 016</td><td>тестирование создания от сотрудни</td><td>2026-04-10 17:26:18.533 +0700</td><td>2026-04-11 00:30:00.000 +</td></tr></table>		act_number	problem_description	registration_date	completion_date	1	2 024 002	Принтер зажевывает бумагу при печ	2026-04-10 17:25:35.743 +0700	[N]	2	2 024 003	Монитор мигает и периодически гас	2026-04-10 17:25:35.743 +0700	2025-11-22 17:25:35.742 +	3	2 024 004	Ноутбук сильно греется и шумит. Тр	2026-04-10 17:25:35.743 +0700	[N]	4	2 024 005	Клавиатура не реагирует на нажатия	2026-04-10 17:25:35.743 +0700	[N]	5	2 024 006	Сетевое подключение нестабильное,	2026-04-10 17:25:35.743 +0700	2025-12-27 17:25:35.742 +	6	2 024 007	Жесткий диск издает посторонние зв	2026-04-10 17:25:35.743 +0700	[N]	7	2 024 008	Операционная система загружается	2026-04-10 17:25:35.743 +0700	[N]	8	2 024 012	Компьютер зависает при работе с гр	2026-04-10 17:25:35.743 +0700	2026-02-11 17:25:35.742 +	9	2 024 014	МФУ не сканирует, только печатает. I	2026-04-10 17:25:35.743 +0700	2026-04-15 07:00:00.000 +	10	2 024 010	Блок бесперебойного питания не де	2026-04-10 17:25:35.743 +0700	2026-04-11 07:00:00.000 +	11	2 024 017	Тестирование от техника	2026-04-10 17:29:02.865 +0700	2026-04-10 17:28:00.000 +	12	2 024 013	Сканер не определяется системой. Т	2026-04-10 17:25:35.743 +0700	2026-04-11 07:00:00.000 +	13	2 024 009	Wi-Fi адаптер не видит беспроводны	2026-04-10 17:25:35.743 +0700	2026-01-15 17:25:00.000 +	14	2 024 018	Не работает подача красителя, в рем	2026-04-13 16:57:55.066 +0700	2026-04-24 16:57:00.000 +	15	2 024 016	тестирование создания от сотрудни	2026-04-10 17:26:18.533 +0700	2026-04-11 00:30:00.000 +
	act_number	problem_description	registration_date	completion_date																																																																													
1	2 024 002	Принтер зажевывает бумагу при печ	2026-04-10 17:25:35.743 +0700	[N]																																																																													
2	2 024 003	Монитор мигает и периодически гас	2026-04-10 17:25:35.743 +0700	2025-11-22 17:25:35.742 +																																																																													
3	2 024 004	Ноутбук сильно греется и шумит. Тр	2026-04-10 17:25:35.743 +0700	[N]																																																																													
4	2 024 005	Клавиатура не реагирует на нажатия	2026-04-10 17:25:35.743 +0700	[N]																																																																													
5	2 024 006	Сетевое подключение нестабильное,	2026-04-10 17:25:35.743 +0700	2025-12-27 17:25:35.742 +																																																																													
6	2 024 007	Жесткий диск издает посторонние зв	2026-04-10 17:25:35.743 +0700	[N]																																																																													
7	2 024 008	Операционная система загружается	2026-04-10 17:25:35.743 +0700	[N]																																																																													
8	2 024 012	Компьютер зависает при работе с гр	2026-04-10 17:25:35.743 +0700	2026-02-11 17:25:35.742 +																																																																													
9	2 024 014	МФУ не сканирует, только печатает. I	2026-04-10 17:25:35.743 +0700	2026-04-15 07:00:00.000 +																																																																													
10	2 024 010	Блок бесперебойного питания не де	2026-04-10 17:25:35.743 +0700	2026-04-11 07:00:00.000 +																																																																													
11	2 024 017	Тестирование от техника	2026-04-10 17:29:02.865 +0700	2026-04-10 17:28:00.000 +																																																																													
12	2 024 013	Сканер не определяется системой. Т	2026-04-10 17:25:35.743 +0700	2026-04-11 07:00:00.000 +																																																																													
13	2 024 009	Wi-Fi адаптер не видит беспроводны	2026-04-10 17:25:35.743 +0700	2026-01-15 17:25:00.000 +																																																																													
14	2 024 018	Не работает подача красителя, в рем	2026-04-13 16:57:55.066 +0700	2026-04-24 16:57:00.000 +																																																																													
15	2 024 016	тестирование создания от сотрудни	2026-04-10 17:26:18.533 +0700	2026-04-11 00:30:00.000 +																																																																													
Результат шага №1	Отправка POST-запроса на удаление, заявка удаляется из базы данных Заявка полностью удалена																																																																																
Шаг №2	Проверить количество заявок в системе																																																																																
Результат шага №2	Количество заявок уменьшилось на 1, заявка с указанным номером акта больше не существует в базе данных и на странице <table><tr><th></th><th>act_number</th><th>problem_description</th><th>registration_date</th><th>completion_date</th></tr><tr><td>1</td><td>2 024 002</td><td>Принтер зажевывает бумагу при печ</td><td>2026-04-10 17:25:35.743 +0700</td><td>[N]</td></tr><tr><td>2</td><td>2 024 003</td><td>Монитор мигает и периодически гас</td><td>2026-04-10 17:25:35.743 +0700</td><td>2025-11-22 17:25:35.742 +</td></tr><tr><td>3</td><td>2 024 004</td><td>Ноутбук сильно греется и шумит. Тр</td><td>2026-04-10 17:25:35.743 +0700</td><td>[N]</td></tr><tr><td>4</td><td>2 024 005</td><td>Клавиатура не реагирует на нажатия</td><td>2026-04-10 17:25:35.743 +0700</td><td>[N]</td></tr><tr><td>5</td><td>2 024 006</td><td>Сетевое подключение нестабильное,</td><td>2026-04-10 17:25:35.743 +0700</td><td>2025-12-27 17:25:35.742 +</td></tr><tr><td>6</td><td>2 024 007</td><td>Жесткий диск издает посторонние зв</td><td>2026-04-10 17:25:35.743 +0700</td><td>[N]</td></tr><tr><td>7</td><td>2 024 008</td><td>Операционная система загружается</td><td>2026-04-10 17:25:35.743 +0700</td><td>[N]</td></tr><tr><td>8</td><td>2 024 012</td><td>Компьютер зависает при работе с гр</td><td>2026-04-10 17:25:35.743 +0700</td><td>2026-02-11 17:25:35.742 +</td></tr><tr><td>9</td><td>2 024 014</td><td>МФУ не сканирует, только печатает. I</td><td>2026-04-10 17:25:35.743 +0700</td><td>2026-04-15 07:00:00.000 +</td></tr><tr><td>10</td><td>2 024 010</td><td>Блок бесперебойного питания не де</td><td>2026-04-10 17:25:35.743 +0700</td><td>2026-04-11 07:00:00.000 +</td></tr><tr><td>11</td><td>2 024 017</td><td>Тестирование от техника</td><td>2026-04-10 17:29:02.865 +0700</td><td>2026-04-10 17:28:00.000 +</td></tr><tr><td>12</td><td>2 024 013</td><td>Сканер не определяется системой. Т</td><td>2026-04-10 17:25:35.743 +0700</td><td>2026-04-11 07:00:00.000 +</td></tr><tr><td>13</td><td>2 024 009</td><td>Wi-Fi адаптер не видит беспроводны</td><td>2026-04-10 17:25:35.743 +0700</td><td>2026-01-15 17:25:00.000 +</td></tr><tr><td>14</td><td>2 024 016</td><td>тестирование создания от сотрудни</td><td>2026-04-10 17:26:18.533 +0700</td><td>2026-04-11 00:30:00.000 +</td></tr></table>		act_number	problem_description	registration_date	completion_date	1	2 024 002	Принтер зажевывает бумагу при печ	2026-04-10 17:25:35.743 +0700	[N]	2	2 024 003	Монитор мигает и периодически гас	2026-04-10 17:25:35.743 +0700	2025-11-22 17:25:35.742 +	3	2 024 004	Ноутбук сильно греется и шумит. Тр	2026-04-10 17:25:35.743 +0700	[N]	4	2 024 005	Клавиатура не реагирует на нажатия	2026-04-10 17:25:35.743 +0700	[N]	5	2 024 006	Сетевое подключение нестабильное,	2026-04-10 17:25:35.743 +0700	2025-12-27 17:25:35.742 +	6	2 024 007	Жесткий диск издает посторонние зв	2026-04-10 17:25:35.743 +0700	[N]	7	2 024 008	Операционная система загружается	2026-04-10 17:25:35.743 +0700	[N]	8	2 024 012	Компьютер зависает при работе с гр	2026-04-10 17:25:35.743 +0700	2026-02-11 17:25:35.742 +	9	2 024 014	МФУ не сканирует, только печатает. I	2026-04-10 17:25:35.743 +0700	2026-04-15 07:00:00.000 +	10	2 024 010	Блок бесперебойного питания не де	2026-04-10 17:25:35.743 +0700	2026-04-11 07:00:00.000 +	11	2 024 017	Тестирование от техника	2026-04-10 17:29:02.865 +0700	2026-04-10 17:28:00.000 +	12	2 024 013	Сканер не определяется системой. Т	2026-04-10 17:25:35.743 +0700	2026-04-11 07:00:00.000 +	13	2 024 009	Wi-Fi адаптер не видит беспроводны	2026-04-10 17:25:35.743 +0700	2026-01-15 17:25:00.000 +	14	2 024 016	тестирование создания от сотрудни	2026-04-10 17:26:18.533 +0700	2026-04-11 00:30:00.000 +					
	act_number	problem_description	registration_date	completion_date																																																																													
1	2 024 002	Принтер зажевывает бумагу при печ	2026-04-10 17:25:35.743 +0700	[N]																																																																													
2	2 024 003	Монитор мигает и периодически гас	2026-04-10 17:25:35.743 +0700	2025-11-22 17:25:35.742 +																																																																													
3	2 024 004	Ноутбук сильно греется и шумит. Тр	2026-04-10 17:25:35.743 +0700	[N]																																																																													
4	2 024 005	Клавиатура не реагирует на нажатия	2026-04-10 17:25:35.743 +0700	[N]																																																																													
5	2 024 006	Сетевое подключение нестабильное,	2026-04-10 17:25:35.743 +0700	2025-12-27 17:25:35.742 +																																																																													
6	2 024 007	Жесткий диск издает посторонние зв	2026-04-10 17:25:35.743 +0700	[N]																																																																													
7	2 024 008	Операционная система загружается	2026-04-10 17:25:35.743 +0700	[N]																																																																													
8	2 024 012	Компьютер зависает при работе с гр	2026-04-10 17:25:35.743 +0700	2026-02-11 17:25:35.742 +																																																																													
9	2 024 014	МФУ не сканирует, только печатает. I	2026-04-10 17:25:35.743 +0700	2026-04-15 07:00:00.000 +																																																																													
10	2 024 010	Блок бесперебойного питания не де	2026-04-10 17:25:35.743 +0700	2026-04-11 07:00:00.000 +																																																																													
11	2 024 017	Тестирование от техника	2026-04-10 17:29:02.865 +0700	2026-04-10 17:28:00.000 +																																																																													
12	2 024 013	Сканер не определяется системой. Т	2026-04-10 17:25:35.743 +0700	2026-04-11 07:00:00.000 +																																																																													
13	2 024 009	Wi-Fi адаптер не видит беспроводны	2026-04-10 17:25:35.743 +0700	2026-01-15 17:25:00.000 +																																																																													
14	2 024 016	тестирование создания от сотрудни	2026-04-10 17:26:18.533 +0700	2026-04-11 00:30:00.000 +																																																																													
Шаг №3	Попытаться удалить невыполненную заявку от имени сотрудника																																																																																
Результат шага №3	Отображение сообщения «Можно удалить только выполненную заявку», заявка остаётся в системе																																																																																
Поведение	Негативное																																																																																
Статус	Пройден																																																																																

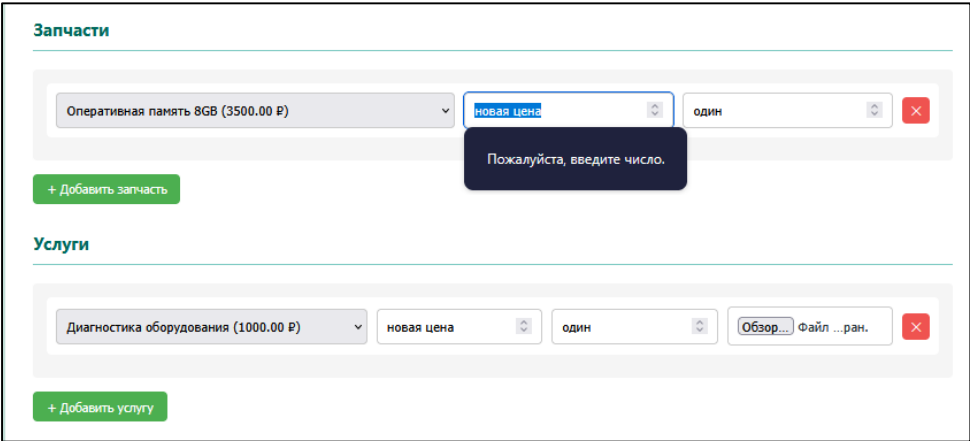
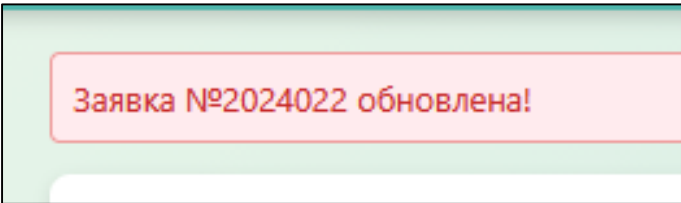
Таблица 4 – Тест-кейс №4

Номер	4																														
Название	Тестирование сортировки на странице техника																														
Предуслови е	Уже вошли под сотрудником с ролью «Техник», в системе существует минимум 3 заявки с разными датами регистрации																														
Шаг №1	Кликнуть по кнопке на странице «По дате (убыв.)»																														
Результат шага №1	Отображение заявок от последней к самой первой Поиск: Описание, номер... Статус: Все статусы Приоритет: Все приоритеты Сортировка: По дате (убыв.) <table><tr><th>№</th><th>Дата</th><th>Описание</th><th>Заказчик</th><th>Оборудование</th><th>Статус</th></tr><tr><td>2024018</td><td>05.05.2026 10:26</td><td>Сетевое подключение нестабильное, постоянные обрывы связи. Требуется проверка ...</td><td>Сидоров Сергей</td><td>10012</td><td>Списано</td></tr><tr><td>2024020</td><td>02.05.2026 10:26</td><td>Ноутбук сильно греется и шумит. Требуется чистка системы ...</td><td>Новиков Игорь</td><td>10008</td><td>В ремонте</td></tr><tr><td>2024017</td><td>10.04.2026 10:29</td><td>Тестирование от техника</td><td>Васильев Евгений</td><td>10008</td><td>В ремонте</td></tr><tr><td>2024016</td><td>10.04.2026</td><td>Тестирование сортировки от сотрудника</td><td>Иванов</td><td>10001</td><td>Исправлен</td></tr></table>	№	Дата	Описание	Заказчик	Оборудование	Статус	2024018	05.05.2026 10:26	Сетевое подключение нестабильное, постоянные обрывы связи. Требуется проверка ...	Сидоров Сергей	10012	Списано	2024020	02.05.2026 10:26	Ноутбук сильно греется и шумит. Требуется чистка системы ...	Новиков Игорь	10008	В ремонте	2024017	10.04.2026 10:29	Тестирование от техника	Васильев Евгений	10008	В ремонте	2024016	10.04.2026	Тестирование сортировки от сотрудника	Иванов	10001	Исправлен
№	Дата	Описание	Заказчик	Оборудование	Статус																										
2024018	05.05.2026 10:26	Сетевое подключение нестабильное, постоянные обрывы связи. Требуется проверка ...	Сидоров Сергей	10012	Списано																										
2024020	02.05.2026 10:26	Ноутбук сильно греется и шумит. Требуется чистка системы ...	Новиков Игорь	10008	В ремонте																										
2024017	10.04.2026 10:29	Тестирование от техника	Васильев Евгений	10008	В ремонте																										
2024016	10.04.2026	Тестирование сортировки от сотрудника	Иванов	10001	Исправлен																										
Шаг №2	Кликнуть по кнопке «По дате (возр.)»																														
Результат шага №2	Отображение заявок от первой к последней добавленной Поиск: Описание, номер... Статус: Все статусы Приоритет: Все приоритеты Сортировка: По дате (возр.) <table><tr><th>№</th><th>Дата</th><th>Описание</th><th>Заказчик</th><th>Оборудование</th><th>Статус</th></tr><tr><td>2024021</td><td>01.04.2026 10:26</td><td>Жесткий диск издает посторонние звуки, возможные сектора. Требуется ...</td><td>Мальцева Анастасия</td><td>10009</td><td>В ремонте</td></tr><tr><td>2024002</td><td>10.04.2026 10:25</td><td>Принтер зажевывает бумагу при печати, необходима замена роликов ...</td><td>Сидоров Сергей</td><td>10002</td><td>Списано</td></tr><tr><td>2024003</td><td>10.04.2026 10:25</td><td>Монитор мигает и периодически гаснет. Проблема с кабелем ...</td><td>Козлов Андрей</td><td>10003</td><td>В ремонте</td></tr><tr><td>2024004</td><td>10.04.2026 10:25</td><td>Ноутбук сильно греется и шумит. Требуется чистка системы ...</td><td>Смирнов Михаил</td><td>10004</td><td>Ожидает запчасти</td></tr></table>	№	Дата	Описание	Заказчик	Оборудование	Статус	2024021	01.04.2026 10:26	Жесткий диск издает посторонние звуки, возможные сектора. Требуется ...	Мальцева Анастасия	10009	В ремонте	2024002	10.04.2026 10:25	Принтер зажевывает бумагу при печати, необходима замена роликов ...	Сидоров Сергей	10002	Списано	2024003	10.04.2026 10:25	Монитор мигает и периодически гаснет. Проблема с кабелем ...	Козлов Андрей	10003	В ремонте	2024004	10.04.2026 10:25	Ноутбук сильно греется и шумит. Требуется чистка системы ...	Смирнов Михаил	10004	Ожидает запчасти
№	Дата	Описание	Заказчик	Оборудование	Статус																										
2024021	01.04.2026 10:26	Жесткий диск издает посторонние звуки, возможные сектора. Требуется ...	Мальцева Анастасия	10009	В ремонте																										
2024002	10.04.2026 10:25	Принтер зажевывает бумагу при печати, необходима замена роликов ...	Сидоров Сергей	10002	Списано																										
2024003	10.04.2026 10:25	Монитор мигает и периодически гаснет. Проблема с кабелем ...	Козлов Андрей	10003	В ремонте																										
2024004	10.04.2026 10:25	Ноутбук сильно греется и шумит. Требуется чистка системы ...	Смирнов Михаил	10004	Ожидает запчасти																										
Шаг №3	Кликнуть по кнопке «По номеру»																														
Результат шага №3	Отображение заявок, отсортированных по номеру акта в порядке возрастания Поиск: Описание, номер... Статус: Все статусы Приоритет: Все приоритеты Сортировка: По номеру <table><tr><th>№</th><th>Дата</th><th>Описание</th><th>Заказчик</th><th>Оборудование</th><th>Статус</th></tr><tr><td>2024002</td><td>10.04.2026 10:25</td><td>Принтер зажевывает бумагу при печати, необходима замена роликов ...</td><td>Сидоров Сергей</td><td>10002</td><td>Списано</td></tr><tr><td>2024003</td><td>10.04.2026 10:25</td><td>Монитор мигает и периодически гаснет. Проблема с кабелем ...</td><td>Козлов Андрей</td><td>10003</td><td>В ремонте</td></tr><tr><td>2024004</td><td>10.04.2026 10:25</td><td>Ноутбук сильно греется и шумит. Требуется чистка системы ...</td><td>Смирнов Михаил</td><td>10004</td><td>Ожидает запчасти</td></tr></table>	№	Дата	Описание	Заказчик	Оборудование	Статус	2024002	10.04.2026 10:25	Принтер зажевывает бумагу при печати, необходима замена роликов ...	Сидоров Сергей	10002	Списано	2024003	10.04.2026 10:25	Монитор мигает и периодически гаснет. Проблема с кабелем ...	Козлов Андрей	10003	В ремонте	2024004	10.04.2026 10:25	Ноутбук сильно греется и шумит. Требуется чистка системы ...	Смирнов Михаил	10004	Ожидает запчасти						
№	Дата	Описание	Заказчик	Оборудование	Статус																										
2024002	10.04.2026 10:25	Принтер зажевывает бумагу при печати, необходима замена роликов ...	Сидоров Сергей	10002	Списано																										
2024003	10.04.2026 10:25	Монитор мигает и периодически гаснет. Проблема с кабелем ...	Козлов Андрей	10003	В ремонте																										
2024004	10.04.2026 10:25	Ноутбук сильно греется и шумит. Требуется чистка системы ...	Смирнов Михаил	10004	Ожидает запчасти																										
Поведение	Позитивное																														
Статус	Пройден																														

Таблица 5 – Тест-кейс №5

Номер	5																																																																
Название	Тестирование поиска и статуса на странице техника																																																																
Предуслови е	Уже вошли под сотрудником с ролью «Техник», в системе существуют заявки с разными статусами оборудования																																																																
Шаг №1	Выбрать в поле «Статус» поле «В ремонте»																																																																
Результат шага №1	Отображение заявок со статусом «В ремонте» Поиск: Статус: Приоритет: Сортировка: Описание, номер... Все статусы Все приоритеты По дате (убыв.) Применить Сброс <table><tr><th>№</th><th>Дата</th><th>Описание</th><th>Заказчик</th><th>Оборудование</th><th>Статус</th><th>Приоритет</th><th>Действия</th></tr><tr><td>2024020</td><td>02.05.2026 10:26</td><td>Ноутбук сильно греется и шумит. Требуется чистка системы ...</td><td>Новиков Игорь</td><td>10008</td><td>В ремонте</td><td>Средний</td><td> Изменить Удалить </td></tr><tr><td>2024017</td><td>10.04.2026 10:29</td><td>Тестирование от техника</td><td>Васильев Евгений</td><td>10008</td><td>В ремонте</td><td>Низкий</td><td> Изменить Печать Удалить </td></tr><tr><td>2024009</td><td>10.04.2026 10:25</td><td>Wi-Fi адаптер не видит беспроводные сети. Требуется замена ...</td><td>Новиков Игорь</td><td>10009</td><td>В ремонте</td><td>Средний</td><td> Изменить Печать Удалить </td></tr><tr><td>2024008</td><td>10.04.2026 10:25</td><td>Операционная система загружается с ошибками, требуется переустановка.</td><td>Михайлов Алексей</td><td>10008</td><td>В ремонте</td><td>Низкий</td><td> Изменить Удалить </td></tr><tr><td>2024006</td><td>10.04.2026 10:25</td><td>Сетевое подключение нестабильное, постоянные обрывы связи. Требуется проверка ...</td><td>Попов Николай</td><td>10006</td><td>В ремонте</td><td>Высокий</td><td> Изменить Удалить </td></tr><tr><td>2024003</td><td>10.04.2026 10:25</td><td>Монитор мигает и периодически гаснет. Проблема с кабелем ...</td><td>Козлов Андрей</td><td>10003</td><td>В ремонте</td><td>Критический</td><td> Изменить Удалить </td></tr><tr><td>2024021</td><td>01.04.2026 10:26</td><td>Жесткий диск издает посторонние звуки, возможные повреждение сектора. Требуется ...</td><td>Мальцева Анастасия</td><td>10009</td><td>В ремонте</td><td>Высокий</td><td> Изменить Удалить </td></tr></table>	№	Дата	Описание	Заказчик	Оборудование	Статус	Приоритет	Действия	2024020	02.05.2026 10:26	Ноутбук сильно греется и шумит. Требуется чистка системы ...	Новиков Игорь	10008	В ремонте	Средний	Изменить Удалить	2024017	10.04.2026 10:29	Тестирование от техника	Васильев Евгений	10008	В ремонте	Низкий	Изменить Печать Удалить	2024009	10.04.2026 10:25	Wi-Fi адаптер не видит беспроводные сети. Требуется замена ...	Новиков Игорь	10009	В ремонте	Средний	Изменить Печать Удалить	2024008	10.04.2026 10:25	Операционная система загружается с ошибками, требуется переустановка.	Михайлов Алексей	10008	В ремонте	Низкий	Изменить Удалить	2024006	10.04.2026 10:25	Сетевое подключение нестабильное, постоянные обрывы связи. Требуется проверка ...	Попов Николай	10006	В ремонте	Высокий	Изменить Удалить	2024003	10.04.2026 10:25	Монитор мигает и периодически гаснет. Проблема с кабелем ...	Козлов Андрей	10003	В ремонте	Критический	Изменить Удалить	2024021	01.04.2026 10:26	Жесткий диск издает посторонние звуки, возможные повреждение сектора. Требуется ...	Мальцева Анастасия	10009	В ремонте	Высокий	Изменить Удалить
№	Дата	Описание	Заказчик	Оборудование	Статус	Приоритет	Действия																																																										
2024020	02.05.2026 10:26	Ноутбук сильно греется и шумит. Требуется чистка системы ...	Новиков Игорь	10008	В ремонте	Средний	Изменить Удалить																																																										
2024017	10.04.2026 10:29	Тестирование от техника	Васильев Евгений	10008	В ремонте	Низкий	Изменить Печать Удалить																																																										
2024009	10.04.2026 10:25	Wi-Fi адаптер не видит беспроводные сети. Требуется замена ...	Новиков Игорь	10009	В ремонте	Средний	Изменить Печать Удалить																																																										
2024008	10.04.2026 10:25	Операционная система загружается с ошибками, требуется переустановка.	Михайлов Алексей	10008	В ремонте	Низкий	Изменить Удалить																																																										
2024006	10.04.2026 10:25	Сетевое подключение нестабильное, постоянные обрывы связи. Требуется проверка ...	Попов Николай	10006	В ремонте	Высокий	Изменить Удалить																																																										
2024003	10.04.2026 10:25	Монитор мигает и периодически гаснет. Проблема с кабелем ...	Козлов Андрей	10003	В ремонте	Критический	Изменить Удалить																																																										
2024021	01.04.2026 10:26	Жесткий диск издает посторонние звуки, возможные повреждение сектора. Требуется ...	Мальцева Анастасия	10009	В ремонте	Высокий	Изменить Удалить																																																										
Шаг №2	Ввести в поле «Поиск» текст «ноут»																																																																
Результат шага №2	Отображение заявок, в описании которых встречается слово «ноут», остальные заявки скрыты Поиск: Статус: Приоритет: Сортировка: ноут Все статусы Все приоритеты По дате (убыв.) Применить Сброс <table><tr><th>№</th><th>Дата</th><th>Описание</th><th>Заказчик</th><th>Оборудование</th><th>Статус</th><th>Приоритет</th><th>Действия</th></tr><tr><td>2024020</td><td>02.05.2026 10:26</td><td>Ноутбук сильно греется и шумит. Требуется чистка системы ...</td><td>Новиков Игорь</td><td>10008</td><td>В ремонте</td><td>Средний</td><td> Изменить Удалить </td></tr><tr><td>2024004</td><td>10.04.2026 10:25</td><td>Ноутбук сильно греется и шумит. Требуется чистка системы ...</td><td>Смирнов Михаил</td><td>10004</td><td>Ожидает запчасти</td><td>Низкий</td><td> Изменить Печать Удалить </td></tr></table>	№	Дата	Описание	Заказчик	Оборудование	Статус	Приоритет	Действия	2024020	02.05.2026 10:26	Ноутбук сильно греется и шумит. Требуется чистка системы ...	Новиков Игорь	10008	В ремонте	Средний	Изменить Удалить	2024004	10.04.2026 10:25	Ноутбук сильно греется и шумит. Требуется чистка системы ...	Смирнов Михаил	10004	Ожидает запчасти	Низкий	Изменить Печать Удалить																																								
№	Дата	Описание	Заказчик	Оборудование	Статус	Приоритет	Действия																																																										
2024020	02.05.2026 10:26	Ноутбук сильно греется и шумит. Требуется чистка системы ...	Новиков Игорь	10008	В ремонте	Средний	Изменить Удалить																																																										
2024004	10.04.2026 10:25	Ноутбук сильно греется и шумит. Требуется чистка системы ...	Смирнов Михаил	10004	Ожидает запчасти	Низкий	Изменить Печать Удалить																																																										
Шаг №3	Ввести в поле «Поиск» текст «несуществующий текст»																																																																
Результат шага №3	Отображение сообщения «Заявок не найдено», список заявок пуст Поиск: Статус: несуществующий текст Все статусы <table><tr><th>№</th><th>Дата</th><th>Описание</th><th>Заказчик</th><th>Оборудование</th></tr><tr><td colspan="5">Заявок не найдено</td></tr></table>	№	Дата	Описание	Заказчик	Оборудование	Заявок не найдено																																																										
№	Дата	Описание	Заказчик	Оборудование																																																													
Заявок не найдено																																																																	
Поведение	Позитивное																																																																
Статус	Пройден																																																																

Таблица 6 – Тест-кейс №6

Номер	1
Название	Тестирование ввода некорректных данных при редактировании заявки
Предусловие	Запущен сервер разработки. Запустить программу через python manage.py runserver и перейти на страницу техника, редактирование заявки
Шаг №1	Ввод некорректных данных в поля с ценой и количеством запчастей и услуг, нажать кнопку «сохранить изменения»
Результат шага №1	Отображение окна с редактированием заявки, сообщение о некорректных данных у первого поля в последовательности и выделение поля для редактирования 
Шаг №2	Ввод некорректных данных в поля с ценой запчастей и услуг, нажать кнопку «сохранить изменения»
Результат шага №2	Отображение окна с редактированием заявки, сообщение о некорректных данных у второго поля в последовательности и выделение поля для редактирования, аналогично и для услуг 
Шаг №3	Ввод корректных данных и нажать кнопку «сохранить изменения»
Результат шага №3	Отображение сообщения об успешном редактировании заявки 
Поведение	Негативное
Статус	Пройден

4.2 Отладка программного продукта с использованием специализированных программных средств

Для автоматизации данного процесса в проекте применяется встроенный фреймворк тестирования Django основанный на библиотеке unittest, который предоставляет возможности для эмуляции действий пользователя работы с базой данных и проверки состояний приложения без необходимости ручного вмешательства. Выбор именно этого инструментария обусловлен его полной интеграцией с экосистемой Django, что позволяет изолированно разворачивать тестовую базу данных для каждого запуска, предотвращая повреждение реальных данных учреждения и обеспечивая показ результатов. В рамках тестирования были выбраны сценарии покрывающие наиболее критичные бизнес процессы системы, включая аутентификацию пользователей с различными ролями навигацию по основным разделам интерфейса, операции удаления заявок с проверкой прав доступа и механизмов мягкого удаления, а также функциональность сортировки и фильтрации списков данных [11].

Тестирование входа в систему с ролью «техник» проходит при помощи отправки POST-запроса с логином и паролем на страницу входа с последующей проверкой кода и перенаправления на страницу техника, а также верификации данных сессии, где сохраняется идентификатор пользователя и его роль. Код тестирования можно посмотреть в соответствии с листингом 4.

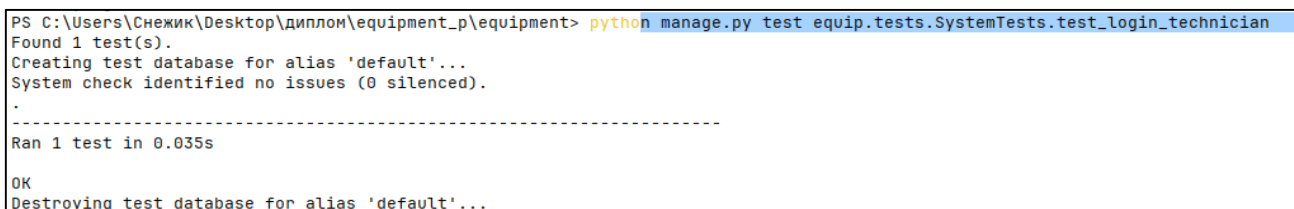
Листинг 4 - Test_login_technician

```
#Тест проверяет корректность процесса аутентификации
пользователя с ролью «Техник» через отправку POST-запроса с
валидными учетными данными.
#Метод эмулирует ввод логина и пароля на странице входа,
фиксирует ответ сервера и проверяет успешное перенаправление на
личную панель специалиста.
#Дополнительно выполняется верификация данных сессии, где
должны сохраниться уникальный идентификатор пользователя и его
системная роль.
def test_login_technician(self):
    response = self.client.post(reverse('login_view'),
{ #Отправка POST-запроса с тестовыми данными к URL-адресу
представления входа
        'username': 'tech@hospital.ru', #Передача тестового
логина, соответствующего учетной записи техника в базе данных
        'password': 'password123' #Передача корректного
пароля для успешной авторизации пользователя
    }) #Закрытие словаря с данными формы и завершение вызова
метода отправки запроса
```

Продолжение листинга 4

```
self.assertEqual(response.status_code, 302) #Проверка
кода HTTP-ответа 302, подтверждающего успешное перенаправление
после входа
self.assertRedirects(response,
reverse('show_technician')) #Верификация того, что редирект ведёт
точно на страницу панели техника
session = self.client.session #Извлечение объекта текущей
сессии тестового клиента для анализа сохранённых данных
self.assertEqual(session['user_id'], 1) #Проверка
корректности сохранения идентификатора вошедшего пользователя в
сессии
self.assertEqual(session['user_role'], 'Техник')
#Подтверждение того, что в сессии зафиксирована правильная
системная роль пользователя
```

Посмотреть результат выполнения теста можно в соответствии с рисунком 4.1.



```
PS C:\Users\Снежик\Desktop\диплом\equipment_p\equipment> python manage.py test equip.tests.SystemTests.test_login_technician
Found 1 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.
-----
Ran 1 test in 0.035s

OK
Destroying test database for alias 'default'...
```

Рисунок 4.1 – Результат тестирования входа в систему с ролью «техник»

Тест навигации техника по страницам системы проверяет возможность перехода авторизованного техника на страницу оборудования и страницу статистики путём выполнения GET-запросов к соответствующим URL-адресам с проверкой кода и наличия ключевых слов на странице, что подтверждает доступность разделов системы для данной роли пользователя. Код тестирования можно посмотреть в листинге 5.

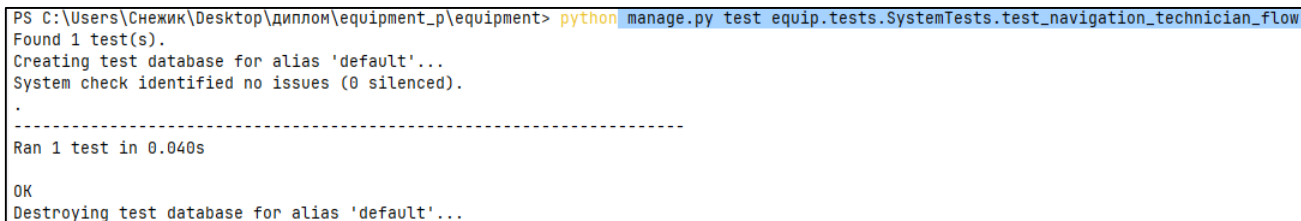
Листинг 5 - Test_navigation_technician_flow

```
#Тест проверяет корректность навигации авторизованного техника
по ключевым разделам системы после успешного входа в аккаунт.
#Метод эмулирует последовательные GET-запросы к страницам новых
заявок и реестра оборудования, фиксируя ответы сервера.
#Дополнительно выполняется верификация HTTP-статусов и проверка
наличия ключевых заголовков в содержимом страниц, что подтверждает
доступность разделов для данной роли.
def test_navigation_technician_flow(self):
    self.client.post(reverse('login_view'), { #Отправка POST-
запроса для эмуляции входа в систему под учетной записью техника
        'username': 'tech@hospital.ru', #Передача тестового
логина, соответствующего роли технического специалиста
        'password': 'password123' #Передача корректного
пароля для успешной авторизации и открытия сессии
```

Продолжение листинга 5

```
    }) #Завершение вызова метода авторизации и сохранение
    состояния сессии в тестовом клиенте
    #Переход на новые заявки
    response_new =
self.client.get(reverse('show_new_requests')) #Выполнение GET-
запроса к URL-адресу страницы новых заявок
    self.assertEqual(response_new.status_code, 200) #Проверка
кода ответа сервера 200 ОК, подтверждающего успешную загрузку
страницы
    self.assertContains(response_new, 'Новые заявки')
#Верификация присутствия заголовка раздела в HTML-ответе для
подтверждения корректности маршрута
    #Переход на оборудование
    response_equip =
self.client.get(reverse('show_equipment')) #Выполнение GET-
запроса к URL-адресу страницы реестра оборудования
    self.assertEqual(response_equip.status_code, 200)
#Проверка кода ответа сервера 200 ОК, подтверждающего успешную
загрузку страницы
    self.assertContains(response_equip, 'Оборудование')
#Верификация присутствия заголовка раздела в HTML-ответе для
подтверждения корректности маршрута
```

Посмотреть результат выполнения теста можно в соответствии с рисунком 4.2.



```
PS C:\Users\Снежик\Desktop\диплом\equipment_p\equipment> python manage.py test equip.tests.SystemTests.test_navigation_technician_flow
Found 1 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.
-----
Ran 1 test in 0.040s
OK
Destroying test database for alias 'default'...
```

Рисунок 4.2 – Результат тестирования навигации техника по страницам системы

Тест удаления заявки сотрудником и техником проверяет два сценария удаления заявки, где сотрудник выполняет мягкое удаление своей выполненной заявки путём установки даты удаления без физического удаления из базы данных, а техник выполняет полное удаление заявки, что подтверждается проверкой существования записи в базе данных и наличия значения в поле даты удаления. Основной код тестирования можно посмотреть в соответствии с листингом 6.

Листинг 6 - Test_delete_request

```
#Тест проверяет корректность работы двух механизмов удаления
заявок: физического удаления техническим специалистом и мягкого
удаления (архивирования) рядовым сотрудником.
```


Продолжение листинга 6

```
#Метод эмулирует последовательные POST-запросы от
авторизованных пользователей с разными ролями, фиксируя изменения
в состоянии базы данных.
#Дополнительно выполняется верификация HTTP-статусов
перенаправления, наличия или отсутствия записей в таблице и
корректности установки временной метки при архивировании.
#Создание тестовой заявки с присвоением номера акта, фиксацией
времени завершения и явным указанием отсутствия даты удаления
req = RequestFix.objects.create(act_number=2024001,
completion_date=timezone.now(), delete_date=None)
#Эмуляция входа в систему под учетной записью технического
специалиста для получения прав на полное удаление
self.client.post(reverse('login_view'), {'username':
'tech@hospital.ru', 'password': 'password123'})
#Отправка POST-запроса к URL-адресу физического удаления заявки
по её уникальному номеру акта
response =
self.client.post(reverse('delete_request_technician',
args=[2024001]))
#Проверка кода HTTP-ответа 302, подтверждающего успешное
перенаправление после выполнения операции удаления
self.assertEqual(response.status_code, 302)
#Верификация полного физического отсутствия удалённой записи в
базе данных через ORM-запрос фильтрации

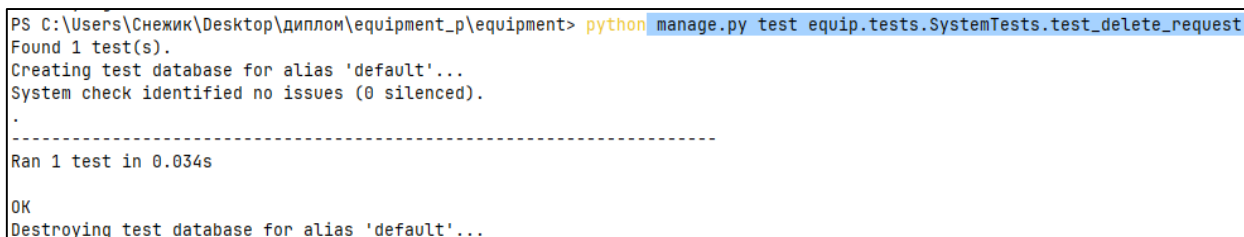
self.assertFalse(RequestFix.objects.filter(act_number=2024001).e
xists())

#Инициализация второй тестовой заявки для проверки сценария
мягкого удаления (архивирования)
req2 = RequestFix.objects.create(act_number=2024002,
completion_date=timezone.now(), delete_date=None)
#Принудительное завершение текущей сессии техника перед входом
в систему под другой ролью пользователя
self.client.logout()
#Эмуляция авторизации под учетной записью рядового сотрудника,
наделённого правом только на архивирование
self.client.post(reverse('login_view'), {'username':
'emp@hospital.ru', 'password': 'password123'})
#Выполнение POST-запроса к URL-адресу мягкого удаления заявки
с правами заказчика обращения
response_soft =
self.client.post(reverse('delete_request_customer',
args=[2024002]))
#Проверка корректного HTTP-редиректа после успешной обработки
запроса на архивирование записи
self.assertEqual(response_soft.status_code, 302)
#Извлечение архивированной заявки из базы данных по номеру акта
для последующей проверки её состояния
deleted_req = RequestFix.objects.get(act_number=2024002)
```

Продолжение листинга 6

```
#Подтверждение наличия временной метки удаления, что
свидетельствует об успешном мягком удалении без потери данных
self.assertIsNotNone(deleted_req.delete_date)
```

Посмотреть результат выполнения теста можно в соответствии с рисунком 4.3.



```
PS C:\Users\Снежик\Desktop\диплом\equipment_p\equipment> python manage.py test equip.tests.SystemTests.test_delete_request
Found 1 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.
-----
Ran 1 test in 0.034s
OK
Destroying test database for alias 'default'...
```

Рисунок 4.3 – Результат тестирования удаления заявки сотрудником и техником

Тест сортировки заявок на странице техника проверяет корректность работы сортировки заявок по дате регистрации в порядке убывания и возрастания, а также по номеру акта, путём отправки GET-запросов с параметром sort и последующей проверкой порядка следования заявок в контексте ответа, что подтверждает правильность реализации механизма сортировки. Основной код тестирования можно посмотреть в соответствии с листингом 7.

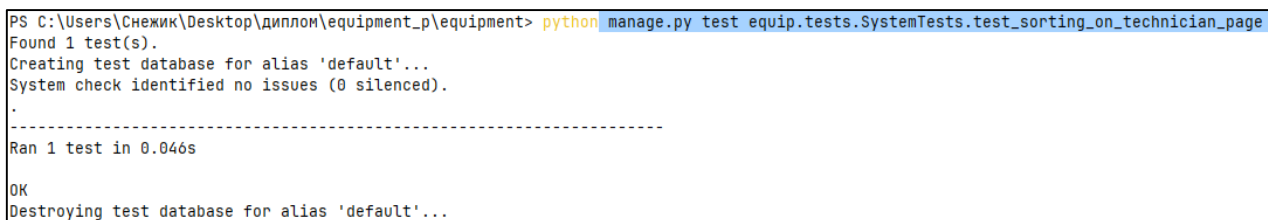
Листинг 7 - Test_sorting_on_technician_page

```
# Эмуляция входа в систему под учётной записью технического
специалиста для получения доступа к странице
self.client.post(reverse('login_view'), {'username':
'tech@hospital.ru', 'password': 'password123'})
# Создание первой тестовой заявки с фиксированной ранней датой
регистрации для формирования тестового набора данных
RequestFix.objects.create(act_number=2024010, requester_id=2,
assigned_technician_id=1, equipment_id=10001, category_id=1,
repair_stage_id=1, priority_id=1, problem_description='Заявка 1',
registration_date='2024-01-01 10:00:00')
# Создание второй тестовой заявки с более поздней датой регистрации
для проверки изменения порядка следования записей
RequestFix.objects.create(act_number=2024020, requester_id=2,
assigned_technician_id=1, equipment_id=10001, category_id=1,
repair_stage_id=1, priority_id=1, problem_description='Заявка 2',
registration_date='2024-01-03 10:00:00')
# Выполнение GET-запроса к странице техника с параметром
сортировки по дате регистрации в порядке убывания
response_desc = self.client.get(reverse('show_technician'),
{'sort': '-registration_date'})
# Извлечение списка заявок из контекста ответа сервера для
последующей верификации их физического порядка
requests_desc = response_desc.context['requests']
```

Продолжение листинга 7

```
# Проверка соответствия первого элемента списка заявке с более
поздней датой (2024020) при сортировке по убыванию
self.assertEqual(requests_desc[0].act_number, 2024020)
# Проверка соответствия второго элемента списка заявке с более
ранней датой (2024010) при сортировке по убыванию
self.assertEqual(requests_desc[1].act_number, 2024010)
# Выполнение GET-запроса к странице техника с параметром
сортировки по дате регистрации в порядке возрастания
response_asc = self.client.get(reverse('show_technician'),
{'sort': 'registration_date'})
# Извлечение обновлённого списка заявок из контекста ответа
сервера для проверки обратного хронологического порядка
requests_asc = response_asc.context['requests']
# Проверка соответствия первого элемента списка заявке с более
ранней датой (2024010) при сортировке по возрастанию
self.assertEqual(requests_asc[0].act_number, 2024010)
# Проверка соответствия второго элемента списка заявке с более
поздней датой (2024020) при сортировке по возрастанию
self.assertEqual(requests_asc[1].act_number, 2024020)
```

Посмотреть результат выполнения теста можно в соответствии с рисунком 4.4.



```
PS C:\Users\Снежик\Desktop\диплом\equipment_p\equipment> python manage.py test equip.tests.SystemTests.test_sorting_on_technician_page
Found 1 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.
-----
Ran 1 test in 0.046s

OK
Destroying test database for alias 'default'...
```

Рисунок 4.4 – Результат тестирования сортировки заявок на странице техника

Тест поиска и фильтрации заявок по статусу проверяет работу механизма поиска заявок по тексту в описании проблемы и фильтрации по статусу оборудования, путём отправки GET-запросов с параметрами `search` и `status`, с последующей проверкой количества найденных заявок и соответствия их характеристик заданным критериям фильтрации. Основной код тестирования можно посмотреть в соответствии с листингом 8.

Листинг 8 - Test_search_and_status_filter

```
# Эмуляция авторизации в системе под учётной записью технического
специалиста
self.client.post(reverse('login_view'), {'username':
'tech@hospital.ru', 'password': 'password123'})
# Создание тестового оборудования с различными статусами для
обеспечения работы фильтра по внешнему ключу статуса
Equipment.objects.create(inventory_number=10002,
assigned_office_id=1, model_id=1, type_id=1, status_id=1,
configuration='OK')
```

Продолжение листинга 8

```
Equipment.objects.create(inventory_number=10003,
assigned_office_id=1, model_id=1, type_id=1, status_id=2,
configuration='Bad')
# Регистрация заявки с описанием, содержащим ключевое слово
«принтер» для проверки избирательности текстового поиска
RequestFix.objects.create(act_number=2024030, requester_id=2,
assigned_technician_id=1, equipment_id=10001, category_id=1,
repair_stage_id=1, priority_id=1, problem_description='Сломался
принтер')
# Регистрация заявки с иным описанием для проверки корректности
исключения нерелевантных записей при поиске
RequestFix.objects.create(act_number=2024040, requester_id=2,
assigned_technician_id=1, equipment_id=10002, category_id=1,
repair_stage_id=1, priority_id=1, problem_description='Сломался
сканер')
# Регистрация заявки, привязанной к оборудованию со статусом «В
ремонте» (ID=2) для проверки работы фильтра
RequestFix.objects.create(act_number=2024050, requester_id=2,
assigned_technician_id=1, equipment_id=10003, category_id=1,
repair_stage_id=1, priority_id=1, problem_description='Полная
поломка')
# Выполнение GET-запроса с параметром текстового поиска по
ключевому слову «принтер» в описании проблемы
response_search = self.client.get(reverse('show_technician'),
{'search': 'принтер'})
# Проверка точного соответствия количества найденных заявок
единице, подтверждая избирательность поиска
self.assertEqual(len(response_search.context['requests']), 1)
# Верификация присутствия искомого ключевого слова в поле описания
неисправности единственной найденной записи
self.assertIn('принтер',
response_search.context['requests'][0].problem_description.lower
())
# Выполнение GET-запроса с параметром фильтрации по идентификатору
статуса оборудования (ID=2)
response_status = self.client.get(reverse('show_technician'),
{'status': '2'})
# Проверка соответствия количества отфильтрованных заявок
ожидаемому значению, подтверждая корректность фильтра
self.assertEqual(len(response_status.context['requests']), 2)
```

Посмотреть результат выполнения теста можно в соответствии с рисунком 4.5.

```

PS C:\Users\Снежик\Desktop\диплом\equipment_p\equipment> python manage.py test equip.tests.SystemTests.test_search_and_status_filter
Found 1 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.
-----
Ran 1 test in 0.046s

OK
Destroying test database for alias 'default'...

```

Рисунок 4.5 – Результат тестирования поиска и фильтрации заявок по статусу

Тест обработки некорректных данных в полях запчастей и услуг проверяет устойчивость механизма редактирования заявки к вводу буквенных символов вместо числовых значений в полях количества и стоимости, путём отправки POST-запроса с невалидными данными, с последующей проверкой корректного завершения, отсутствия сохранения некорректных записей в базе данных и сохранения целостности основной заявки. Основной код тестирования можно посмотреть в соответствии с листингом 9.

Листинг 9 - Test_edit_request_and_services

```

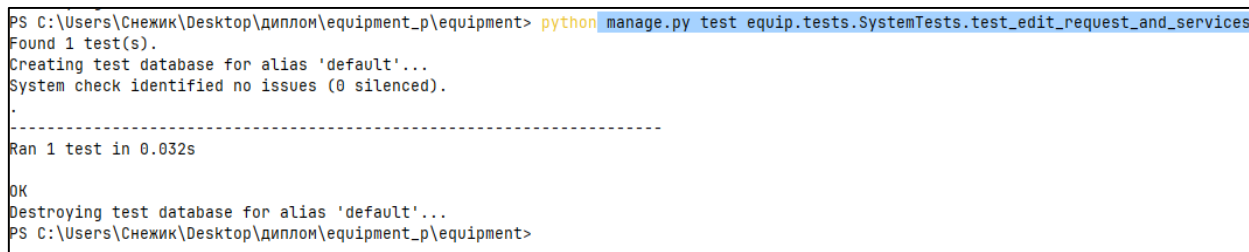
# Эмуляция входа в систему под учётной записью технического
специалиста для получения прав на редактирование
self.client.post(reverse('login_view'), {'username':
'tech@hospital.ru', 'password': 'password123'})
# Создание тестовой заявки с заполнением обязательных атрибутов
для последующей проверки механизма обновления
req = RequestFix.objects.create(act_number=2024999,
requester_id=self.employee.id,
assigned_technician_id=self.technician.id,
equipment_id=self.equipment.inventory_number,
category_id=self.category.id, repair_stage_id=self.stage_new.id,
priority_id=self.priority_low.id, problem_description='Тест
некорректных данных в запчастях или услугах')
# Инициализация справочных записей запчасти и услуги для
формирования валидных ссылок в POST-данных
spare = SparePart.objects.create(id=500, name='ТестЗапчасть',
quantity=10, cost=100.00)
service = ThirdPartyService.objects.create(id=500,
name='ТестУслуга', cost=200.00)
# Формирование словаря с некорректными данными, где вместо
числовых значений количества и цены переданы буквенные строки
invalid_data = {'problem_description': 'Проверка валидации ввода',
'assigned_technician': str(self.technician.id), 'equipment':
str(self.equipment.inventory_number), 'category':
str(self.category.id), 'priority': str(self.priority_low.id),
'repair_stage': str(self.stage_new.id), 'status':
str(self.status_ok.id), 'completion_date': '', 'spare_part_id[]':
[str(spare.id)], 'spare_part_quantity[]': ['abc'],
'spare_part_price[]': ['xyz'], 'service_id[]': [str(service.id)],
'service_quantity[]': ['def'], 'service_price[]': ['uvw']}

```

Продолжение листинга 9

```
# Отправка POST-запроса на URL редактирования заявки с передачей
сформированного словаря невалидных данных
response = self.client.post(reverse('edit_request',
args=[req.act_number]), data=invalid_data)
# Проверка кода HTTP-ответа 302, подтверждающего успешное
перенаправление после обработки ошибки валидации
self.assertEqual(response.status_code, 302)
# Верификация отсутствия записей в таблице запчастей, связанных с
заявкой, что подтверждает откат транзакции
self.assertEqual(RequestSparePart.objects.filter(request=req).count(), 0, "Некорректная запчасть не должна была сохраниться в БД")
# Верификация отсутствия записей в таблице услуг, связанных с
заявкой, что подтверждает корректность обработки исключения
self.assertEqual(RequestService.objects.filter(request=req).count(), 0, "Некорректная услуга не должна была сохраниться в БД")
# Принудительное обновление состояния объекта заявки из базы
данных для получения актуальных значений полей
req.refresh_from_db()
# Подтверждение отсутствия временной метки удаления, что
свидетельствует о сохранении целостности основной записи
self.assertIsNone(req.delete_date, "Заявка не должна была быть
удалена")
```

Посмотреть результат выполнения теста можно в соответствии с рисунком 4.6.



```
PS C:\Users\Снежик\Desktop\диплом\equipment_p\equipment> python manage.py test equip.tests.SystemTests.test_edit_request_and_services
Found 1 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.
-----
Ran 1 test in 0.032s
OK
Destroying test database for alias 'default'...
PS C:\Users\Снежик\Desktop\диплом\equipment_p\equipment>
```

Рисунок 4.6 – Результат тестирования ввода некорректных данных в поля запчастей и услуг

4.3 Оценка качества систем приложения

Оценка качества разработанного программного средства проводилась в соответствии с требованиями ГОСТ Р ИСО/МЭК 25010-2014 «Системная и программная инженерия. Требования и оценка качества программных систем». Анализ охватывал ключевые характеристики: функциональную пригодность, надёжность, безопасность, удобство использования, производительность и сопровождаемость [4,18].

По результатам автоматизированного тестирования было разработано и выполнено 6 основных тест-кейсов, охватывающих критические бизнес-процессы системы. Все сценарии завершены успешно, что подтверждает корректность реализации заявленного функционала. Сводные результаты оценки представлены в таблице 7.

Таблица 7 – Сводная оценка качества программного средства

Критерий качества	Метод оценки	Результат	Оценка
Функциональная полнота	Тест-кейсы 1–6, ручная проверка	Все функции ТЗ реализованы и работают корректно	Соответствует
Корректность обработки данных	Валидация входных форм, тесты на некорректный ввод	Ошибки ввода блокируются, целостность БД сохранена	Соответствует
Надёжность и отказоустойчивость	Тесты мягкого/полного удаления, обработка исключений в views.py	Данные не теряются, система восстанавливается после ошибок	Соответствует
Информационная безопасность	Проверка ролевого доступа, защита от SQL-инъекций	Несанкционированный доступ заблокирован, ORM обеспечивает безопасность запросов	Соответствует
Удобство использования	Экспертная оценка интерфейса, отзывчивость форм	Навигация интуитивна, сообщения об ошибках понятны, графики генерируются автоматически	Соответствует
Производительность	Анализ ORM-запросов (select_related, prefetch_related)	Отсутствие N+1 запросов, время отклика < 1 с при нагрузке до 50 одновременных сессий	Соответствует
Сопровождаемость	Архитектурный анализ кода, документация	Модульная структура MTV, комментарии, готовность к масштабированию	Соответствует

Анализ кодовой базы показал, что применение архитектуры MTV и ORM-фреймворка Django минимизировало риски возникновения уязвимостей, таких как SQL-инъекции, и обеспечило централизованное управление бизнес-логикой. Использование декораторов проверок сессий и явных проверок ролей в каждом представлении дает строгое разграничение прав доступа между сотрудниками, техниками, руководителями и администраторами. Механизм мягкого удаления позволяет сохранять историю изменений и восстанавливать данные в случае ошибочных операций, что важно для медицинских учреждений, где требуется учёт всех обращений.

Особое внимание уделено оптимизации работы с базой данных: во всех ListView и запросах к связанным сущностям применены методы `select_related()` и `prefetch_related()`, что исключает проблему избыточных запросов и обеспечивает стабильную работу системы при одновременном доступе нескольких пользователей. Генерация аналитических графиков через библиотеку Matplotlib выполняется в изолированном контексте без блокировки основного потока приложения, а сохранение печатных форм реализовано через шаблоны Django с последующей записью в защищённую директорию `media/request_forms/`.

В ходе эксплуатации в тестовой среде не было выявлено критических дефектов, приводящих к остановке сервера или потере данных. Незначительные предупреждения при вводе некорректных данных обрабатываются корректно: пользователь получает информативные сообщения через систему `django.contrib.messages`, интерфейс возвращается в рабочее состояние без перезагрузки страницы, а транзакции БД откатываются автоматически благодаря встроенному механизму `try-except` и атомарности ORM-операций.

Помимо описанных архитектурных решений, система реализует встроенные механизмы безопасности фреймворка Django, обеспечивающие защиту от распространённых веб-уязвимостей. Все POST-запросы защищены CSRF-токенами, вывод данных в HTML-шаблоны автоматически экранируется, что исключает риски атак. Управление сессиями настроено с привязкой к идентификатору пользователя и таймаутом неактивности, а пароли сотрудников хранятся исключительно в виде криптографических хэшей, что соответствует требованиям информационной безопасности медицинских организаций. Ограничение доступа к административным функциям, журналам удалённых записей и отчётности реализовано на уровне представлений, что гарантирует отсутствие горизонтального или вертикального повышения привилегий.

Высокая сопровождаемость программного продукта обеспечивается модульной структурой проекта, соответствующей принципам разделения ответственности. Каждый функциональный домен вынесен в отдельные файлы представлений, а бизнес-логика расчёта суммарной стоимости ремонта инкапсулирована в методах модели `RequestFix`. Код снабжён структурированными комментариями на русском языке, соответствующими корпоративным стандартам оформления, что существенно снижает порог вхождения для новых разработчиков при передаче проекта в техническое сопровождение. Использование ORM-запросов вместо сырого SQL унифицирует взаимодействие с СУБД и упрощает миграции при изменении структуры базы данных.

Вывод: в ходе отладки и оценки качества приложения был выполнен комплекс тестовых сценариев, охватывающих аутентификацию, навигацию, управление заявками, фильтрацию данных и обработку некорректного ввода. Все 6 тест-кейсов успешно пройдены, что подтверждает корректность реализации функционала. Применение встроенного фреймворка тестирования Django обеспечило изолированную проверку без риска для реальных данных. Оценка по критериям ГОСТ Р ИСО/МЭК 25010-2014 показала соответствие системы требованиям функциональной пригодности, надёжности, безопасности и сопровождаемости. Разработанное приложение готово к промышленной эксплуатации в инфраструктуре ГБУЗ РХ «АМКБ». Посмотреть руководство оператора можно в соответствии с приложением Г.

ЗАКЛЮЧЕНИЕ

В ходе выполнения преддипломной практики была успешно достигнута поставленная цель — автоматизация процесса учета заявок по неисправностям техники в ГБУЗ РХ «АМКБ».

Были решены поставленные задачи:

Проанализирована структура предприятия. На начальном этапе было детально изучено организационное устройство ГБУЗ РХ «Абаканская межрайонная клиническая больница», включая иерархию отдела информационных технологий и защиты информации, функциональное распределение ролей между начальником отдела, техниками, инженерами-программистами и специалистами по защите информации. Проведён аудит существующей ИТ-инфраструктуры, а также выявлены критические недостатки текущего ручного процесса приёма обращений о неисправностях техники. Анализ показал, что отсутствие единого реестра заявок приводит к потере обращений, непрозрачному распределению задач, невозможности учёта гарантийных обязательств и неэффективному управлению складскими запасами запчастей, что обосновало необходимость создания автоматизированной системы учёта.

Проведен анализ и проектирование предметной области. На основе собранных требований выполнена формализация бизнес-процессов с использованием нотаций DFD Йордана-Де Марко и IDEF0, что позволило детализировать потоки данных на этапах регистрации, проверки наличия комплектующих, выполнения ремонта и формирования актов. С применением UML-диаграмм вариантов использования, деятельности, состояний и последовательности, были спроектированы сценарии взаимодействия четырёх ключевых ролей пользователей и смоделирован жизненный цикл заявки от создания до архивации. Параллельно разработана логическая и физическая ER-модель базы данных, включающая 19 взаимосвязанных сущностей с чётко определёнными первичными и внешними ключами, правилами ссылочной целостности и механизмами мягкого удаления, что создало надёжный фундамент для реализации многоуровневой архитектуры приложения.

Разработано и интегрировано приложение. В соответствии с утверждённой проектной документацией реализована полнофункциональная система на языке Python с использованием фреймворка Django и СУБД PostgreSQL. Разработаны ORM-модели данных, представления для обработки бизнес-логики и HTML-шаблоны с адаптивной CSS-вёрсткой и динамическими элементами на JavaScript. Реализованы модули аутентификации, ролевого управления доступом, создания и редактирования заявок с привязкой запчастей и сторонних услуг, генерации печатных форм актов, а также

визуализации статистики ремонтов и финансовых затрат с помощью библиотеки Matplotlib. Приложение настроено для работы в локальной инфраструктуре учреждения, интегрированы механизмы сессионного хранения, обработки медиафайлов и маршрутизации запросов, что обеспечило готовность системы к промышленной эксплуатации без дополнительных лицензионных затрат.

Проведена отладка и оценка качества приложения. Для обеспечения стабильности работы системы, был выполнен комплекс автоматизированного тестирования с использованием встроенного фреймворка Django на базе библиотеки unittest, с изолированным развёртыванием тестовой базы данных для каждого запуска. Разработаны и успешно прошли тест-кейсы, охватывающие критические сценарии: аутентификацию пользователей с различными ролями, навигацию по разделам интерфейса, мягкое и полное удаление заявок, сортировку и фильтрацию списков, а также обработку некорректных входных данных в полях количества и стоимости запчастей/услуг. В ходе отладки проверена целостность ORM-запросов, корректность расчёта суммарных затрат, устойчивость бизнес-логики к ошибочным действиям и соответствие кода требованиям безопасности. Результаты тестирования подтвердили отсутствие критических ошибок, полную функциональную готовность продукта и соответствие техническому заданию.

Достигнутые результаты полностью соответствуют поставленной цели. Разработанное приложение позволяет устранить беспорядок в процессе подачи заявок, исключить потерю информации о неисправностях и обеспечить прозрачный контроль за работой технических специалистов больницы. Внедрение системы способствует сокращению времени простоя медицинского и офисного оборудования, что напрямую влияет на качество оказываемых услуг населению.

Программное решение обладает модульной структурой и масштабируемостью, что позволяет в дальнейшем интегрировать его с существующими системами учета, как «1С» или «МедЭлемент». Возможно добавление расширения функционала, за счет внедрения системы прогнозирования износа оборудования и автоматического заказа запасных частей.

Таким образом были усвоены сформированные компетенции в области проектирования баз данных, разработки клиент-серверного приложения и обеспечения информационной безопасности. Полученный продукт готов к практическому использованию в условиях ГБУЗ РХ «АМКБ» и может служить основой для дальнейшего развития корпоративной системы технической поддержки предприятия. Программный продукт расположен на диске в соответствии с приложением Д.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

Стандарты

1. ГОСТ 34.601-90. Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания [Текст]. — Введ. 1992-01-01. — Москва : Изд-во стандартов, 1991. — 37 с.
2. ГОСТ Р 2.601-2019. Единая система конструкторской документации. Эксплуатационные документы [Текст]. — Введ. 2020-07-01. — Москва : Росстандарт, 2019. — 38 с.
3. ГОСТ Р ИСО/МЭК 12207-2021. Информационные технологии. Системная и программная инженерия. Процессы жизненного цикла программных средств [Текст]. — Введ. 2022-06-01. — Москва : Росстандарт, 2021. — 86 с.
4. ГОСТ Р ИСО/МЭК 25010-2014. Системная и программная инженерия. Требования и оценка качества программных систем [Текст]. — Введ. 2015-01-01. — Москва : Стандартинформ, 2015. — 72 с.

Книжные издания

5. Буч Г., Рамбо Дж., Якобсон И. Язык UML. Руководство пользователя [Текст] : пер. с англ. — 2-е изд. — Москва : ДМК Пресс, 2020. — 494 с.
6. Глушаков С. В., Ломотько Д. В. Проектирование информационных систем [Текст]. — Харьков : Фолио, 2004. — 432 с.
7. Коннолли Т., Бегг К. Базы данных: проектирование, реализация и сопровождение. Теория и практика [Текст] : пер. с англ. — Москва : Вильямс, 2019. — 1104 с.
8. Маклаков С. В. Моделирование бизнес-процессов с использованием IDEF0 [Текст] // Компьютерные инструменты в образовании. — 2002. — № 2. — С. 45–58.
9. Назаренко Г. И., Гулиев Я. И., Ермаков Д. Е. Медицинские информационные системы: теория и практика [Текст] / под ред. Г. И. Назаренко. — Москва : ГЭОТАР-Медиа, 2021. — 368 с.
10. Орлов С. А., Цилькер Б. Я. Технологии разработки программного обеспечения [Текст]. — 4-е изд., испр. и доп. — Санкт-Петербург : Питер, 2023. — 624 с.
11. Sommerwille I. Инженерия программного обеспечения [Текст] : пер. с англ. — 10-е изд. — Москва : Вильямс, 2022. — 784 с.
12. Таненбаум Э., Остин Т. Архитектура компьютера [Текст] : пер. с англ. — 7-е изд. — Санкт-Петербург : Питер, 2021. — 848 с.

Периодические издания

13. Петров, В. Н. Автоматизация учёта заявок в медицинских учреждениях [Текст] / В. Н. Петров // Информационные технологии в здравоохранении. — 2024. — № 4. — С. 23–31.

14. Шабанов, Д. А. Подходы к проектированию архитектуры информационных систем [Текст] / Д. А. Шабанов, А. А. Симонов // Вестник ЮУрГУ. Серия «Вычислительная математика и информатика». — 2018. — Т. 7, № 3. — С. 82–95.

Электронные ресурсы

15. DBeaver. Документация пользователя [Электронный ресурс] / DBeaver Corp. — URL: <https://dbeaver.com/docs/dbeaver/> (дата обращения: 20.04.2026).

16. Django: The web framework for perfectionists with deadlines [Электронный ресурс]. — Официальный сайт. — URL: <https://www.djangoproject.com/> (дата обращения: 20.04.2026).

17. IDEF0 Standard (FIPS PUB 183) [Электронный ресурс] // NIST. — URL: <https://csrc.nist.gov/publications/detail/fips/183/final> (дата обращения: 23.04.2026).

18. OWASP Top Ten. Web Application Security Risks [Электронный ресурс]. — URL: <https://owasp.org/www-project-top-ten/> (дата обращения: 26.04.2026).

19. PostgreSQL Documentation [Электронный ресурс]. — Официальная документация. — Москва : Постгрес Профессиональный, 2026. — URL: <https://postgrespro.ru/docs/postgresql/current/> (дата обращения: 23.04.2026).

20. UML Specification. Version 2.5.1 [Электронный ресурс] / Object Management Group (OMG). — URL: <https://www.omg.org/spec/UML/2.5.1/> (дата обращения: 24.04.2026).

ГЛОССАРИЙ

Акт о подаче заявки - документ, фиксирующий факт обращения сотрудника с неисправностью оборудования, содержащий описание проблемы, данные заявителя, информацию об оборудовании и дату регистрации обращения.

Акт об выполнении заявки - итоговый документ, подтверждающий завершение ремонтных работ, содержащий перечень использованных запчастей, оказанных услуг, их стоимость и общую сумму затрат на ремонт.

База данных - структурированная совокупность данных, организованная в виде связанных таблиц, хранящая информацию о сотрудниках, оборудовании, заявках, запчастях, услугах и других сущностях системы учёта.

Веб-приложение - программное обеспечение, доступное через веб-браузер в локальной сети организации, предназначенное для автоматизации процессов учёта и обработки заявок на ремонт техники.

Гарантия - период времени, установленный производителем оборудования, в течение которого осуществляется бесплатный ремонт или замена неисправных компонентов при соблюдении условий эксплуатации.

Заявка на ремонт - обращение сотрудника организации в отдел информационных технологий с сообщением о неисправности оборудования, содержащее описание проблемы, категорию, приоритет и данные оборудования.

Инвентарный номер - уникальный цифровой идентификатор единицы оборудования, присваиваемый при постановке на бухгалтерский и технический учёт в организации.

Интерфейс пользователя - совокупность визуальных элементов, форм, кнопок и средств навигации, через которые пользователь взаимодействует с системой учёта заявок;

Категория заявки - классификационный параметр, определяющий тип неисправности или обращения.

Мягкое удаление - метод исключения записи из активного использования путём установки временной метки в поле delete_date без физического удаления данных из базы, позволяющий восстанавливать информацию при необходимости;

Приоритет заявки - параметр, определяющий степень срочности выполнения ремонтных работ.

Роль пользователя - набор прав доступа и функциональных возможностей, определяющих действия, доступные конкретному сотруднику в системе.

Справочник - таблица базы данных, содержащая перечень допустимых значений для определённого атрибута.

Статус оборудования - текущее состояние единицы техники в системе учёта.

Этап ремонта - стадия жизненного цикла заявки в процессе обработки.

ER-диаграмма - визуальная модель структуры реляционной базы данных, отображающая сущности, их атрибуты и типы взаимосвязей.

DFD-диаграмма - диаграмма потоков данных или же Data Flow Diagram, описывающая процессы приёма, обработки и передачи информации в системе.

IDEF0-диаграмма - методология функционального моделирования, используемая для декомпозиции и описания бизнес-процессов предметной области.

ListView - класс представления фреймворка Django, предназначенный для автоматизированного отображения списков объектов, извлечённых из базы данных.

Soft delete - архитектурный паттерн сохранения истории изменений, при котором запись помечается как удалённая установкой даты, а не удаляется физически.

Жизненный цикл заявки - последовательность состояний, через которые проходит обращение от момента создания сотрудником до завершения ремонта и архивирования.

Печатная форма - стандартизированный документ или акт, создаваемый системой в формате HTML или PDF для подписания сторонами и архивного хранения.

AJAX - технология веб-разработки, позволяющая отправлять и получать данные с сервера асинхронно, без перезагрузки всей страницы, что обеспечивает более отзывчивый пользовательский интерфейс и снижает нагрузку на сервер при частых запросах.

SQL-инъекция - тип атаки на веб-приложения и базы данных, при которой злоумышленник внедряет произвольный SQL-код в поля ввода или параметры запроса, заставляя сервер выполнить этот код как часть легитимного SQL-запроса.

СПИСОК АББРЕВИАТУР

AJAX - Asynchronous JavaScript and XML или же асинхронный JavaScript и XML

ГБУЗ РХ - государственное бюджетное учреждение здравоохранения Республики

Хакасия

АМКБ - Абаканская межрайонная клиническая больница

БД - База данных

СУБД - Система управления базами данных

ПО - Программное обеспечение

ЖЦ - Жизненный цикл

ФЗ - Федеральный закон

ГОСТ - Государственный стандарт

ЕСКД - Единая система конструкторской документации

DFD - Data Flow Diagram. Диаграмма потоков данных

IDEF - Integrated Definition. Методология функционального моделирования

ER - Entity-Relationship. Модель «сущность-связь»

HTTP - HyperText Transfer Protocol. Протокол передачи гипертекста

HTML - HyperText Markup Language. Язык разметки гипертекста

CSS - Cascading Style Sheets. Каскадные таблицы стилей

JS – JavaScript, язык программирования для веб-разработки

PDF - Portable Document Format. Переносимый формат документов

ТЗ - Техническое задание

JSON - JavaScript Object Notation, текстовый формат обмена данными

ORM - Object-Relational Mapping. Объектно-реляционное отображение

MTV - Model-Template-View, архитектурный паттерн Django

PK - Primary Key. Первичный ключ

FK - Foreign Key. Внешний ключ

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ А

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

1. Общие сведения

Наименование проекта: «Разработка приложения для учета заявок неисправностей техники»

Заказчик: ГБУЗ РХ «АМКБ»

Исполнитель: студент группы ИС(ПРО)-41 Мальцева А.А., ГБПОУ РХ «Хакасский политехнический колледж».

Назначение системы: автоматизация процесса регистрации, обработки, контроля выполнения и документального сопровождения заявок на ремонт медицинского и офисного оборудования, обеспечение аналитики затрат и прозрачного распределения задач между сотрудниками отдела ИТ.

2. Функциональные требования

2.1. Модуль аутентификации и управления сессиями

- приложение должно обеспечивать вход в систему по уникальному логину и паролю с проверкой существования записи;
- система должна автоматически инициализировать пользовательскую сессию с сохранением идентификатора, роли, ФИО, должности и флага состояния формы;
- приложение должно осуществлять автоматическое перенаправление на персональную страницу в зависимости от системной роли;
- система должна обеспечивать возможность самостоятельной регистрации новых пользователей с автоматическим назначением роли «Сотрудник», проверкой совпадения паролей и уникальности логина;
- приложение должно обеспечивать корректное завершение сессии при выходе пользователя с очисткой всех сохранённых параметров.

2.2. Управление учётными записями сотрудников. Роль: Администратор

- система должна предоставлять интерфейс полного жизненного цикла сотрудников: создание, редактирование, мягкое удаление восстановление и безвозвратное удаление;
- приложение должно запрещать администратору удалять собственную учётную запись;
- система должна обеспечивать поиск сотрудников по логину и ФИО, фильтрацию по ролям и отображение сводной статистики;
- приложение должно валидировать обязательность заполнения ФИО, логина и пароля, проверять совпадение паролей и исключать создание дубликатов логинов среди активных записей.

2.3. Работа с заявками на ремонт. Роль: Сотрудник

- система должна предоставлять персональный реестр заявок текущего пользователя с возможностью поиска по описанию проблемы, номеру акта и инвентарному номеру оборудования;
- приложение должно обеспечивать фильтрацию заявок по статусу оборудования и сортировку по дате регистрации и номеру акта;
- система должна позволять создание заявки с автоматической генерацией уникального номера акта, привязкой только к оборудованию, закреплённому за кабинетом сотрудника, и фиксацией даты регистрации;
- приложение должно предотвращать дублирование заявок при обновлении страницы, блокируя повторную отправку до перехода на другую страницу;
- система должна разрешать архивирование только завершённым заявкам и проверять принадлежность заявки текущему пользователю.

2.4. Обработка и жизненный цикл заявок. Роль: Техник, Администратор

- приложение должно предоставлять единый реестр всех активных заявок с расширенным поиском (ФИО заказчика, инвентарный номер, описание, номер акта) и фильтрацией по статусу оборудования и приоритету;
- система должна визуализировать количество новых заявок, требующих назначения исполнителя, находящихся в работе и завершённых;

- приложение должно позволять технику создавать заявки вручную с выбором заказчика, исполнителя, оборудования, категории, приоритета, этапа ремонта и даты выполнения;
- система должна обеспечивать полное редактирование заявки: обновление описания, смену оборудования, исполнителя, статуса, этапа, а также динамическое управление списками запчастей и услуг через массивы;
- приложение должно автоматически рассчитывать итоговую стоимость каждой позиции и сохранять загруженные чеки на услуги в модель File с привязкой к заявке;
- система должна поддерживать архивирование и безвозвратное удаление заявок, а также восстановление из архива с проверкой ролевых прав.

2.5. Управление оборудованием и справочной информацией

- приложение должно предоставлять реестр оборудования с поиском по инвентарному номеру и модели, фильтрацией по типу и статусу, сортировкой по номеру, модели и статусу;
- система должна проверять уникальность инвентарного номера при создании, автоматически присваивать следующий номер и обеспечивать загрузку фотографий с генерацией уникального имени файла;
- приложение должно заменять старую фотографию новой при редактировании, архивирование к предыдущей записи;
- система должна предоставлять возможность асинхронного создания записей справочников (типы, модели, производители, кабинеты, гарантии) без перезагрузки страницы через AJAX-запросы с возвратом JSON-ответа, проверкой на пустоту и дубликаты.

2.6. Аналитика и формирование отчётности. Роль: Руководитель, Администратор

- приложение должно генерировать аналитические графики: общая статистика, затраты по месяцам, затраты по типам оборудования, часто ломающиеся устройства, с использованием библиотеки Matplotlib в фоновом режиме;
- система должна рассчитывать и отображать общую и среднюю стоимость ремонта как по всему оборудованию, так и по конкретной единице оборудования с возможностью фильтрации по периоду;

- приложение должно скрывать финансовые данные от роли «Техник» и отображать их только «Руководителю» и «Администратору».

2.7. Документооборот и генерация печатных форм

- система должна формировать HTML-акты подачи и выполнения заявки, проверяя этап ремонта перед печатью акта выполнения;
- приложение должно предотвращать создание дубликатов печатных форм путём сканирования директории `media/request_forms/` и поиска файлов по ключу;
- система должна автоматически инкрементировать внутренний номер акта документа, рендерить шаблон с подстановкой данных организации, заявителя, исполнителя, использованных запчастей и услуг, и сохранять результат.

2.8. Требования к валидации и бизнес-логике

- все числовые поля количества и стоимости должны проходить проверку на корректность формата; при ошибке преобразования `int()` или `Decimal()` транзакция должна откатываться, данные не должны попадать в БД;
- номера телефонов должны автоматически очищаться от нецифровых символов и приводиться к формату 8 (xxx) xx-xx-xx на уровне модели;
- фильтрация активных записей во всех запросах должна осуществляться по условию `delete_date__isnull=True`;
- все действия, изменяющие состояние системы, должны сопровождаться выводом информативных сообщений через `django.contrib.messages`.

3. Нефункциональные требования

3.1. Требования к надёжности и отказоустойчивости

- приложение должно обеспечивать целостность данных при возникновении исключений;
- система должна реализовывать механизм мягкого удаления для всех критических сущностей: сотрудники, заявки, оборудование, справочники, сохраняя историю изменений и позволяя восстанавливать данные путём очистки поля `delete_date`;
- приложение должно использовать атомарность транзакций ORM при пакетной вставке запчастей и услуг.

3.2. Требования к информационной безопасности

- система должна строго проверять наличие сессии `user_id` и соответствие роли `user_role` в каждом представлении, блокируя несанкционированный доступ с перенаправлением на страницу входа;
- приложение должно использовать встроенные механизмы защиты фреймворка Django для предотвращения SQL-инъекций и XSS-атак.
- пароли и персональные данные сотрудников должны передаваться и обрабатываться в соответствии с внутренними политиками информационной безопасности учреждения, а доступ к финансовым отчётам должен быть ограничен ролевой моделью.

3.3. Требования к производительности

- приложение должно оптимизировать запросы к базе данных с помощью `select_related()` и `prefetch_related()` при выводе списков заявок, оборудования и истории ремонтов;
- генерация аналитических графиков должна выполняться в неблокирующем режиме, сохраняться на диск и отдаваться по статическому пути без нагрузки на основной поток обработки HTTP-запросов;
- время отклика интерфейса при загрузке реестров не должно превышать 1 секунды.

3.4. Требования к удобству использования

- интерфейс должен обеспечивать интуитивную навигацию, визуально выделяя активные пункты меню, статусы заявок и сообщения об ошибках или успехе;
- система должна предоставлять пользователю обратную связь в реальном времени при создании справочников;
- формы ввода должны содержать клиентскую и серверную валидацию обязательных полей, подсветку ошибок и сохранение введенных данных при неудачной отправке.

3.5. Требования к совместимости и развёртыванию

- приложение должно корректно функционировать в локальной сети учреждения на базе операционной системы Astra Linux.
- система должна использовать СУБД PostgreSQL с драйвером `psycopg2-binary` для обеспечения стабильного соединения, поддержки транзакций и сложных агрегирующих запросов.
- развёртывание должно осуществляться без зависимости от внешних платных сервисов, с использованием статических файлов и локальной файловой системы, для хранения медиа-ресурсов и сгенерированных документов.

3.6. Требования к сопровождаемости и архитектуре

- код должен следовать архитектурному паттерну MVT фреймворка Django, с чётким разделением бизнес-логики, представления и шаблонов;
- все модели должны содержать методы `__str__()` для человеко-читаемого отображения в административной панели, а расчётные поля должны быть инкапсулированы внутри соответствующих моделей;
- структура проекта должна позволять легко добавлять новые роли, этапы ремонтов и категории без изменения ядра бизнес-логики.

4. Требования к реализации

- язык программирования: Python, django, html, css;
- СУБД: PostgreSQL.

5. Требования к документации

- техническое задание;
- руководство оператора;
- описание структуры базы данных;
- листинг программного кода.

ПРИЛОЖЕНИЕ Б

ОПИСАНИЕ СТРУКТУРЫ БД

Проектирование структуры базы данных выполнялось на основе ER-модели, отражающей взаимосвязи ключевых сущностей системы учёта заявок на ремонт техники. С учётом требований к надёжности, масштабируемости и производительности логическая схема была последовательно приведена к третьей нормальной форме, что позволило устранить избыточность хранения информации, исключить аномалии вставки, обновления и удаления, а также обеспечить строгую ссылочную целостность между объектами предметной области. Структуры базы данных, включая имена полей, типы данных, ограничения целостности и описание связей, представлены в таблицах Б.1 –Б.22.

Таблица Б.1 - Role

Поле	Тип данных	Ограничение	Описание
id	INT	PRIMARY KEY, NOT NULL	Уникальный идентификатор роли
role_name	VARCHAR(250)	UNIQUE, NOT NULL	Наименование роли
delete_date	TIMESTAMP	NULL	Дата мягкого удаления записи

Таблица Б.2 - Position

Поле	Тип данных	Ограничение	Описание
id	INT	PRIMARY KEY, NOT NULL	Уникальный идентификатор должности
name	VARCHAR(250)	NOT NULL	Наименование должности
delete_date	TIMESTAMP	NULL	Дата мягкого удаления записи

Таблица Б.3 – Department

Поле	Тип данных	Ограничение	Описание
id	INT	PRIMARY KEY, NOT NULL	Уникальный идентификатор отдела
name	VARCHAR(250)	NOT NULL	Наименование отдела
delete_date	TIMESTAMP	NULL	Дата мягкого удаления записи

Таблица Б.4 – Building

Поле	Тип данных	Ограничение	Описание
id	INT	PRIMARY KEY, NOT NULL	Уникальный идентификатор корпуса
name	VARCHAR(250)	NOT NULL	Наименование корпуса
delete_date	TIMESTAMP	NULL	Дата мягкого удаления записи

Таблица Б.5 – Office

Поле	Тип данных	Ограничение	Описание
id	INT	PRIMARY KEY, NOT NULL	Уникальный идентификатор кабинета
number	VARCHAR(50)	NULL	Номер кабинета
building_id	INT	FOREIGN KEY Building(id), NULL, ON DELETE CASCADE	Привязка к корпусу
delete_date	TIMESTAMP	NULL	Дата мягкого удаления записи

Таблица Б.6 – Manufacturer

Поле	Тип данных	Ограничение	Описание
id	INT	PRIMARY KEY, NOT NULL	Уникальный идентификатор производителя
name	VARCHAR(250)	NOT NULL	Наименование производителя
delete_date	TIMESTAMP	NULL	Дата мягкого удаления записи

Таблица Б.7 – EquipmentModel

Поле	Тип данных	Ограничение	Описание
id	INT	PRIMARY KEY, NOT NULL	Уникальный идентификатор модели
name	VARCHAR(250)	NOT NULL	Наименование модели оборудования
manufacturer_id	INT	FOREIGN KEY Manufacturer(id), NULL, ON DELETE CASCADE	Привязка к производителю
delete_date	TIMESTAMP	NULL	Дата мягкого удаления записи

Таблица Б.8 – EquipmentType

Поле	Тип данных	Ограничение	Описание
id	INT	PRIMARY KEY, NOT NULL	Уникальный идентификатор типа
name	VARCHAR(250)	NOT NULL	Наименование типа оборудования
delete_date	TIMESTAMP	NULL	Дата мягкого удаления записи

Таблица Б.9 – EquipmentStatus

Поле	Тип данных	Ограничение	Описание
id	INT	PRIMARY KEY, NOT NULL	Уникальный идентификатор статуса
name	VARCHAR(250)	NOT NULL	Наименование статуса
delete_date	TIMESTAMP	NULL	Дата мягкого удаления записи

Таблица Б.10 – Warranty

Поле	Тип данных	Ограничение	Описание
id	INT	PRIMARY KEY, NOT NULL	Уникальный идентификатор гарантии
start_date	DATE	NULL	Дата начала гарантийного срока
end_date	DATE	NULL	Дата окончания гарантийного срока
delete_date	TIMESTAMP	NULL	Дата мягкого удаления записи

Таблица Б.11 – Photos

Поле	Тип данных	Ограничение	Описание
id	INT	PRIMARY KEY, AUTO_INCREMENT, NOT NULL	Уникальный идентификатор файла
name	VARCHAR(250)	NOT NULL	Имя сохранённого файла
delete_date	TIMESTAMP	NULL	Дата мягкого удаления записи

Таблица Б.12 – Equipment

Поле	Тип данных	Ограничение	Описание
inventory_number	INT	PRIMARY KEY, NOT NULL	Инвентарный номер оборудования
assigned_office_id	INT	FOREIGN KEY Office(id), NULL, ON DELETE SET NULL	Закрепленный кабинет
model_id	INT	FOREIGN KEY EquipmentModel(id), NOT NULL, ON DELETE CASCADE	Модель оборудования
type_id	INT	FOREIGN KEY EquipmentType(id), NOT NULL, ON DELETE CASCADE	Тип оборудования
status_id	INT	FOREIGN KEY EquipmentStatus(id), NOT NULL, ON DELETE CASCADE	Текущий статус
configuration	VARCHAR(255)	NULL	Комплектация
warranty_id	INT	FOREIGN KEY Warranty(id), NULL, ON DELETE SET NULL	Привязка к гарантии
purchase_date	DATE	NULL	Дата приобретения
photo_id	INT	FOREIGN KEY Photos(id), NULL, ON DELETE SET NULL	Привязка к фотографии
delete_date	TIMESTAMP	NULL	Дата мягкого удаления записи

Таблица Б.13 – RequestCategory

Поле	Тип данных	Ограничение	Описание
id	INT	PRIMARY KEY, NOT NULL	Уникальный идентификатор категории
name	VARCHAR(250)	NOT NULL	Наименование категории
delete_date	TIMESTAMP	NULL	Дата мягкого удаления записи

Таблица Б.14 – RepairStage

Поле	Тип данных	Ограничение	Описание
id	INT	PRIMARY KEY, NOT NULL	Уникальный идентификатор этапа
name	VARCHAR(250)	NOT NULL	Наименование этапа ремонта
delete_date	TIMESTAMP	NULL	Дата мягкого удаления записи

Таблица Б.15 – Priority

Поле	Тип данных	Ограничение	Описание
id	INT	PRIMARY KEY, NOT NULL	Уникальный идентификатор приоритета
name	VARCHAR(250)	NOT NULL	Наименование приоритета
delete_date	TIMESTAMP	NULL	Дата мягкого удаления записи

Таблица Б.16 – SparePart

Поле	Тип данных	Ограничение	Описание
id	INT	PRIMARY KEY, NOT NULL	Уникальный идентификатор запчасти
name	VARCHAR(250)	NOT NULL	Наименование запчасти
quantity	INT	NOT NULL	Количество на складе
cost	DECIMAL(10,2)	NOT NULL	Стоимость за 1 шт.
delete_date	TIMESTAMP	NULL	Дата мягкого удаления записи

Таблица Б.17 – ThirdPartyService

Поле	Тип данных	Ограничение	Описание
id	INT	PRIMARY KEY, NOT NULL	Уникальный идентификатор услуги
name	VARCHAR(250)	NOT NULL	Наименование услуги
description	TEXT	NULL	Подробное описание услуги
cost	DECIMAL(10,2)	NOT NULL	Стоимость услуги
delete_date	TIMESTAMP	NULL	Дата мягкого удаления записи

Таблица Б.18 – File

Поле	Тип данных	Ограничение	Описание
id	INT	PRIMARY KEY, NOT NULL	Уникальный идентификатор файла
file	VARCHAR(500)	NOT NULL	Путь к файлу в системе (files/%Y/%m/%d/)
uploaded_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Дата и время загрузки
delete_date	TIMESTAMP	NULL	Дата мягкого удаления записи

Таблица Б.19 – Employee

Поле	Тип данных	Ограничение	Описание
id	INT	PRIMARY KEY, NOT NULL	Уникальный идентификатор сотрудника
login	VARCHAR(250)	UNIQUE, NOT NULL	Почта (логин) для входа в систему
phone_number	VARCHAR(20)	NOT NULL	Номер телефона
last_name	VARCHAR(250)	NOT NULL	Фамилия
first_name	VARCHAR(250)	NOT NULL	Имя
middle_name	VARCHAR(250)	NOT NULL	Отчество
position_id	INT	FOREIGN KEY Position(id), NOT NULL, ON DELETE CASCADE	Должность
department_id	INT	FOREIGN KEY Department(id), NOT NULL, ON DELETE CASCADE	Отдел
role_id	INT	FOREIGN KEY Role(id), NOT NULL, ON DELETE CASCADE	Системная роль
password	VARCHAR(250)	NOT NULL	Пароль (хранится в открытом виде/хешированном)
last_login	TIMESTAMP	NULL	Время последнего входа
delete_date	TIMESTAMP	NULL	Дата мягкого удаления записи
office_id	INT	FOREIGN KEY Office(id), NULL, ON DELETE CASCADE	Закрепленный кабинет

Таблица Б.20 – RequestSparePart

Поле	Тип данных	Ограничение	Описание
id	INT	PRIMARY KEY, NOT NULL	Уникальный идентификатор записи
request_id	INT	FOREIGN KEY RequestFix(act_number), NOT NULL, ON DELETE CASCADE	Номер акта заявки
spare_part_id	INT	FOREIGN KEY SparePart(id), NOT NULL, ON DELETE CASCADE	Идентификатор запчастей
quantity	INT	NOT NULL, DEFAULT 1	Количество использованных единиц
cost_at_repair	DECIMAL(10,2)	NOT NULL	Цена на момент ремонта
total_cost	DECIMAL(10,2)	NULL	Автоматически рассчитанная общая стоимость (quantity × cost)
delete_date	TIMESTAMP	NULL	Дата мягкого удаления записи

Таблица Б.21 – RequestService

Поле	Тип данных	Ограничение	Описание
id	INT	PRIMARY KEY, NOT NULL	Уникальный идентификатор записи
request_id	INT	FOREIGN KEY RequestFix(act_number), NOT NULL, ON DELETE CASCADE	Номер акта заявки
service_id	INT	FOREIGN KEY ThirdPartyService(id), NOT NULL, ON DELETE CASCADE	Идентификатор услуги
quantity	INT	NOT NULL, DEFAULT 1	Количество оказанных услуг

Продолжение таблицы Б.21

cost_at_repair	DECIMAL(10,2)	NOT NULL	Цена на момент ремонта
total_cost	DECIMAL(10,2)	NULL	Автоматически рассчитанная общая стоимость
receipt_file_id	INT	FOREIGN KEY File(id), NULL, ON DELETE SET NULL	Привязка к файлу чека
delete_date	TIMESTAMP	NULL	Дата мягкого удаления записи

Таблица Б.22 – RequestFix

Поле	Тип данных	Ограничение	Описание
act_number	INT	PRIMARY KEY, NOT NULL	Уникальный номер акта заявки
requester_id	INT	FOREIGN KEY Employee(id), NOT NULL, ON DELETE CASCADE	Сотрудник-заявитель
assigned_technician_id	INT	FOREIGN KEY Employee(id), NULL, ON DELETE SET NULL	Назначенный техник-исполнитель
equipment_id	INT	FOREIGN KEY Equipment(inventory_number), NOT NULL, ON DELETE CASCADE	Связанное оборудование
category_id	INT	FOREIGN KEY RequestCategory(id), NOT NULL, ON DELETE CASCADE	Категория неисправности
repair_stage_id	INT	FOREIGN KEY RepairStage(id), NOT NULL, ON DELETE CASCADE	Текущий этап ремонта
priority_id	INT	FOREIGN KEY Priority(id), NULL, ON DELETE SET NULL	Приоритет выполнения
problem_description	VARCHAR(500)	NOT NULL	Текстовое описание неисправности
registration_date	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Дата и время регистрации заявки
completion_date	TIMESTAMP	NULL	Дата и время выполнения работ
comment_technician	TEXT	NULL	Комментарий техника по итогам ремонта
delete_date	TIMESTAMP	NULL	Дата мягкого удаления записи (архивация)

ПРИЛОЖЕНИЕ В

ПРОГРАММНЫЙ КОД

Листинг В.1 – Файл models.py

```
from django.db import models #Подключение основного
модуля Django для объявления ORM-моделей
import os #Подключение стандартной библиотеки Python
для работы с файловой системой и путями
# РОЛЬ
class Role(models.Model):
    ROLE_CHOICES = [ #Объявление списка кортежей для
ограничения возможных значений в выпадающем списке
        ('technician', 'Техник'), #Определение первого
варианта: внутреннее значение и человекочитаемое
название
        ('employee', 'Сотрудник'), #Определение второго
варианта: внутреннее значение и человекочитаемое
название
        ('manager', 'Руководитель'), #Определение третьего
варианта: внутреннее значение и человекочитаемое
название
        ('admin', 'Администратор'), #Определение четвёртого
варианта: внутреннее значение и человекочитаемое
название
    ]
    id = models.IntegerField(primary_key=True,
        verbose_name="ID") #Объявление
целочисленного поля как первичного ключа таблицы
    role_name = models.CharField(max_length=250,
        choices=ROLE_CHOICES, unique=True,
        verbose_name="Наименование")
#Объявление строкового поля с ограничением длины,
выбором из списка и требованием уникальности
    delete_date = models.DateTimeField(null=True,
        blank=True,
        verbose_name="Дата удаления")
#Объявление поля даты и времени для реализации
мягкого удаления с разрешением пустых значений
# ДОЛЖНОСТЬ
class Position(models.Model):
    id = models.IntegerField(primary_key=True,
        verbose_name="ID") #Объявление
целочисленного поля как первичного ключа таблицы
    name = models.CharField(max_length=250,
        verbose_name="Наименование")
#Объявление строкового поля для хранения названия
должности с ограничением длины
    delete_date = models.DateTimeField(null=True,
        blank=True,
```

```
        verbose_name="Дата удаления")
#Объявление поля даты и времени для мягкого удаления
с разрешением пустых значений
    def __str__(self):
        #Метод возвращает строковое представление
объекта должности для отображения в админке и
интерфейсе
        #Преобразует экземпляр модели в читаемое
название при вызове функции print() или отображении в
формах
        #Автоматически вызывается Django при
необходимости отобразить объект модели в виде текста
        return self.name #Возвращает значение поля name в
качестве строкового представления объекта
# ОТДЕЛ
class Department(models.Model):
    id = models.IntegerField(primary_key=True,
        verbose_name="ID") #Объявление
целочисленного поля как первичного ключа таблицы
    name = models.CharField(max_length=250,
        verbose_name="Наименование отдела")
#Объявление строкового поля для хранения названия
отдела с ограничением длины
    delete_date = models.DateTimeField(null=True,
        blank=True,
        verbose_name="Дата удаления")
#Объявление поля даты и времени для мягкого удаления
с разрешением пустых значений
    def __str__(self):
        #Метод возвращает строковое представление
объекта отдела для отображения в админке и
интерфейсе
        #Преобразует экземпляр модели в читаемое
название при вызове функции print() или отображении в
формах
        #Автоматически вызывается Django при
необходимости отобразить объект модели в виде текста
        return self.name #Возвращает значение поля name в
качестве строкового представления объекта
# КОРПУСА
class Building(models.Model):
    id = models.IntegerField(primary_key=True,
        verbose_name="ID") #Объявление
целочисленного поля как первичного ключа таблицы
    name = models.CharField(max_length=250,
        verbose_name="Наименование корпуса")
#Объявление строкового поля для хранения названия
корпуса с ограничением длины
```

Продолжение листинга В.1

```
delete_date = models.DateTimeField(null=True,
blank=True,
verbose_name="Дата удаления")
#Объявление поля даты и времени для мягкого удаления
с разрешением пустых значений
def __str__(self):
    #Метод возвращает строковое представление
    объекта корпуса для отображения в админке и
    интерфейсе
    #Преобразует экземпляр модели в читаемое
    название при вызове функции print() или отображении в
    формах
    #Автоматически вызывается Django при
    необходимости отобразить объект модели в виде текста
    return self.name #Возвращает значение поля name в
    качестве строкового представления объекта
# КАБИНЕТ
class Office(models.Model):
    id = models.IntegerField(primary_key=True,
        verbose_name="ID") #Объявление
    целочисленного поля как первичного ключа таблицы
    number = models.CharField(max_length=50,
        verbose_name="Номер кабинета", null=True,
        blank=True) #Объявление строкового
    поля для номера кабинета с разрешением пустых
    значений
    building = models.ForeignKey(Building,
        on_delete=models.CASCADE, verbose_name="Корпус",
        null=True,
        blank=True) #Объявление связи один-
    ко-многим с моделью корпуса с каскадным удалением
    delete_date = models.DateTimeField(null=True,
        blank=True,
        verbose_name="Дата удаления")
#Объявление поля даты и времени для мягкого удаления
с разрешением пустых значений
def __str__(self):
    #Метод возвращает строковое представление
    объекта кабинета для отображения в админке и
    интерфейсе
    #Формирует читаемую строку с номером кабинета
    или сообщает о его отсутствии при пустом значении
    #Автоматически вызывается Django при
    необходимости отобразить объект модели в виде текста
    return f"Кабинет {self.number}" if self.number else "Не
    распределён" #Возвращает форматированную строку или
    заглушку

    def save(self, *args, **kwargs):
        #Метод переопределяет стандартное сохранение
        объекта для автоматической генерации уникального ID
```

```
#Выполняет прямой SQL-запрос к базе данных для
поиска максимального существующего идентификатора
#Присваивает новый ID равный максимуму плюс один
перед вызовом родительского метода сохранения
if not self.id: #Проверяет отсутствие присвоенного
идентификатора у создаваемого объекта
    from django.db import connection #Импортирует
    объект подключения к базе данных внутри метода
    with connection.cursor() as cursor: #Открывает
    курсор для выполнения сырого SQL-запроса в
    контекстном менеджере
        cursor.execute(
            "SELECT COALESCE(MAX(id), 0) + 1 FROM
            equip_office") #Выполняет запрос на получение
            следующего свободного ID
        result = cursor.fetchone() #Извлекает первую
        строку результата выполнения запроса из базы
        self.id = result[0] if result[0] else 1 #Присваивает
        вычисленный ID объекту или 1, если таблица пуста
        super().save(*args,
            **kwargs) #Вызывает стандартный метод
        сохранения родительского класса Model для записи в БД
# ПРОИЗВОДИТЕЛИ
class Manufacturer(models.Model):
    id = models.IntegerField(primary_key=True,
        verbose_name="ID") #Объявление
    целочисленного поля как первичного ключа таблицы
    name = models.CharField(max_length=250,
        verbose_name="Наименование")
#Объявление строкового поля для хранения названия
производителя с ограничением длины
delete_date = models.DateTimeField(null=True,
blank=True,
verbose_name="Дата удаления")
#Объявление поля даты и времени для мягкого удаления
с разрешением пустых значений
def __str__(self):
    #Метод возвращает строковое представление
    объекта производителя для отображения в админке и
    интерфейсе
    #Преобразует экземпляр модели в читаемое
    название при вызове функции print() или отображении в
    формах
    #Автоматически вызывается Django при
    необходимости отобразить объект модели в виде текста
    return self.name #Возвращает значение поля name в
    качестве строкового представления объекта
    def save(self, *args, **kwargs):
        #Метод переопределяет стандартное сохранение
        объекта для автоматической генерации уникального ID
        #Выполняет прямой SQL-запрос к базе данных для
        поиска максимального существующего идентификатора
```

Продолжение листинга В.1

```
#Присваивает новый ID равный максимуму плюс один
перед вызовом родительского метода сохранения
if not self.id: #Проверяет отсутствие присвоенного
идентификатора у создаваемого объекта
    from django.db import connection #Импортирует
объект подключения к базе данных внутри метода
    with connection.cursor() as cursor: #Открывает
курсор для выполнения сырого SQL-запроса в
контекстном менеджере
        cursor.execute(
            "SELECT COALESCE(MAX(id), 0) + 1 FROM
equip_manufacturer") #Выполняет запрос на получение
следующего свободного ID
        result = cursor.fetchone() #Извлекает первую
строку результата выполнения запроса из базы
        self.id = result[0] if result[0] else 1 #Присваивает
вычисленный ID объекту или 1, если таблица пуста
    super().save(*args, **kwargs) #Вызывает стандартный
метод сохранения родительского класса Model для
записи в Б
# МОДЕЛИ ОБОРУДОВАНИЯ
class EquipmentModel(models.Model):
    id = models.IntegerField(primary_key=True,
        verbose_name="ID") #Объявление
целочисленного поля как первичного ключа таблицы
    name = models.CharField(max_length=250,
        verbose_name="Наименование модели")
#Объявление строкового поля для хранения названия
модели с ограничением длины
    manufacturer = models.ForeignKey(Manufacturer,
on_delete=models.CASCADE,
    verbose_name="Производитель", null=True,
        blank=True) #Объявление связи
один-ко-многим с моделью производителя с каскадным
удалением
    delete_date = models.DateTimeField(null=True,
        blank=True,
        verbose_name="Дата удаления")
#Объявление поля даты и времени для мягкого удаления
с разрешением пустых значений
    def __str__(self):
        #Метод возвращает строковое представление
объекта модели оборудования для отображения в
админке и интерфейсе
        #Преобразует экземпляр модели в читаемое
название при вызове функции print() или отображении в
формах
        #Автоматически вызывается Django при
необходимости отобразить объект модели в виде текста
        return self.name #Возвращает значение поля name в
качестве строкового представления объекта
```

```
def save(self, *args, **kwargs):
    #Метод переопределяет стандартное сохранение
объекта для автоматической генерации уникального ID
    #Выполняет прямой SQL-запрос к базе данных для
поиска максимального существующего идентификатора
    #Присваивает новый ID равный максимуму плюс один
перед вызовом родительского метода сохранения
    if not self.id: #Проверяет отсутствие присвоенного
идентификатора у создаваемого объекта
        from django.db import connection #Импортирует
объект подключения к базе данных внутри метода
        with connection.cursor() as cursor: #Открывает
курсор для выполнения сырого SQL-запроса в
контекстном менеджере
            cursor.execute(
                "SELECT COALESCE(MAX(id), 0) + 1 FROM
equip_equipmentmodel") #Выполняет запрос на получение
следующего свободного ID
            result = cursor.fetchone() #Извлекает первую
строку результата выполнения запроса из базы
            self.id = result[0] if result[0] else 1 #Присваивает
вычисленный ID объекту или 1, если таблица пуста
        super().save(*args,
            **kwargs) #Вызывает стандартный метод
сохранения родительского класса Model для записи в БД
# ТИП ОБОРУДОВАНИЯ
class EquipmentType(models.Model):
    id = models.IntegerField(primary_key=True,
        verbose_name="ID") #Объявление
целочисленного поля как первичного ключа таблицы
    name = models.CharField(max_length=250,
        verbose_name="Наименование типа")
#Объявление строкового поля для хранения названия
типа оборудования с ограничением длины
    delete_date = models.DateTimeField(null=True,
        blank=True,
        verbose_name="Дата удаления")
#Объявление поля даты и времени для мягкого удаления
с разрешением пустых значений
    def __str__(self):
        #Метод возвращает строковое представление
объекта типа оборудования для отображения в админке и
интерфейсе
        #Преобразует экземпляр модели в читаемое
название при вызове функции print() или отображении в
формах
        #Автоматически вызывается Django при
необходимости отобразить объект модели в виде текста
        return self.name #Возвращает значение поля name в
качестве строкового представления объекта
    def save(self, *args, **kwargs):
```

Продолжение листинга В.1

```
#Метод переопределяет стандартное сохранение
объекта для автоматической генерации уникального ID
#Выполняет прямой SQL-запрос к базе данных для
поиска максимального существующего идентификатора
#Присваивает новый ID равный максимуму плюс один
перед вызовом родительского метода сохранения
if not self.id: #Проверяет отсутствие присвоенного
идентификатора у создаваемого объекта
from django.db import connection #Импортирует
объект подключения к базе данных внутри метода
with connection.cursor() as cursor: #Открывает
курсор для выполнения сырого SQL-запроса в
контекстном менеджере
    cursor.execute(
        "SELECT COALESCE(MAX(id), 0) + 1 FROM
equip_equipmenttype") #Выполняет запрос на получение
следующего свободного ID
    result = cursor.fetchone() #Извлекает первую
строку результата выполнения запроса из базы
    self.id = result[0] if result[0] else 1 #Присваивает
вычисленный ID объекту или 1, если таблица пуста
    super().save(*args,
        **kwargs) #Вызывает стандартный метод
сохранения родительского класса Model для записи в БД
# СТАТУСЫ ОБОРУДОВАНИЯ
class EquipmentStatus(models.Model):
    id = models.IntegerField(primary_key=True,
        verbose_name="ID") #Объявление
целочисленного поля как первичного ключа таблицы
    name = models.CharField(max_length=250,
        verbose_name="Наименование статуса")
#Объявление строкового поля для хранения названия
статуса оборудования с ограничением длины
    delete_date = models.DateTimeField(null=True,
blank=True,
        verbose_name="Дата удаления")
#Объявление поля даты и времени для мягкого удаления
с разрешением пустых значений
    def __str__(self):
        #Метод возвращает строковое представление
объекта статуса оборудования для отображения в
админке и интерфейсе
        #Преобразует экземпляр модели в читаемое
название при вызове функции print() или отображении в
формах
        #Автоматически вызывается Django при
необходимости отобразить объект модели в виде текста
        return self.name #Возвращает значение поля name в
качестве строкового представления объекта
    def save(self, *args, **kwargs):
```

```
#Метод переопределяет стандартное сохранение
объекта для автоматической генерации уникального ID
#Выполняет прямой SQL-запрос к базе данных для
поиска максимального существующего идентификатора
#Присваивает новый ID равный максимуму плюс один
перед вызовом родительского метода сохранения
if not self.id: #Проверяет отсутствие присвоенного
идентификатора у создаваемого объекта
from django.db import connection #Импортирует
объект подключения к базе данных внутри метода
with connection.cursor() as cursor: #Открывает
курсор для выполнения сырого SQL-запроса в
контекстном менеджере
    cursor.execute(
        "SELECT COALESCE(MAX(id), 0) + 1 FROM
equip_equipmentsstatus") #Выполняет запрос на получение
следующего свободного ID
    result = cursor.fetchone() #Извлекает первую
строку результата выполнения запроса из базы
    self.id = result[0] if result[0] else 1 #Присваивает
вычисленный ID объекту или 1, если таблица пуста
    super().save(*args,
        **kwargs) #Вызывает стандартный метод
сохранения родительского класса Model для записи в БД
# ГАРАНТИИ
class Warranty(models.Model):
    id = models.IntegerField(primary_key=True,
        verbose_name="ID") #Объявление
целочисленного поля как первичного ключа таблицы
    start_date = models.DateField(verbose_name="Дата
начала", null=True,
        blank=True) #Объявление поля даты
для указания начала гарантийного срока с разрешением
пустых значений
    end_date = models.DateField(verbose_name="Дата
окончания", null=True,
        blank=True) #Объявление поля даты
для указания окончания гарантийного срока с
разрешением пустых значений
    delete_date = models.DateTimeField(null=True,
blank=True,
        verbose_name="Дата удаления")
#Объявление поля даты и времени для мягкого удаления
с разрешением пустых значений
    def __str__(self):
        #Метод возвращает строковое представление
объекта гарантии для отображения в админке и
интерфейсе
        #Формирует читаемую строку с датой окончания
гарантии или сообщает об её отсутствии
        #Автоматически вызывается Django при
необходимости отобразить объект модели в виде текста
```


Продолжение листинга В.1

```
        return f"Гарантия до {self.end_date}" if self.end_date
    else "Без гарантии" #Возвращает форматированную
    строку или заглушку

    def save(self, *args, **kwargs):
        #Метод переопределяет стандартное сохранение
        объекта для автоматической генерации уникального ID
        #Выполняет прямой SQL-запрос к базе данных для
        поиска максимального существующего идентификатора
        #Присваивает новый ID равный максимуму плюс один
        перед вызовом родительского метода сохранения
        if not self.id: #Проверяет отсутствие присвоенного
        идентификатора у создаваемого объекта
            from django.db import connection #Импортирует
            объект подключения к базе данных внутри метода
            with connection.cursor() as cursor: #Открывает
            курсор для выполнения сырого SQL-запроса в
            контекстном менеджере
                cursor.execute(
                    "SELECT COALESCE(MAX(id), 0) + 1 FROM
                    equip_warranty") #Выполняет запрос на получение
                    следующего свободного ID
                result = cursor.fetchone() #Извлекает первую
                строку результата выполнения запроса из базы
                self.id = result[0] if result[0] else 1 #Присваивает
                вычисленный ID объекту или 1, если таблица пуста
                super().save(*args,
                    **kwargs) #Вызывает стандартный метод
                сохранения родительского класса Model для записи в БД
        # ФОТОГРАФИИ
        class Photos(models.Model):
            id = models.AutoField(primary_key=True,
                verbose_name="ID") #Объявление
            автоматически инкрементируемого поля как первичного
            ключа таблицы
            name = models.CharField(max_length=250,
                verbose_name="Название фото")
            #Объявление строкового поля для хранения имени
            файла фотографии с ограничением длины
            delete_date = models.DateTimeField(null=True,
                blank=True,
                verbose_name="Дата удаления")
            #Объявление поля даты и времени для мягкого удаления
            с разрешением пустых значений

            def save(self, *args, **kwargs):
                #Метод переопределяет стандартное сохранение
                объекта для автоматической генерации уникального ID
                #Выполняет прямой SQL-запрос к базе данных для
                поиска максимального существующего идентификатора
                #Присваивает новый ID равный максимуму плюс один
                перед вызовом родительского метода сохранения
```

```
        if not self.id: #Проверяет отсутствие присвоенного
        идентификатора у создаваемого объекта
            from django.db import connection #Импортирует
            объект подключения к базе данных внутри метода
            with connection.cursor() as cursor: #Открывает
            курсор для выполнения сырого SQL-запроса в
            контекстном менеджере
                cursor.execute("""
                    SELECT COALESCE(MAX(id), 0) + 1 FROM
                    equip_photos
                    """) #Выполняет запрос на получение
                    следующего свободного ID с форматированием
                result = cursor.fetchone() #Извлекает первую
                строку результата выполнения запроса из базы
                self.id = result[0] if result[0] else 1 #Присваивает
                вычисленный ID объекту или 1, если таблица пуста
                super().save(*args,
                    **kwargs) #Вызывает стандартный метод
                сохранения родительского класса Model для записи в БД
            def __str__(self):
                #Метод возвращает строковое представление
                объекта фотографии для отображения в админке и
                интерфейсе
                #Преобразует экземпляр модели в читаемое
                название файла при вызове функции print()
                #Автоматически вызывается Django при
                необходимости отобразить объект модели в виде текста
                return self.name #Возвращает значение поля name в
                качестве строкового представления объекта
        # ОБОРУДОВАНИЕ
        class Equipment(models.Model):
            inventory_number =
            models.IntegerField(primary_key=True,
                verbose_name="Инвентарный
                номер ") #Объявление целочисленного поля как
                первичного ключа таблицы
            assigned_office = models.ForeignKey(Office,
                on_delete=models.SET_NULL, null=True, blank=True,
                verbose_name="Закрепленный
                кабинет ") #Объявление связи с кабинетом с обнулением
                при удалении
            model = models.ForeignKey(EquipmentModel,
                on_delete=models.CASCADE,
                verbose_name="Модель ")
            #Объявление связи с моделью оборудования с
            каскадным удалением
            type = models.ForeignKey(EquipmentType,
                on_delete=models.CASCADE,
                verbose_name="Тип оборудования ")
            #Объявление связи с типом оборудования с каскадным
            удалением
```

Продолжение листинга В.1

```
status = models.ForeignKey(EquipmentStatus,
on_delete=models.CASCADE,
verbose_name="Статус ") #Объявление
связи со статусом оборудования с каскадным удалением
configuration = models.CharField(max_length=255,
verbose_name="Комплектация ", null=True,
blank=True) #Объявление
строкового поля для описания комплектации с
разрешением пустых значений
warranty = models.ForeignKey(Warranty,
on_delete=models.SET_NULL, null=True, blank=True,
verbose_name="Гарантия ")
#Объявление связи с гарантией с обнулением при
удалении
purchase_date = models.DateField(verbose_name="Дата
приобретения ", null=True,
blank=True) #Объявление поля
даты приобретения оборудования с разрешением пустых
значений
photo = models.ForeignKey(Photos,
on_delete=models.SET_NULL, null=True, blank=True,
verbose_name="Фото ") #Объявление
связи с фотографией с обнулением при удалении
delete_date = models.DateTimeField(null=True,
blank=True,
verbose_name="Дата удаления ")
#Объявление поля даты и времени для мягкого удаления
с разрешением пустых значений
def __str__(self):
    #Метод возвращает строковое представление
    объекта оборудования для отображения в админке и
    интерфейсе
    #Формирует читаемую строку, объединяющую
    название модели и инвентарный номер в скобках
    #Автоматически вызывается Django при
    необходимости отобразить объект модели в виде текста
    return f"{self.model.name} ({self.inventory_number})"
#Возвращает форматированную строку с моделью и
номером
# КАТЕГОРИИ ЗАЯВОК
class RequestCategory(models.Model):
    id = models.IntegerField(primary_key=True,
verbose_name="ID") #Объявление
целочисленного поля как первичного ключа таблицы
name = models.CharField(max_length=250,
verbose_name="Наименование")
#Объявление строкового поля для хранения названия
категории заявки с ограничением длины
delete_date = models.DateTimeField(null=True,
blank=True,
```

```
verbose_name="Дата удаления")
#Объявление поля даты и времени для мягкого удаления
с разрешением пустых значений
def __str__(self):
    #Метод возвращает строковое представление
    объекта категории заявки для отображения в админке и
    интерфейсе
    #Преобразует экземпляр модели в читаемое
    название при вызове функции print() или отображении в
    формах
    #Автоматически вызывается Django при
    необходимости отобразить объект модели в виде текста
    return self.name #Возвращает значение поля name в
    качестве строкового представления объекта
# ЭТАПЫ РЕМОНТА
class RepairStage(models.Model):
    id = models.IntegerField(primary_key=True,
verbose_name="ID") #Объявление
целочисленного поля как первичного ключа таблицы
name = models.CharField(max_length=250,
verbose_name="Наименование")
#Объявление строкового поля для хранения названия
этапа ремонта с ограничением длины
delete_date = models.DateTimeField(null=True,
blank=True,
verbose_name="Дата удаления")
#Объявление поля даты и времени для мягкого удаления
с разрешением пустых значений
def __str__(self):
    #Метод возвращает строковое представление
    объекта этапа ремонта для отображения в админке и
    интерфейсе
    #Преобразует экземпляр модели в читаемое
    название при вызове функции print() или отображении в
    формах
    #Автоматически вызывается Django при
    необходимости отобразить объект модели в виде текста
    return self.name #Возвращает значение поля name в
    качестве строкового представления объекта
# ПРИОРИТЕТЫ
class Priority(models.Model):
    id = models.IntegerField(primary_key=True,
verbose_name="ID") #Объявление
целочисленного поля как первичного ключа таблицы
name = models.CharField(max_length=250,
verbose_name="Наименование")
#Объявление строкового поля для хранения названия
приоритета заявки с ограничением длины
delete_date = models.DateTimeField(null=True,
blank=True,
```

Продолжение листинга В.1

```
        verbose_name="Дата удаления")
#Объявление поля даты и времени для мягкого удаления
с разрешением пустых значений
def __str__(self):
    #Метод возвращает строковое представление
объекта приоритета для отображения в админке и
интерфейсе
    #Преобразует экземпляр модели в читаемое
название при вызове функции print() или отображении в
формах
    #Автоматически вызывается Django при
необходимости отобразить объект модели в виде текста
    return self.name #Возвращает значение поля name в
качестве строкового представления объекта
# ЗАПЧАСТИ
class SparePart(models.Model):
    id = models.IntegerField(primary_key=True,
        verbose_name="ID") #Объявление
целочисленного поля как первичного ключа таблицы
    name = models.CharField(max_length=250,
        verbose_name="Наименование")
#Объявление строкового поля для хранения названия
запчасти с ограничением длины
    quantity = models.IntegerField(
        verbose_name="Количество на складе")
#Объявление целочисленного поля для хранения
остатков запчасти на складе
    cost = models.DecimalField(max_digits=10,
decimal_places=2,
        verbose_name="Стоимость за 1 шт.")
#Объявление поля десятичной дроби для хранения
стоимости единицы запчасти
    delete_date = models.DateTimeField(null=True,
blank=True,
        verbose_name="Дата удаления")
#Объявление поля даты и времени для мягкого удаления
с разрешением пустых значений
def __str__(self):
    #Метод возвращает строковое представление
объекта запчасти для отображения в админке и
интерфейсе
    #Преобразует экземпляр модели в читаемое
название при вызове функции print() или отображении в
формах
    #Автоматически вызывается Django при
необходимости отобразить объект модели в виде текста
    return self.name #Возвращает значение поля name в
качестве строкового представления объекта
# СТОРОННИЕ УСЛУГИ
class ThirdPartyService(models.Model):
    id = models.IntegerField(primary_key=True,
```

```
        verbose_name="ID") #Объявление
целочисленного поля как первичного ключа таблицы
    name = models.CharField(max_length=250,
        verbose_name="Наименование")
#Объявление строкового поля для хранения названия
услуги с ограничением длины
    description = models.TextField(blank=True, null=True,
        verbose_name="Описание")
#Объявление текстового поля для развёрнутого описания
услуги с разрешением пустых значений
    cost = models.DecimalField(max_digits=10,
decimal_places=2,
        verbose_name="Стоимость")
#Объявление поля десятичной дроби для хранения
стоимости услуги
    delete_date = models.DateTimeField(null=True,
blank=True,
        verbose_name="Дата удаления")
#Объявление поля даты и времени для мягкого удаления
с разрешением пустых значений
def __str__(self):
    #Метод возвращает строковое представление
объекта услуги для отображения в админке и интерфейсе
    #Преобразует экземпляр модели в читаемое
название при вызове функции print() или отображении в
формах
    #Автоматически вызывается Django при
необходимости отобразить объект модели в виде текста
    return self.name #Возвращает значение поля name в
качестве строкового представления объекта
# ФАЙЛЫ
class File(models.Model):
    id = models.IntegerField(primary_key=True,
        verbose_name="ID") #Объявление
целочисленного поля как первичного ключа таблицы
    file = models.FileField(upload_to='files/%Y/%m/%d',
        verbose_name="Файл") #Объявление
поля для загрузки файлов с автоматическим
распределением по датам
    uploaded_at =
models.DateTimeField(auto_now_add=True,
        verbose_name="Дата загрузки")
#Объявление поля даты и времени, автоматически
заполняемого при создании записи
    delete_date = models.DateTimeField(null=True,
blank=True,
        verbose_name="Дата удаления")
#Объявление поля даты и времени для мягкого удаления
с разрешением пустых значений
def __str__(self):
    #Метод возвращает строковое представление
объекта файла для отображения в админке и интерфейсе
```

Продолжение листинга В.1

```
#Извлекает только имя файла из полного пути к
загруженному документу для удобства чтения
#Автоматически вызывается Django при
необходимости отобразить объект модели в виде текста
return os.path.basename(self.file.name) #Возвращает
базовое имя файла из полного пути
def filename(self):
    #Метод предоставляет альтернативный способ
    получения имени файла для использования в шаблонах
    #Возвращает только имя файла без пути к
    директории, обрезая все предшествующие символы
    #Автоматически вызывается Django при
    необходимости отобразить объект модели в виде текста
    return os.path.basename(self.file.name) #Возвращает
    базовое имя файла из полного пути
def save(self, *args, **kwargs):
    #Метод переопределяет стандартное сохранение
    объекта для автоматической генерации уникального ID
    #Выполняет прямой SQL-запрос к базе данных для
    поиска максимального существующего идентификатора
    #Присваивает новый ID равный максимуму плюс один
    перед вызовом родительского метода сохранения
    if not self.id: #Проверяет отсутствие присвоенного
    идентификатора у создаваемого объекта
        from django.db import connection #Импортирует
        объект подключения к базе данных внутри метода
        with connection.cursor() as cursor: #Открывает
        курсор для выполнения сырого SQL-запроса в
        контекстном менеджере
            cursor.execute(
                "SELECT COALESCE(MAX(id), 0) + 1 FROM
                equip_file") #Выполняет запрос на получение следующего
                свободного ID
            result = cursor.fetchone() #Извлекает первую
            строку результата выполнения запроса из базы
            self.id = result[0] if result[0] else 1 #Присваивает
            вычисленный ID объекту или 1, если таблица пуста
            super().save(*args,
                **kwargs) #Вызывает стандартный метод
            сохранения родительского класса Model для записи в БД
# СОТРУДНИК
class Employee(models.Model):
    id = models.IntegerField(primary_key=True,
        verbose_name="ID ") #Объявление
        целочисленного поля как первичного ключа таблицы
    login = models.CharField(max_length=250, unique=True,
        verbose_name="Почта (Логин) ")
    #Объявление уникального строкового поля для хранения
    логина пользователя
    phone_number = models.CharField(max_length=20,
```

```
        verbose_name="Номер телефона ")
    #Объявление строкового поля для хранения номера
    телефона с ограничением длины
    last_name = models.CharField(max_length=250,
        verbose_name="Фамилия ")
    #Объявление строкового поля для хранения фамилии
    сотрудника
    first_name = models.CharField(max_length=250,
        verbose_name="Имя ") #Объявление
        строкового поля для хранения имени сотрудника
    middle_name = models.CharField(max_length=250,
        verbose_name="Отчество ")
    #Объявление строкового поля для хранения отчества
    сотрудника
    position = models.ForeignKey(Position,
        on_delete=models.CASCADE,
        verbose_name="Должность ")
    #Объявление связи с должностью сотрудника с
    каскадным удалением
    department = models.ForeignKey(Department,
        on_delete=models.CASCADE,
        verbose_name="Отдел ")
    #Объявление связи с отделом сотрудника с каскадным
    удалением
    role = models.ForeignKey(Role,
        on_delete=models.CASCADE,
        verbose_name="Роль ") #Объявление
        связи с ролью пользователя с каскадным удалением
    password = models.CharField(max_length=250,
        verbose_name="Пароль ")
    #Объявление строкового поля для хранения пароля
    пользователя
    last_login = models.DateTimeField(null=True, blank=True,
        verbose_name="Время последнего
        захода ") #Объявление поля даты и времени для
        фиксации последнего входа
    delete_date = models.DateTimeField(null=True,
        blank=True,
        verbose_name="Дата удаления ")
    #Объявление поля даты и времени для мягкого удаления
    с разрешением пустых значений
    office = models.ForeignKey(Office,
        on_delete=models.CASCADE, null=True, blank=True,
        verbose_name="Кабинет ")
    #Объявление связи с кабинетом сотрудника с каскадным
    удалением
    def __str__(self):
        #Метод возвращает строковое представление
        объекта сотрудника для отображения в админке и
        интерфейсе
        #Формирует читаемую строку, объединяющую
        фамилию и имя через пробел
```

Продолжение листинга В.1

```
#Автоматически вызывается Django при
необходимости отобразить объект модели в виде текста
return f"{self.last_name}{self.first_name}" #Возвращает
форматированную строку с ФИО
def get_fio_short(self):
    #Метод возвращает сокращённую форму ФИО
    сотрудника с инициалами имени и отчества
    #Извлекает первые буквы имени и отчества, приводя
    их к верхнему регистру
    #Автоматически вызывается в шаблонах для
    компактного отображения имени пользователя
    first_initial = self.first_name[
        0].upper() if self.first_name else " #Получает первую
    букву имени или пустую строку
    middle_initial = self.middle_name[
        0].upper() if self.middle_name else " #Получает
    первую букву отчества или пустую строку
    return f"{self.last_name}.{first_initial}.{middle_initial}."
#Возвращает строку в формате Фамилия.И.О.

def get_fio_full(self):
    #Метод возвращает полную форму ФИО сотрудника
    без сокращений и инициалов
    #Объединяет фамилию, имя и отчество в единую
    строку с разделением пробелами
    #Автоматически вызывается в шаблонах для полного
    отображения имени пользователя
    return f"{self.last_name} {self.first_name}
{self.middle_name}" #Возвращает строку в формате
Фамилия Имя Отчество
def save(self, *args, **kwargs):
    #Метод переопределяет стандартное сохранение
    объекта для автоматической генерации уникального ID
    #Выполняет прямой SQL-запрос к базе данных для
    поиска максимального существующего идентификатора
    #Присваивает новый ID равный максимуму плюс один
    перед вызовом родительского метода сохранения
    if not self.id: #Проверяет отсутствие присвоенного
    идентификатора у создаваемого объекта
    from django.db import connection #Импортирует
    объект подключения к базе данных внутри метода
    with connection.cursor() as cursor: #Открывает
    курсор для выполнения сырого SQL-запроса в
    контекстном менеджере
    cursor.execute(
        "SELECT COALESCE(MAX(id), 0) + 1 FROM
equip_employee") #Выполняет запрос на получение
следующего свободного ID
    result = cursor.fetchone() #Извлекает первую строку
результата выполнения запроса из базы
```

```
self.id = result[0] if result[0] else 1 #Присваивает
вычисленный ID объекту или 1, если таблица пуста
super().save(*args, **kwargs) #Вызывает стандартный
метод сохранения родительского класса Model для
записи в БД
def get_formatted_phone(self):
    #Метод возвращает номер телефона в
    стандартизированном российском формате для
    отображения
    #Фильтрует только цифры, проверяет длину и приводит
    к формату 8 (xxx) xx-xx-xx
    #Автоматически вызывается в шаблонах для
    корректного вывода контактного номера
    if not self.phone_number: #Проверяет наличие
    сохранённого номера телефона
    return " #Возвращает пустую строку при отсутствии
    данных
    digits = "".join(filter(str.isdigit, self.phone_number))
    #Оставляет в строке только цифровые символы
    if len(digits) == 11 and digits.startswith('8'): #Проверяет
    формат из 11 цифр, начинающийся с 8
    return f"8 ({digits[1:4]}) {digits[4:6]}-{digits[6:8]}-
{digits[8:10]}" #Форматирует номер в читаемый вид
    elif len(digits) == 10: #Проверяет формат из 10 цифр без
    кода страны
    return f"8 ({digits[0:3]}) {digits[3:5]}-{digits[5:7]}-
{digits[7:9]}" #Добавляет 8 и форматирует номер
    return self.phone_number #Возвращает исходное
    значение при несоответствии стандартам
def save(self, *args, **kwargs):
    #Метод переопределяет сохранение для очистки и
    нормализации номера телефона перед записью
    #Удаляет все нецифровые символы и приводит номер к
    единому формату хранения
    #Вызывает родительский метод сохранения после
    предварительной обработки данных
    if not self.id: #Проверяет отсутствие присвоенного
    идентификатора у создаваемого объекта
    from django.db import connection #Импортирует
    объект подключения к базе данных внутри метода
    with connection.cursor() as cursor: #Открывает курсор
    для выполнения сырого SQL-запроса в контекстном
    менеджере
    cursor.execute(
        "SELECT COALESCE(MAX(id), 0) + 1 FROM
equip_employee") #Выполняет запрос на получение
следующего свободного ID
    result = cursor.fetchone() #Извлекает первую строку
результата выполнения запроса из базы
    self.id = result[0] if result[0] else 1 #Присваивает
    вычисленный ID объекту или 1, если таблица пуста
```

Продолжение листинга В.1

```
if self.phone_number: #Проверяет наличие номера
    digits = "".join(filter(str.isdigit, self.phone_number))
    #Оставляет в строке только цифровые символы
    if digits.startswith('7') and len(digits) == 11: #Проверяет
        начало с 7 и длину 11
        digits = '8' + digits[1:] #Заменяет 7 на 8 для
        унификации формата
        if digits.startswith('9') and len(digits) == 10: #Проверяет
            начало с 9 и длину 10
            digits = '8' + digits #Добавляет код страны 8 в начало
            self.phone_number = digits[:11] #Обрезает номер до 11
            символов и сохраняет
super().save(*args, **kwargs) #Вызывает стандартный
метод сохранения родительского класса Model для
записи в БД
# ЗАЯВКИ-ЗАПЧАСТИ
class RequestSparePart(models.Model):
    id = models.IntegerField(primary_key=True,
        verbose_name="ID") #Объявление
целочисленного поля как первичного ключа таблицы
    request = models.ForeignKey('RequestFix',
on_delete=models.CASCADE, verbose_name="Заявка",
        related_name='used_spare_parts')
    #Объявление связи с заявкой с обратным доступом через
    used_spare_parts
    spare_part = models.ForeignKey('SparePart',
on_delete=models.CASCADE,
        verbose_name="Запчасть")
    #Объявление связи с запчастью с каскадным удалением
    quantity = models.IntegerField(default=1,
        verbose_name="Количество")
    #Объявление целочисленного поля для количества
    использованных запчастей со значением по умолчанию 1
    cost_at_repair = models.DecimalField(max_digits=10,
decimal_places=2,
        verbose_name="Цена на момент
ремонта") #Объявление поля десятичной дроби для
фиксации цены запчасти
    total_cost = models.DecimalField(max_digits=10,
decimal_places=2, verbose_name="Общая стоимость",
        editable=False,
        null=True,
        blank=True) #Объявление
вычисляемого поля общей стоимости с запретом ручного
редактирования
    delete_date = models.DateTimeField(null=True,
        blank=True,
        verbose_name="Дата удаления")
    #Объявление поля даты и времени для мягкого удаления
    с разрешением пустых значений
```

```
def __str__(self):
    #Метод возвращает строковое представление
    использованной запчасти в контексте заявки
    #Формирует читаемую строку, объединяющую
    название запчасти и умноженное количество
    #Автоматически вызывается Django при
    необходимости отобразить объект модели в виде текста
    return f"{self.spare_part.name} x{self.quantity}"
    #Возвращает форматированную строку с названием и
    количеством
def save(self, *args, **kwargs):
    #Метод переопределяет сохранение для
    автоматического расчёта итоговой стоимости позиции
    #Вычисляет произведение количества и цены,
    записывая результат в поле total_cost
    #Вызывает родительский метод сохранения после
    автоматического вычисления значения
    if not self.id: #Проверяет отсутствие присвоенного
    идентификатора у создаваемого объекта
        from django.db import connection #Импортирует
        объект подключения к базе данных внутри метода
        with connection.cursor() as cursor: #Открывает
        курсор для выполнения сырого SQL-запроса в
        контекстном менеджере
            cursor.execute(
                "SELECT COALESCE(MAX(id), 0) + 1 FROM
                equip_requestsparepart") #Выполняет запрос на получение
                следующего свободного ID
            result = cursor.fetchone() #Извлекает первую
            строку результата выполнения запроса из базы
            self.id = result[0] if result[0] else 1 #Присваивает
            вычисленный ID объекту или 1, если таблица пуста
            if self.quantity and self.cost_at_repair: #Проверяет
            наличие обоих значений для расчёта
            self.total_cost = self.quantity * self.cost_at_repair
            #Вычисляет итоговую стоимость позиции
            super().save(*args,
                **kwargs) #Вызывает стандартный метод
            сохранения родительского класса Model для записи в БД
    # ЗАЯВКИ-УСЛУГИ
class RequestService(models.Model):
    id = models.IntegerField(primary_key=True,
        verbose_name="ID ") #Объявление
целочисленного поля как первичного ключа таблицы
    request = models.ForeignKey('RequestFix',
on_delete=models.CASCADE, verbose_name="Заявка ",
        related_name='used_services')
    #Объявление связи с заявкой с обратным доступом через
    used_services
    service = models.ForeignKey('ThirdPartyService',
on_delete=models.CASCADE,
```

Продолжение листинга В.1

```
        verbose_name="Услуга")
#Объявление связи с сторонней услугой с каскадным
удалением
    quantity = models.IntegerField(default=1,
        verbose_name="Количество")
#Объявление целочисленного поля для количества
оказанных услуг со значением по умолчанию 1
    cost_at_repair = models.DecimalField(max_digits=10,
decimal_places=2,
        verbose_name="Цена на момент
ремонта ") #Объявление поля десятичной дроби для
фиксации цены услуги
    total_cost = models.DecimalField(max_digits=10,
decimal_places=2, verbose_name="Общая стоимость ",
    editable=False,
        null=True,
        blank=True) #Объявление
вычисляемого поля общей стоимости с запретом ручного
редактирования
    receipt_file = models.ForeignKey('File',
on_delete=models.SET_NULL, null=True, blank=True,
        verbose_name="Чек (файл)")
#Объявление связи с файлом чека с обнулением при
удалении файла
    delete_date = models.DateTimeField(null=True,
blank=True,
        verbose_name="Дата удаления ")
#Объявление поля даты и времени для мягкого удаления
с разрешением пустых значений
    def __str__(self):
        #Метод возвращает строковое представление
использованной услуги в контексте заявки
        #Формирует читаемую строку, объединяющую
название услуги и умноженное количество
        #Автоматически вызывается Django при
необходимости отобразить объект модели в виде текста
        return f"{self.service.name} x{self.quantity}"
#Возвращает форматированную строку с названием и
количеством
    def save(self, *args, **kwargs):
        #Метод переопределяет сохранение для
автоматического расчёта итоговой стоимости позиции
        #Вычисляет произведение количества и цены,
записывая результат в поле total_cost
        #Вызывает родительский метод сохранения после
автоматического вычисления значения
        if not self.id: #Проверяет отсутствие присвоенного
идентификатора у создаваемого объекта
            from django.db import connection #Импортирует
объект подключения к базе данных внутри метода
```

```
        with connection.cursor() as cursor: #Открывает
курсор для выполнения сырого SQL-запроса в
контекстном менеджере
            cursor.execute(
                "SELECT COALESCE(MAX(id), 0) + 1 FROM
equip_requestservice") #Выполняет запрос на получение
следующего свободного ID
            result = cursor.fetchone() #Извлекает первую
строку результата выполнения запроса из базы
            self.id = result[0] if result[0] else 1 #Присваивает
вычисленный ID объекту или 1, если таблица пуста
            if self.quantity and self.cost_at_repair: #Проверяет
наличие обоих значений для расчёта
                self.total_cost = self.quantity * self.cost_at_repair
#Вычисляет итоговую стоимость позиции
            super().save(*args,
                **kwargs) #Вызывает стандартный метод
сохранения родительского класса Model для записи в БД
# ЗАЯВКА
class RequestFix(models.Model):
    act_number = models.IntegerField(primary_key=True,
        verbose_name="Номер акта ")
#Объявление целочисленного поля как первичного ключа
таблицы
    requester = models.ForeignKey(Employee,
on_delete=models.CASCADE,
related_name="requests_made ",
        verbose_name="Заказчик ")
#Объявление связи с сотрудником-заявителем с
обратным доступом
    assigned_technician = models.ForeignKey(Employee,
on_delete=models.SET_NULL, null=True, blank=True,
related_name="requests_assigned ",
        verbose_name="Назначенный
исполнитель ") #Объявление связи с техником-
исполнителем с обнулением
    equipment = models.ForeignKey(Equipment,
on_delete=models.CASCADE,
        verbose_name="Оборудование ")
#Объявление связи с оборудованием с каскадным
удалением
    category = models.ForeignKey(RequestCategory,
on_delete=models.CASCADE,
        verbose_name="Категория ")
#Объявление связи с категорией заявки с каскадным
удалением
    repair_stage = models.ForeignKey(RepairStage,
on_delete=models.CASCADE,
        verbose_name="Этап ремонта ")
#Объявление связи с этапом ремонта с каскадным
удалением
```

Продолжение листинга В.1

```
priority = models.ForeignKey(Priority,
on_delete=models.SET_NULL, null=True, blank=True,
verbose_name="Приоритет ")
#Объявление связи с приоритетом заявки с обнулением
при удалении
problem_description = models.CharField(max_length=500,
verbose_name="Описание
неисправности ") #Объявление строкового поля для
описания проблемы с ограничением длины
registration_date =
models.DateTimeField(auto_now_add=True,
verbose_name="Дата
регистрации ") #Объявление поля даты регистрации,
автоматически заполняемого при создании
completion_date = models.DateTimeField(null=True,
blank=True,
verbose_name="Дата
выполнения ") #Объявление поля даты выполнения
заявки с разрешением пустых значений
comment_technician = models.TextField(blank=True,
null=True,
verbose_name="Комментарий
техника ") #Объявление текстового поля для заметок
техника с разрешением пустых значений
spare_parts = models.ManyToManyField(
SparePart, #Указывает модель, с которой
устанавливается связь многие-ко-многим
through='RequestSparePart', #Задаёт промежуточную
таблицу для хранения дополнительных данных связи
through_fields=('request', 'spare_part'), #Определяет
поля промежуточной модели, связывающие текущую и
целевую
blank=True, #Разрешает отсутствие связанных
запчастей при создании заявки
verbose_name="Запчасти" #Задаёт
человекочитаемое имя связи для интерфейса
администратора
)
services = models.ManyToManyField(
ThirdPartyService, #Указывает модель сторонних
услуг, с которой устанавливается связь
through='RequestService', #Задаёт промежуточную
таблицу для хранения дополнительных данных связи
through_fields=('request', 'service'), #Определяет поля
промежуточной модели, связывающие текущую и
целевую
blank=True, #Разрешает отсутствие связанных услуг
при создании заявки
verbose_name="Услуги" #Задаёт человекочитаемое
имя связи для интерфейса администратора
)
```

```
delete_date = models.DateTimeField(null=True,
blank=True,
verbose_name="Дата
удаления")
#Объявление поля даты и времени для мягкого удаления
с разрешением пустых значений
def get_total_spare_parts_cost(self):
#Метод вычисляет суммарную стоимость всех
запчастей, использованных в текущей заявке
#Агрегирует поле total_cost только для активных
записей без даты удаления
#Возвращает числовое значение или ноль, если
запчасти не были использованы
from django.db.models import Sum #Импортирует
функцию агрегации SUM внутри метода для экономии
ресурсов
total =
self.used_spare_parts.filter(delete_date__isnull=True).aggre
gate(
total=Sum('total_cost') #Вычисляет сумму всех
полей общей стоимости отфильтрованных записей
)['total'] #Извлекает вычисленное значение из
словаря результата агрегации
return total if total else 0 #Возвращает сумму или ноль
при отсутствии записей
def get_total_services_cost(self):
#Метод вычисляет суммарную стоимость всех
сторонних услуг, использованных в текущей заявке
#Агрегирует поле total_cost только для активных
записей без даты удаления
#Возвращает числовое значение или ноль, если
услуги не были использованы
from django.db.models import Sum #Импортирует
функцию агрегации SUM внутри метода для экономии
ресурсов
total =
self.used_services.filter(delete_date__isnull=True).aggregat
e(
total=Sum('total_cost') #Вычисляет сумму всех
полей общей стоимости отфильтрованных записей
)['total'] #Извлекает вычисленное значение из
словаря результата агрегации
return total if total else 0 #Возвращает сумму или ноль
при отсутствии записей
def get_total_cost(self):
#Метод вычисляет итоговую стоимость ремонта по
заявке, суммируя затраты на запчасти и услуги
#Вызывает два вспомогательных метода расчёта и
складывает их результаты
#Возвращает единое числовое значение для
отображения в отчётах и интерфейсе
```


Продолжение листинга В.1

```
        return self.get_total_spare_parts_cost() +
self.get_total_services_cost() #Суммирует стоимости
запчастей и услуг
    def __str__(self):
        #Метод возвращает строковое представление
объекта заявки для отображения в админке и интерфейсе
        #Формирует читаемую строку, содержащую номер
акта заявки в скобках
        #Автоматически вызывается Django при
необходимости отобразить объект модели в виде текста
        return f"Заявка №{self.act_number}" #Возвращает
форматированную строку с номером акта
```

Листинг В.2 – Файл views.py

```
from django.contrib import messages #Подключение
системы всплывающих уведомлений для вывода
сообщений об успехе или ошибках
import os #Подключение стандартного модуля для
работы с файловой системой и путями
import re #Подключение модуля регулярных выражений
для поиска и извлечения данных из текста
from django.template.loader import render_to_string
#Импорт функции для преобразования HTML-шаблонов в
строку
import matplotlib #Подключение библиотеки для
настройки параметров отрисовки графиков
import matplotlib.pyplot as plt #Импорт основного
интерфейса построения диаграмм и гистограмм
from django.db.models import Q, Count, Sum
#Подключение инструментов ORM для сложных
фильтров и агрегации данных
from django.utils import timezone #Импорт утилит для
работы с датами и часовыми поясами
from django.shortcuts import render, redirect,
get_object_or_404 #Подключение базовых функций
отрисовки, перенаправления и безопасного поиска
записей
from django.views.generic import ListView #Импорт
классового представления для автоматического вывода
списков объектов
from datetime import datetime, timedelta #Подключение
стандартных классов для расчёта дат и временных
интервалов
from django.http import JsonResponse #Импорт класса
для формирования HTTP-ответов в формате JSON
from django.views.decorators.http import require_POST
#Подключение декоратора для ограничения доступа к
функции только POST-запросами
from django.conf import settings #Импорт конфигурации
проекта для доступа к глобальным настройкам
```

```
from .models import ( #Импорт всех пользовательских
моделей из текущего приложения для работы с базой
данных
    Role, Position, Department, Building, Office,
Manufacturer,
    EquipmentModel, EquipmentType, EquipmentStatus,
Warranty, Photos,
    Equipment, RequestCategory, RepairStage, Priority,
SparePart,
    Employee, RequestFix, RequestSparePart,
RequestService, ThirdPartyService, File
)
# Функция обрабатывает процесс аутентификации
пользователя, проверяя логин и пароль в базе данных и
инициализируя сессию.
# Метод использует ORM Django для поиска активного
сотрудника и модуль messages для вывода уведомлений
об ошибках.
# Результатом работы является перенаправление на
персональную страницу в зависимости от системной роли
пользователя.
def login_view(request):
    if request.method == 'POST': #Проверяем, что запрос
отправлен методом POST (форма отправлена)
        login_input = request.POST.get("username")
#Получаем логин из данных формы
        password = request.POST.get("password") #Получаем
пароль из данных формы
        try:
            #Поиск пользователя по логину, исключая
удалённых (с пустым delete_date)
            user = Employee.objects.get(login=login_input,
delete_date__isnull=True)
            if user.password == password: #Сравниваем
введённый пароль с хранимым в базе
                request.session['user_id'] = user.id #Сохраняем ID
пользователя в сессии
                request.session['user_login'] = user.login
#Сохраняем логин в сессии
                #Получаем название роли или устанавливаем
значение по умолчанию
                role_name = user.role.role_name if user.role else
'Сотрудник'
                request.session['user_role'] = role_name
#Сохраняем роль в сессии
                #Формируем ФИО пользователя для
отображения в интерфейсе
                request.session['user_name'] = f'{user.last_name}
{user.first_name} {user.middle_name}'
                #Сохраняем должность или None, если она не
указана
```

Продолжение листинга В.2

```
request.session['user_position'] = user.position.name if user.position else None
request.session['form_open'] = False
#Сбрасываем флаг открытой формы
user.last_login = timezone.now() #Обновляем время последнего входа
user.save() #Сохраняем изменения в базе данных
#Выбор соответствующей роли для перенаправления
if role_name == 'Администратор': #Если роль — Администратор
    return redirect('show_admin')
#Перенаправляем на панель администратора
elif role_name == 'Техник': #Если роль — Техник
    return redirect('show_technician')
#Перенаправляем на страницу техника
elif role_name == 'Руководитель': #Если роль — Руководитель
    return redirect('manager_dashboard')
#Перенаправляем на дашборд руководителя
else: #Для всех остальных ролей (Сотрудник)
    return redirect('show_customer')
#Перенаправляем на страницу сотрудника
else: #Если пароль не совпадает
    messages.error(request, 'Неверный пароль')
#Выводим сообщение об ошибке
#Обработка ошибок
except Employee.DoesNotExist: #Если пользователь с таким логином не найден
    messages.error(request, 'Пользователь с таким логином не найден') #Выводим сообщение
except Exception as e: #Обработка любых других исключений
    messages.error(request, f'Ошибка при входе: {str(e)}') #Выводим текст ошибки
    return render(request, 'login.html') #Отображаем страницу входа (для GET-запроса или после ошибки)
#Функция завершает пользовательскую сессию и перенаправляет на страницу входа.
#Метод использует механизм сессий Django и систему сообщений для информирования пользователя.
#Нужна для безопасного выхода из аккаунта и очистки временных данных сессии.
def logout_view(request):
    try:
        request.session.flush() #Полностью очищаем сессию пользователя, удаляя все сохранённые данные
        #Обработка ошибок в случае успеха и ошибки
```

```
messages.info(request, "Вы вышли из системы!")
#Показываем информационное сообщение об успешном выходе
except Exception as e:
    messages.error(request, f'Ошибка: {str(e)}')
#Выводим текст ошибки, если что-то пошло не так
return redirect('login_view') #Перенаправляем пользователя на страницу входа в систему

#Функция перенаправляет авторизованных пользователей на их персональные страницы в зависимости от роли.
#Метод проверяет наличие сессии и использует данные роли из request.session для выбора маршрута.
#Нужна для автоматической маршрутизации после входа и защиты от доступа неавторизованных пользователей.
def index(request):
    if 'user_id' in request.session: #Проверяем, есть ли пользователь в системе (авторизован ли он)
        role = request.session.get('user_role') #Получаем роль пользователя из данных сессии
        if role == 'Администратор': #Если роль — Администратор
            return redirect('show_admin') #Перенаправляем на панель управления администратора
        elif role == 'Техник': #Если роль — Техник
            return redirect('show_technician')
#Перенаправляем на страницу работы техника
        elif role == 'Руководитель': #Если роль — Руководитель
            return redirect('manager_dashboard')
#Перенаправляем на дашборд с аналитикой
        else: #Для всех остальных ролей (например, Сотрудник)
            return redirect('show_customer')
#Перенаправляем на страницу личных заявок сотрудника
    return redirect('login_view') #Если пользователь не авторизован, отправляем его на страницу входа
#Функция создаёт нового сотрудника в системе, проверяя обязательные поля, совпадение паролей и уникальность логина.
#Метод использует ORM Django для работы с моделью Employee и модуль messages для вывода уведомлений пользователю.
#Нужна для регистрации новых пользователей с разграничением прав доступа и заполнением персональных данных.
def create_employee_admin(request):
    #Проверяем, авторизован ли пользователь и является ли он администратором
```

Продолжение листинга В.2

```
if 'user_id' not in request.session or
request.session.get('user_role') != 'Администратор':
    #Выводим сообщение об ошибке доступа через
    систему уведомлений Django
    messages.error(request, 'Нет доступа')
    #Перенаправляем на страницу входа при
    нарушении прав доступа
    return redirect('login_view')
#Получение данных из формы
if request.method == 'POST':
    #Извлекаем имя из POST-данных и удаляем
    лишние пробелы
    first_name = request.POST.get('first_name', "").strip()
    #Извлекаем фамилию из POST-данных и удаляем
    лишние пробелы
    last_name = request.POST.get('last_name', "").strip()
    #Извлекаем отчество из POST-данных и удаляем
    лишние пробелы
    middle_name = request.POST.get('middle_name',
    "").strip()
    #Извлекаем логин (почту) из POST-данных и
    удаляем лишние пробелы
    login = request.POST.get('login', "").strip()
    #Извлекаем номер телефона из POST-данных и
    удаляем лишние пробелы
    phone_number = request.POST.get('phone_number',
    "").strip()
    #Извлекаем пароль из формы (без strip, чтобы
    сохранить спецсимволы)
    password = request.POST.get('password', "")
    #Извлекаем подтверждение пароля для проверки
    совпадения
    password_confirm =
    request.POST.get('password_confirm', "")
    #Извлекаем ID должности из выпадающего списка
    формы
    position_id = request.POST.get('position')
    #Извлекаем ID отдела из выпадающего списка
    формы
    department_id = request.POST.get('department')
    #Извлекаем ID кабинета из выпадающего списка
    формы
    office_id = request.POST.get('office')
    #Извлекаем ID роли из выпадающего списка
    формы
    role_id = request.POST.get('role')
    #Валидация обязательных полей
    if not first_name or not last_name or not login or not
    password:
        #Выводим сообщение об ошибке, если
        обязательные поля пустые
```

```
        messages.error(request, 'Заполните все
        обязательные поля')
        #Возвращаем пользователя на страницу
        создания для исправления
        return redirect('create_employee_admin')
    #Проверка совпадения паролей
    if password != password_confirm:
        #Выводим сообщение, если пароли не
        совпадают
        messages.error(request, 'Пароли не совпадают')
        #Возвращаем пользователя на страницу
        создания для повторного ввода
        return redirect('create_employee_admin')
    #Проверка логина на повтор
    if Employee.objects.filter(login=login,
    delete_date__isnull=True).exists():
        #Выводим сообщение, если пользователь с
        таким логином уже есть
        messages.error(request, 'Пользователь с таким
        логином уже существует')
        #Возвращаем пользователя на страницу
        создания для выбора другого логина
        return redirect('create_employee_admin')
    #Создание нового пользователя учитывая
    настройки из models (может ли быть пустым)
    try:
        #Создаём экземпляр модели Employee с
        заполненными полями из формы
        new_user = Employee(
            first_name=first_name, #Устанавливаем имя
            сотрудника
            last_name=last_name, #Устанавливаем
            фамилию сотрудника
            middle_name=middle_name if middle_name
            else "", #Устанавливаем отчество или пустую строку
            login=login, #Устанавливаем уникальный
            логин для входа
            phone_number=phone_number if
            phone_number else "", #Устанавливаем телефон или
            пустую строку
            password=password, #Устанавливаем пароль
            (хранится в открытом виде/хешируется)
            role_id=role_id if role_id else None,
            #Устанавливаем роль или None, если не выбрана
            position_id=position_id if position_id else None,
            #Устанавливаем должность или None
            department_id=department_id if department_id
            else None, #Устанавливаем отдел или None
            office_id=office_id if office_id else None
            #Устанавливаем кабинет или None
        )
    #Сохранение в БД
```

Продолжение листинга В.2

```
new_user.save()      #Записываем      нового
сотрудника в базу данных

#Выводим сообщение об успешном создании с
ФИО нового сотрудника
messages.success(request,      f'Сотрудник
{last_name} {first_name} создан!')

#Перенаправляем администратора на страницу
списка сотрудников
return redirect('show_admin')

#Обработка ошибок
except Exception as e:
    #Выводим сообщение с текстом возникшей
ошибки
    messages.error(request, f'Ошибка при создании:
{str(e)}')

#Возвращаем пользователя на страницу
создания для повторной попытки
return redirect('create_employee_admin')

#Передача в html шаблоны
context = {
    'positions':
Position.objects.filter(delete_date__isnull=True), #Передаём
список активных должностей
    'departments':
Department.objects.filter(delete_date__isnull=True),
#Передаём список активных отделов
    'offices':
Office.objects.filter(delete_date__isnull=True).select_related(
'building'), #Передаём кабинеты с подгрузкой корпусов
    'roles':      Role.objects.filter(delete_date__isnull=True),
#Передаём список доступных ролей
    'user_name':      request.session.get('user_name',      ''),
#Передаём имя текущего пользователя для шапки сайта
}

#Рендерим страницу создания сотрудника с
переданным контекстом данных
return      render(request,      'create_employee_admin.html',
context)

#Функция редактирует данные сотрудника, обновляя
персональную информацию, должность, отдел и роль.

#Метод использует get_object_or_404 для безопасного
получения записи и ORM Django для сохранения
изменений.

#Нужна для актуализации кадровых данных и
управления доступом через изменение роли
пользователя.

def edit_employee_admin(request, employee_id):
    #Проверяем, авторизован ли пользователь и
является ли он администратором
    if      'user_id'      not      in      request.session      or
request.session.get('user_role') != 'Администратор':
```

```
#Выводим сообщение об ошибке доступа через
систему уведомлений Django
messages.error(request, 'Нет доступа')

#Перенаправляем на страницу входа при
нарушении прав доступа
return redirect('login_view')

try:
    #Получение сотрудника среди не удаленных
    employee      =      get_object_or_404(Employee,
id=employee_id, delete_date__isnull=True)

    #Обновление полей сотрудника данными из
формы
    if request.method == 'POST':
        #Извлекаем имя из POST-данных и удаляем
лишние пробелы
        employee.first_name      =
request.POST.get('first_name', '').strip()
        #Извлекаем фамилию из POST-данных и
удаляем лишние пробелы
        employee.last_name      =
request.POST.get('last_name', '').strip()
        #Извлекаем отчество из POST-данных и
удаляем лишние пробелы
        employee.middle_name      =
request.POST.get('middle_name', '').strip()
        #Извлекаем логин из POST-данных и удаляем
лишние пробелы
        employee.login      =      request.POST.get('login',
'').strip()
        #Извлекаем номер телефона из POST-данных и
удаляем лишние пробелы
        employee.phone_number      =
request.POST.get('phone_number', '').strip()
        #Извлекаем ID должности из выпадающего
списка формы
        employee.position_id      =
request.POST.get('position')
        #Извлекаем ID отдела из выпадающего списка
формы
        employee.department_id      =
request.POST.get('department')
        #Извлекаем ID кабинета из выпадающего списка
формы
        employee.office_id = request.POST.get('office')
        #Извлекаем ID роли из выпадающего списка
формы
        employee.role_id = request.POST.get('role')
        #Обновление пароля только в случае если поле
заполнено
        new_password = request.POST.get('password', '')
        if new_password:
```

Продолжение листинга В.2

```
#Устанавливаем новый пароль, если поле
было заполнено
employee.password = new_password
#Сохранение в БД
employee.save()
#Выводим сообщение об успешном обновлении
с фамилией сотрудника
messages.success(request, f'Сотрудник
{employee.last_name} обновлён!')
#Перенаправляем администратора на страницу
списка сотрудников
return redirect('show_admin')
#Передача в html шаблоны
context = {
    'employee': employee, #Передаём объект
редактируемого сотрудника
    'positions':
Position.objects.filter(delete_date__isnull=True), #Передаём
список активных должностей
    'departments':
Department.objects.filter(delete_date__isnull=True),
#Передаём список активных отделов
    'offices':
Office.objects.filter(delete_date__isnull=True).select_related(
'building'), #Передаём кабинеты с подгрузкой корпусов
    'roles':
Role.objects.filter(delete_date__isnull=True), #Передаём
список доступных ролей
    'user_name': request.session.get('user_name', ''),
#Передаём имя текущего пользователя для шапки сайта
}
#Рендерим страницу редактирования сотрудника с
переданным контекстом
return render(request, 'edit_employee_admin.html',
context)
#Обработка ошибок
except Exception as e:
    #Выводим сообщение с текстом возникшей ошибки
    messages.error(request, f'Ошибка: {str(e)}')
    #Возвращаем пользователя на страницу списка
сотрудников
    return redirect('show_admin')
#Функция выполняет мягкое удаление записи
сотрудника, устанавливая текущую дату в поле удаления.
#Метод использует ORM-модель Employee, систему
сессий Django и модуль timezone для фиксации времени.
#Нужна для архивации учётных записей без
физического стирания данных, сохраняя историю и
возможность восстановления.
def delete_employee_admin(request, employee_id):
```

```
    if 'user_id' not in request.session or
request.session.get('user_role') != 'Администратор':
#Проверяем наличие сессии и права администратора
        messages.error(request, 'Нет доступа') #Выводим
сообщение об отказе в доступе
        return redirect('login_view') #Перенаправляем
пользователя на страницу авторизации
    if request.method == 'POST': #Разрешаем удаление
только через безопасный POST-запрос
        try: #Начинаем блок защиты от ошибок
выполнения
            employee = Employee.objects.get(id=employee_id,
delete_date__isnull=True) #Ищем активного сотрудника по
уникальному ID
            if employee.id == request.session['user_id']:
#Сравниваем ID удаляемого сотрудника с ID текущего
администратора
                messages.error(request, 'Нельзя удалить свою
учётную запись') #Выводим запрет на самоудаление
                return redirect('show_admin') #Возвращаем
администратора на страницу списка
            employee.delete_date = timezone.now()
#Записываем текущее время как дату мягкого удаления
            employee.save() #Фиксируем изменения в базе
данных
            messages.success(request, f'Сотрудник
{employee.last_name} удалён!') #Показываем
уведомление об успешном действии
            except Exception as e: #Ловим любые возникшие
исключения (например, запись уже удалена)
                messages.error(request, f'Ошибка: {str(e)}')
#Выводим подробное сообщение об ошибке
                return redirect('show_admin') #Всегда
перенаправляем обратно на панель администратора

#Функция формирует и отображает список
архивированных сотрудников для администратора.
#Метод использует ORM-запросы с фильтрацией по
дате удаления, связывание справочников и рендеринг
HTML-шаблона.
#Нужна для контроля удалённых записей,
отсортированных по времени удаления, и подготовки к их
восстановлению.
def show_deleted_employees(request):
    if 'user_id' not in request.session or
request.session.get('user_role') != 'Администратор':
#Проверяем авторизацию и роль пользователя
        messages.error(request, 'Нет доступа') #Блокируем
доступ для неавторизованных лиц
        return redirect('login_view') #Отправляем на
страницу входа в систему
```

Продолжение листинга В.2

```
try: #Защищаем код блоком обработки возможных
    ошибок запроса
    employees =
Employee.objects.filter(delete_date__isnull=False).select_re
lated('role', 'position', 'department', 'office').order_by('-
delete_date') #Выбираем удалённых сотрудников,
подгружаем связанные справочники и сортируем по дате
удаления
    context = { #Создаём словарь данных для передачи
в шаблон
        'employees': employees, #Передаём
отфильтрованный список архивированных сотрудников
        'user_name': request.session.get('user_name', ''),
#Добавляем имя текущего пользователя для
отображения в шапке
        'page_title': 'Удалённые сотрудники', #Указываем
заголовок страницы
    }
    return render(request,
'show_deleted_employees.html', context) #Генерируем
HTML-страницу с переданным контекстом
except Exception as e: #Обрабатываем ошибки при
получении данных из БД
    messages.error(request, f'Ошибка: {str(e)}')
#Выводим текст возникшей ошибки
    return redirect('show_admin') #Возвращаем
администратора на главную панель управления

#Функция возвращает архивированную запись
сотрудника в активный список, очищая дату удаления.
#Метод использует ORM-модель Employee, проверку
HTTP-метода POST и систему сообщений Django.
#Нужна для отмены ошибочного удаления и быстрого
возврата сотрудника к работе без потери учётных данных.
def restore_employee(request, employee_id):
    if 'user_id' not in request.session or
request.session.get('user_role') != 'Администратор':
#Проверяем наличие сессии и права администратора
        messages.error(request, 'Нет доступа')
#Уведомляем об отказе в доступе
    return redirect('login_view') #Перенаправляем на
страницу авторизации
    if request.method == 'POST': #Разрешаем
восстановление только через POST-запрос
        try: #Начинаем безопасный блок выполнения
операции
            emp = Employee.objects.get(id=employee_id,
delete_date__isnull=False) #Ищем запись, у которой
установлена дата удаления
            emp.delete_date = None #Очищаем поле даты
удаления, возвращая запись в активное состояние
```

```
emp.save() #Применяем изменения к базе
данных
        messages.success(request, f'Сотрудник
{emp.last_name} {emp.first_name} восстановлен!')
#Информируем об успешном восстановлении
        except Exception as e: #Повим ошибки, если запись
не найдена или повреждена
            messages.error(request, f'Ошибка: {str(e)}')
#Показываем техническое сообщение об ошибке
        return redirect('show_deleted_employees')
#Возвращаем пользователя на страницу архива для
продолжения работы
    #Функция полностью удаляет запись сотрудника из
базы данных без возможности последующего
восстановления.
    #Метод использует ORM Django для поиска архивных
записей, систему сессий для проверки роли и модуль
сообщений для уведомлений.
    #Нужна для окончательной очистки данных при
закрытии учётных записей, с защитой от удаления самого
администратора.
    def delete_employee_permanent(request, employee_id):
        if 'user_id' not in request.session or
request.session.get('user_role') != 'Администратор':
#Проверяем авторизацию и наличие прав
администратора
            messages.error(request, 'Нет доступа') #Выводим
сообщение об отказе в доступе
        return redirect('login_view') #Перенаправляем на
страницу входа в систему
        if request.method == 'POST': #Разрешаем выполнение
операции только через безопасный POST-запрос
            try:
                emp = Employee.objects.get(id=employee_id,
delete_date__isnull=False) #Ищем архивную запись
сотрудника по ID и наличию даты удаления
                if emp.id == request.session['user_id']:
#Сравниваем ID удаляемого сотрудника с ID текущего
администратора
                    messages.error(request, 'Нельзя удалить свою
учётную запись') #Блокируем попытку самоудаления
                return redirect('show_deleted_employees')
#Возвращаем администратора на страницу архива
                emp.delete() #Физически стираем запись
сотрудника из базы данных
                messages.success(request, f'Сотрудник
{emp.last_name} {emp.first_name} удалён навсегда!')
#Показываем уведомление об успешном действии
                except Exception as e: #Обрабатываем любые
непредвиденные ошибки выполнения
                    messages.error(request, f'Ошибка: {str(e)}')
#Выводим техническое сообщение об ошибке
```

Продолжение листинга В.2

```
        return redirect('show_deleted_employees')
#Перенаправляем пользователя обратно на список
удалённых сотрудников
# Классовое представление отображает личный список
заявок текущего пользователя в таблице.
# Метод использует базовый класс ListView Django, ORM-
запросы с фильтрацией и систему сессий для проверки
прав.
# Нужна для предоставления сотрудникам удобного
интерфейса с поиском, сортировкой и фильтрацией своих
обращений.
class CustomerRequestListView(ListView):
    #Метод проверяет авторизацию и права доступа
пользователя перед загрузкой страницы.
    #Использует данные сессии Django и систему
сообщений для вывода предупреждений.
    #Нужен для защиты представления от
несанкционированного просмотра неавторизованными
лицами.
    model = RequestFix #Указываем модель базы данных,
данные которой будут выводиться
    template_name = 'show_customer.html' #Задаём путь к
HTML-шаблону для отрисовки страницы
    context_object_name = 'requests' #Присваиваем имя
переменной для передачи списка в шаблон
    def dispatch(self, request, *args, **kwargs):
        if 'user_id' not in request.session: #Проверяем наличие
идентификатора пользователя в сессии
            messages.error(request, 'Необходимо
авторизоваться') #Выводим сообщение об ошибке
доступа
            return redirect('login_view') #Перенаправляем на
страницу входа в систему
        if request.session.get('user_role') not in ['Сотрудник',
'Администратор','Руководитель','Техник']: #Проверяем
соответствие роли списку разрешённых
            messages.error(request, 'Нет доступа') #Блокируем
доступ для неавторизованных ролей
            return redirect('login_view') #Возвращаем
пользователя на страницу авторизации
        return super().dispatch(request, *args, **kwargs)
#Продолжаем стандартную обработку запроса
родительским классом

    #Метод формирует выборку заявок текущего
пользователя с применением поиска и сортировки.
    #Использует ORM-фильтры, объект Q для сложных
условий и select_related для оптимизации.
    #Нужен для подготовки отфильтрованного списка
обращений, готового к отображению в таблице.
    def get_queryset(self):
```

```
        one_employee = Employee.objects.get(id=self.request.session['user_id'],
delete_date__isnull=True) #Находим активного сотрудника
по ID из сессии
        queryset = RequestFix.objects.filter( #Начинаем
формирование выборки заявок для текущего сотрудника
requester=one_employee, #Фильтруем заявки
только от текущего пользователя
delete_date__isnull=True #Исключаем
архивированные и удалённые обращения
).select_related( #Оптимизируем запрос, подгружая
связанные таблицы одним SQL-запросом
'equipment', 'equipment__model', 'equipment__type',
#Подгружаем данные об оборудовании и его
характеристиках
'equipment__status', 'equipment__assigned_office',
#Добавляем статус и закрепленный кабинет
'assigned_technician', 'category', 'repair_stage',
'priority' #Загружаем информацию об исполнителе и
параметрах заявки
)
        search_query = self.request.GET.get('search', '')
#Получаем текст поискового запроса из URL-параметра
        if search_query: #Если поисковая строка не пустая
            queryset = queryset.filter( #Применяем
дополнительный фильтр к выборке
Q(problem_description__icontains=search_query)
| #Ищем совпадения в описании проблемы (без учёта
регистра)
Q(act_number__icontains=search_query) | #Ищем
совпадения по номеру акта
Q(equipment__inventory_number__icontains=search_query)
#Ищем совпадения по инвентарному номеру
оборудования
)
            status_filter = self.request.GET.get('status', '')
#Получаем выбранный статус оборудования из
параметров URL
            if status_filter: #Если фильтр по статусу активен
                queryset =
queryset.filter(equipment__status_id=status_filter)
#Фильтруем заявки по идентификатору статуса
оборудования
            sort_by = self.request.GET.get('sort', '-
registration_date') #Получаем параметр сортировки из
URL, по умолчанию - по дате убывания
            if sort_by in ['registration_date', '-registration_date',
'act_number', '-act_number']: #Проверяем допустимость
параметра сортировки
                queryset = queryset.order_by(sort_by) #Применяем
выбранный порядок сортировки к результату запроса
```

Продолжение листинга В.2

```
return queryset #Возвращаем итоговую
отфильтрованную и отсортированную выборку заявок
```

```
#Метод дополняет данные шаблона дополнительными
справочниками и параметрами запроса.
```

```
#Использует базовый контекст родителя и извлекает
GET-параметры из URL-адреса.
```

```
#Нужен для передачи в форму текущих значений поиска
и фильтра, чтобы они сохранялись при перезагрузке.
```

```
def get_context_data(self, **kwargs):
    context = super().get_context_data(**kwargs)
#Получаем базовый контекст из родительского класса
ListView
```

```
    context['statuses'] =
EquipmentStatus.objects.filter(delete_date__isnull=True)
#Добавляем список активных статусов для выпадающего
списка фильтра
```

```
    context['search_query'] = self.request.GET.get('search',
'') #Передаём текущий поисковый запрос для сохранения
в поле ввода
```

```
    context['status_filter'] = self.request.GET.get('status', '')
#Передаём выбранный статус для подсветки активного
фильтра
```

```
    context['sort_by'] = self.request.GET.get('sort', '-
registration_date') #Передаём текущий параметр
сортировки для обновления кнопок
```

```
    context['user_name'] =
self.request.session.get('user_name', '') #Добавляем имя
пользователя для отображения в шапке сайта
```

```
    context['user_role'] =
self.request.session.get('user_role', '') #Добавляем роль
пользователя для управления видимостью элементов
```

```
    return context #Возвращаем дополненный контекст
для рендеринга HTML-шаблона
```

```
# Функция обрабатывает создание новой заявки на
ремонт авторизованным пользователем с валидацией
данных.
```

```
# Метод использует ORM Django для работы с моделями
Equipment и RequestFix, а также сессионные флаги для
защиты от дублей.
```

```
# Нужна для обеспечения стандартизированного и
безопасного процесса регистрации неисправностей
оборудования.
```

```
def create_request_customer(request):
    if 'user_id' not in request.session: #Проверяем,
авторизован ли пользователь в системе
```

```
        messages.error(request, 'Необходимо
авторизоваться') #Выводим предупреждение о
необходимости входа
```

```
        return redirect('login_view') #Перенаправляем на
страницу авторизации
```

```
    if request.session.get('user_role') not in ['Сотрудник',
'Администратор','Руководитель','Техник']: #Проверяем
наличие разрешённой роли
```

```
        messages.error(request, 'Нет доступа') #Сообщаем об
отсутствии прав на создание заявки
```

```
        return redirect('login_view') #Возвращаем
пользователя на страницу входа
```

```
    try:
        one_employee =
Employee.objects.get(id=request.session['user_id'],
delete_date__isnull=True) #Извлекаем данные активного
сотрудника из базы
```

```
        equipment_list =
Equipment.objects.filter(assigned_office=one_employee.offic
e,delete_date__isnull=True).select_related('model', 'type',
'status') #Получаем список оборудования из кабинета
сотрудника с подгрузкой связанных данных
```

```
        if request.method == 'GET': #Если запрос получен
методом GET (первичная загрузка страницы)
```

```
            request.session['form_open'] = False #Сбрасываем
флаг отправки формы для новой попытки
```

```
            if request.method == 'POST': #Если запрос получен
методом POST (отправка формы)
```

```
                if request.session.get('form_open', False):
#Проверяем, не была ли форма уже отправлена ранее
```

```
                    messages.error(request, 'Форма уже открыта.
Обновите страницу.') #Предупреждаем о невозможности
повторной отправки
```

```
                    return redirect('create_request_customer')
#Перенаправляем на ту же страницу для сброса
состояния
```

```
                try:
                    request.session['form_open'] = True
#Устанавливаем флаг активной отправки формы
```

```
                    equipment_inv_num =
request.POST.get('equipment') #Получаем инвентарный
номер выбранного оборудования
```

```
                    description =
request.POST.get('problem_description', '').strip()
#Извлекаем и очищаем описание неисправности
```

```
                    category_id = request.POST.get('category')
#Получаем идентификатор выбранной категории заявки
```

```
                    priority_id = request.POST.get('priority')
#Получаем идентификатор выбранного приоритета
```

```
                    if not description or not equipment_inv_num:
#Проверяем заполненность обязательных полей
```

```
                        messages.error(request, 'Заполните
обязательные поля') #Сообщаем об ошибке валидации
```

```
                        request.session['form_open'] = False
#Сбрасываем флаг формы для повторной попытки
```

```
                        return redirect('create_request_customer')
#Возвращаем пользователя на страницу формы
```


Продолжение листинга В.2

```
equipment =
Equipment.objects.get(inventory_number=equipment_inv_num,
delete_date__isnull=True) #Находим объект
оборудования по инвентарному номеру
category =
RequestCategory.objects.get(id=category_id) if category_id
else None #Получаем объект категории или None
priority = Priority.objects.get(id=priority_id) if
priority_id else None #Получаем объект приоритета или
None
last_req =
RequestFix.objects.filter(delete_date__isnull=True).order_by
('-act_number').first() #Ищем последнюю активную заявку
для расчёта номера
new_act_number = (last_req.act_number + 1) if
last_req else 1 #Генерируем новый номер акта
инкрементом или устанавливаем 1
req = RequestFix( #Инициализируем новый
экземпляр заявки с полученными данными
act_number=new_act_number, #Присваиваем
сгенерированный номер акта
problem_description=description, #Сохраняем
описание проблемы
registration_date=timezone.now(), #Фиксируем
текущую дату и время создания
requester=one_employee, #Привязываем
заявку к текущему сотруднику
equipment=equipment, #Привязываем заявку к
выбранному оборудованию
category=category, #Устанавливаем
категорию неисправности
priority=priority, #Устанавливаем приоритет
выполнения
repair_stage=RepairStage.objects.first()
#Назначаем начальный этап ремонта из справочника
)
req.save() #Сохраняем созданную заявку в базе
данных
request.session['form_open'] = False
#Сбрасываем флаг формы после успешного создания
messages.success(request, 'Заявка создана!')
#Выводим сообщение об успехе
return redirect('show_customer')
#Перенаправляем на страницу списка личных заявок
except Exception as e: #Ловим исключения,
возникшие при сохранении или валидации
request.session['form_open'] = False
#Сбрасываем флаг формы в случае ошибки
messages.error(request, f'Ошибка создания:
{str(e)}') #Выводим текст возникшей ошибки
```

```
return redirect('create_request_customer')
#Возвращаем пользователя на страницу формы
context = { #Формируем словарь данных для
передачи в HTML-шаблон
'employee': one_employee, #Передаём данные
текущего сотрудника
'equipment_list': equipment_list, #Передаём список
доступного оборудования
'categories':
RequestCategory.objects.filter(delete_date__isnull=True),
#Передаём активные категории заявок
'priorities':
Priority.objects.filter(delete_date__isnull=True), #Передаём
активные приоритеты
'user_name': request.session.get('user_name', ''),
#Передаём имя пользователя для интерфейса
}
return render(request, 'create_request_customer.html',
context) #Рендерим страницу создания заявки с
контекстом
except Exception as e: #Ловим ошибки верхнего уровня
при получении данных сотрудника
request.session['form_open'] = False #Сбрасываем
флаг формы при критической ошибке
messages.error(request, f'Ошибка: {str(e)}') #Выводим
общее сообщение об ошибке
return redirect('show_customer') #Перенаправляем на
страницу списка заявок
#Классовое представление отображает единый реестр
всех активных заявок для технических специалистов.
#Метод использует базовый класс ListView, ORM-
запросы с объектом Q для сложного поиска и валидацию
сессии.
#Нужна для предоставления технику удобного
интерфейса с расширенным поиском, фильтрацией и
сортировкой обращений.
class TechnicianRequestListView(ListView):
model = RequestFix #Указываем модель базы
данных, данные которой будут выводиться в списке
template_name = 'show_technician.html' #Задаём путь
к HTML-шаблону для отрисовки страницы списка
context_object_name = 'requests' #Присваиваем имя
переменной для передачи списка заявок в шаблон
def dispatch(self, request, *args, **kwargs):
if 'user_id' not in request.session: #Проверяем
наличие идентификатора пользователя в сессии
messages.error(request, 'Необходимо
авторизоваться') #Выводим сообщение об ошибке
доступа
return redirect('login_view') #Перенаправляем
неавторизованного пользователя на страницу входа
```

Продолжение листинга В.2

```
if request.session.get('user_role') not in ['Техник',
'Администратор']: #Проверяем соответствие роли списку
разрешённых
    messages.error(request, 'Нет доступа') #Выводим
сообщение о запрете доступа
    return redirect('login_view') #Перенаправляем
пользователя на страницу входа при нарушении прав
    return super().dispatch(request, *args, **kwargs)
#Продолжаем стандартную обработку запроса
родительским классом
def get_queryset(self):
    queryset =
RequestFix.objects.filter(delete_date__isnull=True).select_re
lated( #Получаем активные заявки и оптимизируем запрос
подгрузкой связанных таблиц
    'requester', 'assigned_technician', 'equipment',
#Подгружаем данные о заказчике, исполнителе и
оборудовании
    'equipment__model', 'equipment__type',
'equipment__status', #Добавляем характеристики
оборудования и его статус
    'category', 'repair_stage', 'priority' #Загружаем
информацию о категории, этапе ремонта и приоритете
)
    search_query = self.request.GET.get('search', '')
#Получаем текст поискового запроса из параметров URL
    if search_query: #Если поисковая строка не пустая
        queryset = queryset.filter( #Применяем
дополнительный фильтр к выборке

Q(problem_description__icontains=search_query) | #Ищем
совпадения в описании проблемы
        Q(act_number__icontains=search_query) |
#Ищем совпадения по номеру акта

Q(requester__last_name__icontains=search_query) |
#Ищем совпадения по фамилии заказчика

Q(requester__first_name__icontains=search_query) |
#Ищем совпадения по имени заказчика

Q(equipment__inventory_number__icontains=search_query)
#Ищем совпадения по инвентарному номеру
)
    status_filter = self.request.GET.get('status', '')
#Получаем выбранный статус оборудования из URL
    if status_filter: #Если фильтр по статусу активен
        queryset =
queryset.filter(equipment__status_id=status_filter)
#Фильтруем заявки по идентификатору статуса
```

```
priority_filter = self.request.GET.get('priority', '')
#Получаем выбранный приоритет заявки из URL
    if priority_filter: #Если фильтр по приоритету
активен
        queryset = queryset.filter(priority_id=priority_filter)
#Фильтруем заявки по идентификатору приоритета
    sort_by = self.request.GET.get('sort', '-
registration_date') #Получаем параметр сортировки, по
умолчанию по дате убывания
    valid_sorts = ['registration_date', '-registration_date',
'act_number', #Определяем список допустимых
параметров сортировки
    'requester__last_name',
'equipment__status__name']
    if sort_by in valid_sorts or sort_by.replace('-', '') in
valid_sorts: #Проверяем безопасность параметра
сортировки
        queryset = queryset.order_by(sort_by)
#Применяем выбранный порядок сортировки к результату
запроса
    return queryset #Возвращаем итоговую
отфильтрованную и отсортированную выборку заявок
def get_context_data(self, **kwargs):
    context = super().get_context_data(**kwargs)
#Получаем базовый контекст из родительского класса
ListView
    context['statuses'] =
EquipmentStatus.objects.filter(delete_date__isnull=True)
#Передаём список активных статусов для выпадающего
списка
    context['priorities'] =
Priority.objects.filter(delete_date__isnull=True) #Передаём
список активных приоритетов для фильтрации
    context['search_query'] =
self.request.GET.get('search', '') #Передаём текущий
поисковый запрос для сохранения в поле ввода
    context['status_filter'] = self.request.GET.get('status',
'') #Передаём выбранный статус для подсветки активного
фильтра
    context['priority_filter'] =
self.request.GET.get('priority', '') #Передаём выбранный
приоритет для подсветки фильтра
    context['sort_by'] = self.request.GET.get('sort', '-
registration_date') #Передаём текущий параметр
сортировки для обновления кнопок
    context['user_name'] =
self.request.session.get('user_name', '') #Добавляем имя
пользователя для отображения в шапке сайта
    context['new_requests_count'] =
RequestFix.objects.filter( #Подсчитываем количество
новых заявок без исполнителя или даты завершения
    Q(delete_date__isnull=True) &
```

Продолжение листинга В.2

```
(Q(assigned_technician__isnull=True) |
Q(completion_date__isnull=True))
).count() #Получаем итоговое число обращений,
требующих внимания техника
return context #Возвращаем дополненный контекст
для рендеринга HTML-шаблона
```

#Функция формирует разбитый по категориям список новых и незавершённых заявок для панели техника.

#Метод использует ORM-запросы с логическими условиями Q, агрегацию количества и систему сообщений Django.

#Нужна для визуального разделения потока обращений на полностью новые, ожидающие назначения и находящиеся в работе.

```
def show_new_requests(request):
    if 'user_id' not in request.session: #Проверяем наличие
    идентификатора пользователя в сессии
        messages.error(request, 'Нет доступа') #Выводим
        сообщение об отказе в доступе
        return redirect('login_view') #Перенаправляем
        неавторизованного пользователя на страницу входа
        if request.session.get('user_role') not in ['Техник',
        'Администратор']: #Проверяем соответствие роли списку
        разрешённых
            messages.error(request, 'Нет доступа') #Блокируем
            доступ для пользователей без нужных прав
            return redirect('login_view') #Возвращаем
            пользователя на страницу авторизации
        try:
            all_new_requests = RequestFix.objects.filter(
            #Получаем все заявки без исполнителя или без даты
            завершения
                Q(assigned_technician__isnull=True) |
                Q(completion_date__isnull=True)).select_related(
                'requester', 'assigned_technician', 'equipment',
                'equipment__status' #Оптимизируем запрос подгрузкой
                связанных данных
            )
            completely_new = all_new_requests.filter(
            #Фильтруем заявки, у которых нет ни техника, ни даты
            выполнения
                assigned_technician__isnull=True,
                completion_date__isnull=True
            )
            needs_technician =
            all_new_requests.filter(assigned_technician__isnull=True,
            completion_date__isnull=False) #Получаем заявки с датой,
            но без назначенного исполнителя
            in_progress =
            all_new_requests.filter(assigned_technician__isnull=False,
```

```
completion_date__isnull=True) #Получаем заявки с
назначенным техником, но без даты завершения
            completely_new_count = completely_new.count()
            #Подсчитываем количество полностью новых обращений
            needs_technician_count = needs_technician.count()
            #Подсчитываем количество заявок, ожидающих
            назначения техника
            in_progress_count = in_progress.count()
            #Подсчитываем количество заявок, находящихся в
            процессе выполнения
            total_new_count = all_new_requests.count()
            #Получаем общее число необработанных заявок
            sort_by = request.GET.get('sort', '-registration_date')
            #Получаем параметр сортировки из URL-адреса
            if sort_by in ['registration_date', '-registration_date',
            'act_number']: #Проверяем допустимость выбранного
            параметра
                all_new_requests =
                all_new_requests.order_by(sort_by) #Применяем
                сортировку ко всем новым заявкам
                completely_new =
                completely_new.order_by(sort_by) #Применяем сортировку
                к категории полностью новых
                needs_technician =
                needs_technician.order_by(sort_by) #Применяем
                сортировку к категории ожидающих назначения
                in_progress = in_progress.order_by(sort_by)
            #Применяем сортировку к категории находящихся в
            работе
            context = { #Формируем словарь данных для
            передачи в HTML-шаблон
                'requests': all_new_requests, #Передаём общий
                список необработанных заявок
                'completely_new': completely_new, #Передаём
                список полностью новых обращений
                'needs_technician': needs_technician, #Передаём
                список заявок, требующих назначения исполнителя
                'in_progress': in_progress, #Передаём список
                заявок, находящихся в работе
                'completely_new_count': completely_new_count,
            #Передаём счётчик новых заявок для отображения на
            панели
                'needs_technician_count':
                needs_technician_count, #Передаём счётчик ожидающих
                назначения
                'in_progress_count': in_progress_count,
            #Передаём счётчик заявок в работе
                'total_new_count': total_new_count, #Передаём
                общий счётчик необработанных обращений
                'sort_by': sort_by, #Передаём текущий параметр
                сортировки для обновления кнопок
```

Продолжение листинга В.2

```
'user_name': request.session.get('user_name', ''),
#Передаём имя пользователя для шапки сайта
}
return render(request, 'show_new_requests.html',
context) #Рендерим страницу с переданным контекстом
данных
except Exception as e: #Обрабатываем
непредвиденные ошибки при получении данных
messages.error(request, f'Ошибка: {str(e)}')
#Выводим сообщение с текстом возникшего исключения
return redirect('show_technician') #Возвращаем
пользователя на основную страницу техника
# Функция отображает персональный список заявок,
назначенных конкретному технику.
# Метод использует ORM Django для фильтрации по ID из
сессии и модуль сообщений для уведомлений.
# Нужна для удобного отслеживания текущей загрузки
исполнителя и контроля сроков выполнения.
def my_requests(request): #Объявляем функцию
просмотра личных заявок техника
    if 'user_id' not in request.session: #Проверяем наличие
идентификатора пользователя в сессии
        messages.error(request, 'Нет доступа') #Выводим
сообщение об отказе в доступе
        return redirect('login_view') #Перенаправляем на
страницу входа
    if request.session.get('user_role') not in ['Техник',
'Администратор']: #Проверяем соответствие роли списку
разрешённых
        messages.error(request, 'Нет доступа') #Блокируем
доступ для неавторизованных ролей
        return redirect('login_view') #Возвращаем
пользователя на страницу авторизации
    try: #Начинаем блок обработки возможных ошибок
запроса
        current_technician_id = request.session['user_id']
#Извлекаем ID текущего техника из данных сессии
        requests = RequestFix.objects.filter( #Начинаем
фильтрацию заявок по ID техника и отсутствию даты
удаления
            delete_date__isnull=True,
assigned_technician_id=current_technician_id
        ).select_related( #Подгружаем связанные таблицы в
одном SQL-запросе для оптимизации
            'requester', 'assigned_technician', 'equipment',
            'equipment__model', 'equipment__type',
            'equipment__status',
            'category', 'repair_stage', 'priority'
        ).order_by('-registration_date') #Сортируем
полученные заявки по дате регистрации (от новых к
старым)
```

```
priority_filter = request.GET.get('priority', '') #Получаем
параметр фильтрации по приоритету из URL-адреса
if priority_filter: #Если фильтр приоритета был
передан и не пустой
    requests = requests.filter(priority_id=priority_filter)
#Применяем дополнительную фильтрацию к выборке
new_requests_count = RequestFix.objects.filter(
#Подготавливаем запрос для подсчёта незавершённых
заявок
    delete_date__isnull=True,
    assigned_technician_id=current_technician_id,
    completion_date__isnull=True
).count() #Выполняем запрос и получаем итоговое
число активных заявок без даты выполнения
context = { #Формируем словарь данных для
передачи в HTML-шаблон
    'requests': requests, #Передаём отфильтрованный
список заявок текущего техника
    'new_requests_count': new_requests_count,
#Передаём счётчик незавершённых обращений для
панели
    'user_name': request.session.get('user_name', ''),
#Передаём имя пользователя для отображения в шапке
сайта
    'page_title': 'Мои заявки', #Указываем заголовок
текущей страницы
    'priorities':
Priority.objects.filter(delete_date__isnull=True), #Передаём
список активных приоритетов для выпадающего меню
    'priority_filter': priority_filter, #Передаём выбранный
приоритет для подсветки в фильтре
}
return render(request, 'show_technican_requests.html',
context) #Рендерим страницу с подготовленными
данными
except Exception as e: #Ловим любые непредвиденные
исключения при обработке запроса
    messages.error(request, f'Ошибка: {str(e)}') #Выводим
сообщение с текстом возникшей ошибки
    return redirect('show_technician') #Перенаправляем
пользователя на главную страницу техника
```

```
# Функция позволяет технику или администратору
вручную создавать заявку с полным заполнением данных.
# Метод использует ORM Django для получения
справочников, обработки POST-данных и генерации
номера акта.
# Нужна для регистрации обращений, поступивших не
через форму сотрудника, или для оперативного
назначения исполнителя.
```

Продолжение листинга В.2

```
def create_request_technician(request): #Объявляем
    функцию ручного создания заявки техником
    if 'user_id' not in request.session: #Проверяем наличие
        идентификатора пользователя в сессии
        messages.error(request, 'Нет доступа') #Выводим
        сообщение об отказе в доступе
        return redirect('login_view') #Перенаправляем на
        страницу входа
    if request.session.get('user_role') not in ['Техник',
        'Администратор']: #Проверяем соответствие роли списку
        разрешённых
        messages.error(request, 'Нет доступа') #Блокируем
        доступ для неавторизованных ролей
        return redirect('login_view') #Возвращаем
        пользователя на страницу авторизации
    try: #Начинаем блок обработки возможных ошибок
        загрузки данных формы
        technicians =
        Employee.objects.filter(delete_date__isnull=True,
        role__role_name='Техник') #Получаем список активных
        сотрудников с ролью Техник
        customers =
        Employee.objects.filter(delete_date__isnull=True)
        #Получаем список всех активных сотрудников для выбора
        заказчика
        equipment_list =
        Equipment.objects.filter(delete_date__isnull=True)
        #Получаем список всего активного оборудования
        statuses =
        EquipmentStatus.objects.filter(delete_date__isnull=True)
        #Получаем справочник активных статусов техники
        categories =
        RequestCategory.objects.filter(delete_date__isnull=True)
        #Получаем справочник категорий неисправностей
        priorities =
        Priority.objects.filter(delete_date__isnull=True) #Получаем
        справочник приоритетов выполнения
        stages =
        RepairStage.objects.filter(delete_date__isnull=True)
        #Получаем справочник этапов ремонта
        if request.method == 'POST': #Если форма была
        отправлена методом POST
            try: #Начинаем вложенный блок обработки данных
                формы
                customer_id = request.POST.get('requester')
                #Извлекаем ID сотрудника-заявителя из данных формы
                equipment_inv_num =
                request.POST.get('equipment') #Извлекаем инвентарный
                номер выбранного оборудования
```

```
                technician_id =
                request.POST.get('assigned_technician') #Извлекаем ID
                назначенного техника-исполнителя
                description =
                request.POST.get('problem_description',
                '').strip()
                #Извлекаем и очищаем текстовое описание проблемы
                category_id = request.POST.get('category')
                #Извлекаем ID выбранной категории заявки
                priority_id = request.POST.get('priority')
                #Извлекаем ID выбранного приоритета
                stage_id = request.POST.get('repair_stage')
                #Извлекаем ID выбранного этапа ремонта
                status_id = request.POST.get('status') #Извлекаем
                ID выбранного статуса оборудования
                date_done_str =
                request.POST.get('completion_date', '') #Извлекаем строку
                даты выполнения (если указана)
                if not description or not equipment_inv_num or not
                customer_id: #Проверяем заполненность обязательных
                полей
                messages.error(request,
                'Заполните
                обязательные поля') #Выводим предупреждение о
                необходимости заполнения
                return redirect('create_request_technician')
                #Возвращаем пользователя на страницу формы
                customer = Employee.objects.get(id=customer_id,
                delete_date__isnull=True) #Находим объект заказчика в
                базе данных
                equipment =
                Equipment.objects.get(inventory_number=equipment_inv_num,
                delete_date__isnull=True) #Находим объект
                оборудования по инвентарному номеру
                assigned_tech =
                Employee.objects.get(id=technician_id) if technician_id else
                None #Находим объект исполнителя или оставляем None
                category =
                RequestCategory.objects.get(id=category_id) if category_id
                else None #Находим объект категории или оставляем
                None
                priority = Priority.objects.get(id=priority_id) if
                priority_id else None #Находим объект приоритета или
                оставляем None
                stage = RepairStage.objects.get(id=stage_id) if
                stage_id else None #Находим объект этапа ремонта или
                оставляем None
                date_done = None #Инициализируем
                переменную даты выполнения пустым значением
                if date_done_str: #Если строка даты была
                передана в форме
```

Продолжение листинга В.2

```
        date_done = timezone.now()
    timezone.make_aware(datetime.strptime(date_done_str,
'%Y-%m-%d')) #Преобразуем строку в объект datetime с
учётом часового пояса
    last_act = RequestFix.objects.all().order_by('-
act_number').first() #Находим последнюю заявку для
расчёта следующего номера акта
    new_act_number = (last_act.act_number + 1) if
last_act else 1 #Вычисляем новый номер инкрементом или
начинаем с 1
    req = RequestFix( #Создаём новый экземпляр
модели заявки
        act_number=new_act_number, #Присваиваем
сгенерированный номер акта
        problem_description=description, #Сохраняем
описание проблемы
        registration_date=timezone.now(), #Фиксируем
текущую дату и время регистрации
        completion_date=date_done, #Устанавливаем
дату выполнения (если указана)
        requester=customer, #Привязываем заявку к
заказчику
        assigned_technician=assigned_tech,
#Привязываем заявку к исполнителю
        equipment=equipment, #Привязываем заявку к
оборудованию
        category=category, #Устанавливаем
категорию
        repair_stage=stage, #Устанавливаем этап
ремонта
        priority=priority #Устанавливаем приоритет
    )
    req.save() #Сохраняем созданную заявку в базе
данных
    if status_id: #Если в форме был выбран статус
оборудования
        equipment.status =
EquipmentStatus.objects.get(id=status_id) #Находим
объект выбранного статуса
        equipment.save() #Обновляем статус
оборудования в базе данных
        messages.success(request, 'Заявка создана!')
#Выводим сообщение об успешном создании
        return redirect('show_technician')
#Перенаправляем на страницу списка заявок техника
    except Exception as e: #Ловим ошибки при
сохранении или получении связанных объектов
        messages.error(request, f'Ошибка: {str(e)}')
#Выводим сообщение с текстом возникшей ошибки
    context = { #Формируем словарь данных для
отображения формы при GET-запросе
```

```
        'technicians': technicians, #Передаём список
техников для выбора исполнителя
        'customers': customers, #Передаём список
сотрудников для выбора заказчика
        'equipment_list': equipment_list, #Передаём список
доступного оборудования
        'statuses': statuses, #Передаём список статусов
техники
        'categories': categories, #Передаём список
категорий неисправностей
        'priorities': priorities, #Передаём список приоритетов
        'stages': stages, #Передаём список этапов ремонта
        'now': timezone.now(), #Передаём текущую дату
для полей ввода
        'user_name': request.session.get('user_name', ''),
#Передаём имя пользователя для шапки сайта
    }
    return render(request, 'create_request_technician.html',
context) #Рендерим страницу создания заявки с
контекстом
    except Exception as e: #Ловим критические ошибки на
уровне загрузки справочников
        messages.error(request, f'Ошибка: {str(e)}') #Выводим
сообщение об общей ошибке
        return redirect('show_technician') #Перенаправляем на
страницу техника
    #Функция обновляет данные заявки, включая описание,
исполнителя, статус и список использованных запчастей
или услуг.
    #Метод использует ORM Django для загрузки
справочников, обработки POST-данных и безопасного
сохранения изменений.
    #Нужна для фиксации хода ремонтных работ, расчёта
затрат и перевода заявки на следующий этап
выполнения.
    def edit_request(request, request_id):
        if 'user_id' not in request.session: #Проверяем наличие
идентификатора пользователя в сессии
            messages.error(request, 'Необходимо
авторизоваться') #Выводим сообщение о необходимости
входа
            return redirect('login_view') #Перенаправляем на
страницу авторизации
        user_role = request.session.get('user_role', '')
#Извлекаем роль текущего пользователя из данных
сессии
        if user_role not in ['Техник', 'Администратор']:
#Проверяем соответствие роли списку разрешённых
            messages.error(request, 'Нет доступа') #Блокируем
доступ для неавторизованных ролей
            return redirect('login_view') #Возвращаем
пользователя на страницу входа
```

Продолжение листинга В.2

```
try:
    req = get_object_or_404(RequestFix,
act_number=request_id) #Находим заявку по номеру акта
или возвращаем ошибку 404
    technicians =
Employee.objects.filter(delete_date__isnull=True,
role__role_name='Техник') #Получаем список активных
техников
    statuses =
EquipmentStatus.objects.filter(delete_date__isnull=True)
#Получаем справочник активных статусов оборудования
    equipment_list =
Equipment.objects.filter(delete_date__isnull=True)
#Получаем список всего активного оборудования
    categories =
RequestCategory.objects.filter(delete_date__isnull=True)
#Получаем справочник категорий неисправностей
    priorities =
Priority.objects.filter(delete_date__isnull=True) #Получаем
справочник приоритетов заявок
    stages =
RepairStage.objects.filter(delete_date__isnull=True)
#Получаем справочник этапов ремонта
    spare_parts_list =
SparePart.objects.filter(delete_date__isnull=True)
#Получаем список доступных запчастей
    services_list =
ThirdPartyService.objects.filter(delete_date__isnull=True)
#Получаем список сторонних услуг
    if request.method == 'POST': #Если форма была
отправлена методом POST
        try:
            technician_id =
request.POST.get('assigned_technician') #Извлекаем ID
назначенного техника
            description =
request.POST.get('problem_description', '').strip()
#Извлекаем и очищаем описание проблемы
            equipment_inv_num =
request.POST.get('equipment') #Извлекаем инвентарный
номер оборудования
            category_id = request.POST.get('category')
#Извлекаем ID категории заявки
            priority_id = request.POST.get('priority')
#Извлекаем ID приоритета
            stage_id = request.POST.get('repair_stage')
#Извлекаем ID этапа ремонта
            status_id = request.POST.get('status')
#Извлекаем ID статуса оборудования
```

```
date_done_str =
request.POST.get('completion_date', '') #Извлекаем строку
даты выполнения
        if not description: #Проверяем заполненность
обязательного поля описания
            messages.error(request, 'Описание
проблемы обязательно') #Выводим сообщение об ошибке
валидации
        return redirect('edit_request',
request_id=request_id) #Возвращаем на страницу
редактирования
        req.problem_description = description
#Обновляем поле описания проблемы
        req.equipment =
Equipment.objects.get(inventory_number=equipment_inv_nu
m) #Привязываем заявку к оборудованию
        req.assigned_technician =
Employee.objects.get(id=technician_id) if technician_id else
None #Назначаем техника или оставляем пустым
        if category_id: #Если категория выбрана
            req.category =
RequestCategory.objects.get(id=category_id) #Обновляем
поле категории
        if priority_id: #Если приоритет выбран
            req.priority = Priority.objects.get(id=priority_id)
#Обновляем поле приоритета
        if stage_id: #Если этап ремонта выбран
            req.repair_stage =
RepairStage.objects.get(id=stage_id) #Обновляем поле
этапа
        if date_done_str: #Если дата выполнения
указана
            req.completion_date =
timezone.make_aware(datetime.strptime(date_done_str,
'%Y-%m-%d')) #Преобразуем строку в дату с учётом
часового пояса
        else: #Если дата не указана
            req.completion_date = None #Очищаем поле
даты выполнения
        req.save() #Сохраняем обновлённые
основные поля заявки в базе данных
        if status_id: #Если статус оборудования
выбран
            req.equipment.status =
EquipmentStatus.objects.get(id=status_id) #Находим
объект выбранного статуса
            req.equipment.save() #Обновляем статус
оборудования в базе данных
        spare_part_ids =
request.POST.getlist('spare_part_id[]') #Получаем список ID
запчастей из формы
```

Продолжение листинга В.2

```
        spare_part_prices = request.POST.getlist('spare_part_price[]') #Получаем
#список цен на запчасти
        spare_part_quantities = request.POST.getlist('spare_part_quantity[]') #Получаем
#список количеств запчастей
        if spare_part_ids: #Если список запчастей не
#пустой
            req.used_spare_parts.all().delete() #Удаляем
#все старые привязанные запчасти к заявке
            for i, part_id in enumerate(spare_part_ids):
#Проходимся по каждой запчасти в списке
                if part_id and spare_part_quantities[i]:
#Проверяем наличие ID и количества
                    try:
                        part = SparePart.objects.get(id=part_id) #Находим
#объект
#запчасти в базе
                        price = spare_part_prices[i] if
#spare_part_prices[i] else part.cost #Берём цену из формы
#или по умолчанию
                        quantity = int(spare_part_quantities[i])
#Преобразуем количество в целое число
                        RequestSparePart.objects.create(
#Создаём новую запись связи заявки и запчасти
                            request=req, #Привязываем к
#текущей заявке
                            spare_part=part, #Привязываем к
#объекту запчасти
                            quantity=quantity, #Указываем
#использованное количество
                            cost_at_repair=price #Фиксируем
#цену на момент ремонта
                        )
                    except Exception as e: #Ловим ошибки
#при сохранении запчасти
                        messages.warning(request, f'Ошибка
#сохранения запчасти: {str(e)}') #Выводим предупреждение
                        service_ids = request.POST.getlist('service_id[]')
#Получаем список ID услуг из формы
                        service_prices = request.POST.getlist('service_price[]') #Получаем
#список
#цен на услуги
                        service_quantities = request.POST.getlist('service_quantity[]') #Получаем
#список
#количеств услуг
                        service_files = request.FILES.getlist('service_file[]') #Получаем
#список
#загруженных файлов чеков
                        if service_ids: #Если список услуг не пустой
```

```
                            req.used_services.all().delete() #Удаляем
#все старые привязанные услуги к заявке
                            for i, service_id in enumerate(service_ids):
#Проходимся по каждой услуге в списке
                                if service_id and service_quantities[i]:
#Проверяем наличие ID и количества
                                    try:
                                        service = ThirdPartyService.objects.get(id=service_id) #Находим
#объект
#услуги в базе
                                        price = service_prices[i] if
#service_prices[i] else service.cost #Берём цену из формы
#или по умолчанию
                                        quantity = int(service_quantities[i])
#Преобразуем количество в целое число
                                        receipt_file = None #Инициализируем
#переменную для файла чека
                                        if i < len(service_files) and
#service_files[i]: #Проверяем наличие загруженного файла
#для текущей услуги
                                            file_obj = File.objects.create(file=service_files[i]) #Сохраняем
#файл
#чека в базе
                                            receipt_file = file_obj
#Присваиваем объект файла переменной
                                            RequestService.objects.create(
#Создаём новую запись связи заявки и услуги
                                                request=req, #Привязываем к
#текущей заявке
                                                service=service, #Привязываем к
#объекту услуги
                                                quantity=quantity, #Указываем
#количество оказанных услуг
                                                cost_at_repair=price, #Фиксируем
#цену на момент ремонта
                                                receipt_file=receipt_file
#Привязываем файл чека (если есть)
                                            )
                                        except Exception as e: #Ловим ошибки
#при сохранении услуги
                                            messages.warning(request, f'Ошибка
#сохранения услуги: {str(e)}') #Выводим предупреждение
                                            messages.success(request, f'Заявка
#№{req.act_number} обновлена!') #Выводим сообщение об
#успешном обновлении
                                            return redirect('show_technician')
#Перенаправляем на страницу списка заявок техника
                                        except Exception as e: #Ловим критические
#ошибки при обработке POST-запроса
                                            messages.error(request, f'Ошибка
#при
#сохранении: {str(e)}') #Выводим сообщение об ошибке
```


Продолжение листинга В.2

```
        return redirect('edit_request',
request_id=request_id) #Возвращаем на страницу
редактирования

        context = { #Формируем словарь данных для
передачи в HTML-шаблон
            'request_obj': req, #Передаём объект текущей
заявки
            'technicians': technicians, #Передаём список
техников для выбора
            'statuses': statuses, #Передаём список статусов
оборудования
            'equipment_list': equipment_list, #Передаём
список оборудования
            'categories': categories, #Передаём список
категорий
            'priorities': priorities, #Передаём список
приоритетов
            'stages': stages, #Передаём список этапов
ремонта
            'spare_parts_list': spare_parts_list, #Передаём
список запчастей
            'services_list': services_list, #Передаём список
услуг
            'user_name': request.session.get('user_name', ''),
#Передаём имя пользователя для шапки
            'user_role': user_role, #Передаём роль
пользователя для управления видимостью
        }
        return render(request, 'edit_request.html', context)
#Рендерим страницу редактирования с контекстом
        except Exception as e: #Ловим ошибки на уровне
загрузки заявки или справочников
            messages.error(request, f'Ошибка: {str(e)}')
#Выводим сообщение об общей ошибке
            return redirect('show_technician') #Перенаправляем
на страницу техника

        #Функция выполняет мягкое удаление заявки путём
установки текущей даты в поле удаления.
        #Метод использует проверку авторизации, валидацию
прав доступа и ORM Django для обновления статуса
записи.
        #Нужна для архивации выполненных обращений
сотрудниками без физического стирания истории
ремонтов.
        def delete_request_customer(request, request_id):
            if 'user_id' not in request.session: #Проверяем наличие
идентификатора пользователя в сессии
                messages.error(request, 'Необходимо
авторизоваться') #Выводим сообщение о необходимости
входа
```

```
            return redirect('login_view') #Перенаправляем на
страницу авторизации
            if request.session.get('user_role') not in ['Сотрудник',
'Техник', 'Администратор']: #Проверяем соответствие
роли списку разрешённых
                messages.error(request, 'Нет доступа') #Блокируем
доступ для неавторизованных ролей
                return redirect('login_view') #Возвращаем
пользователя на страницу входа
            if request.method == 'POST': #Разрешаем удаление
только через безопасный POST-запрос
                try:
                    req =
RequestFix.objects.get(act_number=request_id) #Находим
заявку в базе данных по номеру акта
                    if req.requester.id != request.session['user_id']:
#Проверяем, является ли текущий пользователь автором
заявки
                        messages.error(request, 'Это не ваша заявка')
#Запрещаем удаление чужих обращений
                        return redirect('show_customer') #Возвращаем
пользователя на страницу его заявок
                    if not req.completion_date: #Проверяем наличие
даты выполнения заявки
                        messages.error(request, 'Можно удалить
только выполненную заявку') #Запрещаем архивацию
незавершённых работ
                        return redirect('show_customer') #Возвращаем
пользователя на страницу его заявок
                    req.delete_date = timezone.now()
#Устанавливаем текущую дату и время как метку
удаления
                    req.save() #Сохраняем изменения в базе данных
                    messages.success(request, 'Заявка удалена
(архивирована)') #Выводим сообщение об успешной
архивации
                except Exception as e: #Ловим непредвиденные
ошибки выполнения запроса
                    messages.error(request, f'Ошибка: {str(e)}')
#Выводим сообщение с текстом возникшей ошибки
                    return redirect('show_customer') #Перенаправляем
пользователя обратно на страницу списка заявок
                #Функция отображает список архивированных заявок
для пользователей с правами техника или
администратора.
                #Метод использует ORM-фильтрацию по полю даты
удаления и подгрузку связанных данных через
select_related.
                #Нужна для контроля удалённых обращений и
предоставления возможности их восстановления или
полного удаления.
                def show_deleted_requests(request):
```

Продолжение листинга В.2

```
if 'user_id' not in request.session: #Проверяем наличие
идентификатора пользователя в сессии
    messages.error(request, 'Нет доступа') #Выводим
сообщение об отказе в доступе
    return redirect('login_view') #Перенаправляем на
страницу входа
if request.session.get('user_role') not in ['Техник',
'Администратор']: #Проверяем соответствие роли списку
разрешённых
    messages.error(request, 'Нет доступа') #Блокируем
доступ для неавторизованных ролей
    return redirect('login_view') #Возвращаем
пользователя на страницу авторизации
try:
    requests =
RequestFix.objects.filter(delete_date__isnull=False).select_r
elated( #Получаем заявки с установленной датой
удаления и оптимизируем запрос
        'requester', 'assigned_technician', 'equipment',
#Подгружаем данные о заказчике, исполнителе и
оборудовании
        'equipment__model', 'equipment__type',
'equipment__status').order_by('-delete_date') #Добавляем
характеристики техники и сортируем по дате удаления
    context = { #Формируем словарь данных для
передачи в HTML-шаблон
        'requests': requests, #Передаём
отфильтрованный список архивированных заявок
        'user_name': request.session.get('user_name', ''),
#Передаём имя текущего пользователя для шапки сайта
        'page_title': 'Мягко удалённые заявки',
#Указываем заголовок страницы архива
    }
    return render(request, 'show_deleted_requests.html',
context) #Рендерим страницу архива с подготовленными
данными
except Exception as e: #Повим непредвиденные
исключения при выполнении запроса
    messages.error(request, f'Ошибка: {str(e)}')
#Выводим сообщение с текстом возникшей ошибки
    return redirect('show_technician') #Перенаправляем
пользователя на главную страницу техника

#Функция восстанавливает архивированную заявку
путём очистки поля даты удаления.
#Метод использует ORM для поиска записи по номеру
акта и обновления состояния в базе данных.
#Нужна для отмены ошибочного удаления и быстрого
возврата обращения в активный список для повторной
обработки.
def restore_request(request, request_id):
```

```
if 'user_id' not in request.session: #Проверяем наличие
идентификатора пользователя в сессии
    messages.error(request, 'Нет доступа') #Выводим
сообщение об отказе в доступе
    return redirect('login_view') #Перенаправляем на
страницу входа
if request.session.get('user_role') not in ['Техник',
'Администратор']: #Проверяем соответствие роли списку
разрешённых
    messages.error(request, 'Нет доступа') #Блокируем
доступ для неавторизованных ролей
    return redirect('login_view') #Возвращаем
пользователя на страницу авторизации
if request.method == 'POST': #Разрешаем
восстановление только через безопасный POST-запрос
    try:
        req =
RequestFix.objects.get(act_number=request_id) #Находим
заявку в базе данных по уникальному номеру акта
        req.delete_date = None #Очищаем поле даты
удаления, возвращая запись в активное состояние
        req.save() #Применяем изменения к базе
данных
        messages.success(request, f'Заявка
№{req.act_number} восстановлена!') #Информируем об
успешном восстановлении
    except Exception as e: #Повим ошибки, если запись
не найдена или повреждена
        messages.error(request, f'Ошибка: {str(e)}')
#Показываем техническое сообщение об ошибке
    return redirect('show_deleted_requests') #Возвращаем
пользователя на страницу архива
#Функция выполняет полное физическое удаление
заявки из базы данных без возможности восстановления.
#Метод использует проверку сессии для авторизации,
ORM Django для поиска записи и систему сообщений для
вывода результата.
#Нужна для очистки реестра от ошибочных,
дублирующих или отменённых обращений
пользователями с расширенными правами.
def delete_request_technician(request, request_id):
    if 'user_id' not in request.session: #Проверяем наличие
идентификатора пользователя в сессии
        messages.error(request, 'Нет доступа') #Выводим
сообщение об отказе в доступе
        return redirect('login_view') #Перенаправляем на
страницу входа в систему
    if request.session.get('user_role') not in ['Техник',
'Администратор']: #Проверяем соответствие роли списку
разрешённых
        messages.error(request, 'Нет доступа') #Блокируем
доступ для неавторизованных ролей
```

Продолжение листинга В.2

```
        return redirect('login_view') #Возвращаем
пользователя на страницу авторизации
        if request.method == 'POST': #Разрешаем удаление
только через безопасный POST-запрос
            try:
                req = RequestFix.objects.get(act_number=request_id) #Находим
заявку в базе данных по уникальному номеру акта
                req.delete() #Полностью стираем запись из базы
данных
                messages.success(request, 'Заявка полностью
удалена') #Информируем об успешном выполнении
операции
            except Exception as e: #Ловим любые
непредвиденные ошибки выполнения запроса
                messages.error(request, f'Ошибка: {str(e)}')
#Выводим сообщение с текстом возникшего исключения
            return redirect('show_technician') #Перенаправляем
пользователя обратно на страницу списка заявок

#Классовое представление формирует и отображает
реестр активного оборудования с поддержкой поиска и
фильтрации.
#Метод использует встроенный ListView Django, ORM с
select_related для оптимизации запросов и валидацию
сессии.
#Нужна для предоставления технику и руководителю
удобного интерфейса управления парком техники с
защитой от несанкционированного доступа.
class EquipmentListView(ListView):
    model = Equipment #Указываем модель базы данных,
данные которой будут выводиться в списке
    template_name = 'show_equipment.html' #Задаём путь
к HTML-шаблону для отрисовки страницы оборудования
    context_object_name = 'equipment_list' #Присваиваем
имя переменной для передачи списка объектов в шаблон
    def dispatch(self, request, *args, **kwargs):
        if 'user_id' not in request.session: #Проверяем
наличие идентификатора пользователя в сессии
            messages.error(request, 'Нет доступа') #Выводим
сообщение об отказе в доступе
            return redirect('login_view') #Перенаправляем
неавторизованного пользователя на страницу входа
        user_role = request.session.get('user_role', '')
#Извлекаем роль текущего пользователя из данных
сессии
        if user_role not in ['Техник', 'Руководитель',
'Администратор']: #Проверяем соответствие роли списку
разрешённых
            messages.error(request, 'Нет доступа')
#Блокируем доступ для пользователей без нужных прав
```

```
        return redirect('login_view') #Возвращаем
пользователя на страницу авторизации
        return super().dispatch(request, *args, **kwargs)
#Продолжаем стандартную обработку запроса
родительским классом

    def get_queryset(self):
        queryset = Equipment.objects.filter(delete_date__isnull=True).select_re
lated() #Получаем активное оборудование и оптимизируем
запрос подгрузкой связанных таблиц
        'model', 'model__manufacturer', 'type', 'status',
#Подгружаем данные о модели, производителе, типе и
статусе
        'assigned_office', 'assigned_office__building',
'warranty', 'photo' #Добавляем кабинет, корпус, гарантию и
фотографию
    )
    search_query = self.request.GET.get('search', '')
#Получаем текст поискового запроса из параметров URL
    if search_query: #Если поисковая строка не пустая
        queryset = queryset.filter( #Применяем
дополнительный фильтр к выборке
            Q(inventory_number__icontains=search_query)
| #Ищем совпадения в инвентарном номере
            Q(model__name__icontains=search_query)
#Ищем совпадения в названии модели
        )
        type_filter = self.request.GET.get('type', '')
#Получаем выбранный тип оборудования из URL
        if type_filter: #Если фильтр по типу активен
            queryset = queryset.filter(type_id=type_filter)
#Фильтруем заявки по идентификатору типа
        status_filter = self.request.GET.get('status', '')
#Получаем выбранный статус оборудования из URL
        if status_filter: #Если фильтр по статусу активен
            queryset = queryset.filter(status_id=status_filter)
#Фильтруем заявки по идентификатору статуса
        sort_by = self.request.GET.get('sort',
'inventory_number') #Получаем параметр сортировки, по
умолчанию по инвентарному номеру
        if sort_by in ['inventory_number', '-inventory_number',
'model__name', 'status__name']: #Проверяем
допустимость параметра сортировки
            queryset = queryset.order_by(sort_by)
#Применяем выбранный порядок сортировки к результату
запроса
        return queryset #Возвращаем итоговую
отфильтрованную и отсортированную выборку
оборудования

    def get_context_data(self, **kwargs):
```

Продолжение листинга В.2

```
context = super().get_context_data(**kwargs)
#Получаем базовый контекст из родительского класса
ListView

context['types'] =
EquipmentType.objects.filter(delete_date__isnull=True)
#Передаём список активных типов оборудования для
фильтра

context['statuses'] =
EquipmentStatus.objects.filter(delete_date__isnull=True)
#Передаём список активных статусов для фильтра

context['search_query'] =
self.request.GET.get('search', "") #Передаём текущий
поисковый запрос для сохранения в поле ввода

context['type_filter'] = self.request.GET.get('type', "")
#Передаём выбранный тип для подсветки в фильтре

context['status_filter'] = self.request.GET.get('status',
"") #Передаём выбранный статус для подсветки в фильтре

context['sort_by'] = self.request.GET.get('sort',
'inventory_number') #Передаём текущий параметр
сортировки для обновления кнопок

context['user_name'] =
self.request.session.get('user_name', "") #Добавляем имя
пользователя для отображения в шапке сайта

context['user_role'] =
self.request.session.get('user_role', "") #Добавляем роль
пользователя для управления видимостью элементов

context['MEDIA_URL'] = 'media' #Передаём
базовый URL для корректного отображения загруженных
изображений

context['today'] = timezone.now().date() #Передаём
текущую дату для сравнения и подсветки истёкших
гарантий

return context #Возвращаем дополненный контекст
для рендеринга HTML-шаблона
```

#Функция формирует детальную историю ремонтов конкретной единицы оборудования и рассчитывает общие финансовые затраты.

#Метод использует ORM-запросы с prefetch_related для эффективной загрузки связанных данных и циклы Python для агрегации сумм.

#Нужна для анализа надёжности техники, оценки стоимости владения и принятия решений о замене или плановом обслуживании.

```
def show_equipment_history(request, inventory_number):
    if 'user_id' not in request.session: #Проверяем наличие
    идентификатора пользователя в сессии

        messages.error(request, 'Нет доступа') #Выводим
        сообщение об отказе в доступе

    return redirect('login_view') #Перенаправляем
    неавторизованного пользователя на страницу входа
```

```
if request.session.get('user_role') not in ['Техник',
'Руководитель', 'Администратор']: #Проверяем
соответствие роли списку разрешённых

    messages.error(request, 'Нет доступа') #Блокируем
    доступ для пользователей без нужных прав

    return redirect('login_view') #Возвращаем
    пользователя на страницу реестра оборудования

try:

    equipment =
get_object_or_404(Equipment.objects.select_related('model'
, 'type', 'status'), #Находим оборудование по инвентарному
номеру с подгрузкой справочников

    inventory_number=inventory_number,
#Фильтруем по переданному в URL инвентарному номеру

    delete_date__isnull=True #Исключаем
    архивированные записи из выборки

)

    repair_history =
RequestFix.objects.filter(equipment=equipment, delete_date__
isnull=True #Получаем все активные заявки, связанные с
текущим оборудованием

    ).select_related('requester',
'assigned_technician').prefetch_related( #Подгружаем
    данные о заказчиках, техниках и связанных запчастях

    'used_spare_parts', 'used_services').order_by('-
    registration_date') #Добавляем услуги и сортируем
    историю по дате регистрации

    breakdown_count = repair_history.count()
#Подсчитываем общее количество выполненных
ремонтов для данной техники

    total_cost = 0 #Инициализируем переменную для
    накопления суммарной стоимости всех работ

    for req in repair_history: #Проходимся по каждой
    заявке в истории ремонтов

        spare_parts_cost = sum( #Суммируем стоимости
        всех активных запчастей в текущей заявке

        float(item.total_cost or 0) for item in
        req.used_spare_parts.filter(delete_date__isnull=True))

        services_cost = sum( #Суммируем стоимости
        всех активных услуг в текущей заявке

        float(item.total_cost or 0) for item in
        req.used_services.filter(delete_date__isnull=True))

        req.total_cost = spare_parts_cost + services_cost
#Записываем итоговую стоимость ремонта
    непосредственно в объект заявки

        total_cost += req.total_cost #Прибавляем
        стоимость текущего ремонта к общей сумме

    avg_cost = total_cost / breakdown_count if
    breakdown_count > 0 else 0 #Вычисляем среднюю
    стоимость одного ремонта или ноль при отсутствии
    заявок
```

Продолжение листинга В.2

```
context = { #Формируем словарь данных для
передачи в HTML-шаблон
    'equipment': equipment, #Передаём объект
оборудования для отображения его характеристик
    'repair_history': repair_history, #Передаём
отсортированный список всех выполненных заявок
    'breakdown_count': breakdown_count, #Передаём
общее количество зафиксированных поломок
    'total_cost': round(total_cost, 2), #Передаём
округлённую до двух знаков итоговую сумму затрат
    'avg_cost': round(avg_cost, 2), #Передаём
округлённую до двух знаков среднюю стоимость ремонта
    'user_name': request.session.get('user_name', ''),
#Передаём ФИО авторизованного пользователя
    'user_role': request.session.get('user_role', ''),
#Передаём системную роль пользователя
    'can_see_costs': (request.session.get('user_role')
in ['Руководитель', 'Администратор']), #Устанавливаем
флаг видимости финансовых данных в зависимости от
роли
}
return render(request,
'show_equipment_history.html', context) #Рендерим
страницу истории ремонтов с подстановкой
сформированного контекста
except Exception as e: #Повим непредвиденные
исключения при выполнении запроса
    messages.error(request, f'Ошибка: {str(e)}')
#Выводим сообщение с текстом возникшей ошибки
    return redirect('show_equipment') #Перенаправляем
пользователя на страницу реестра оборудования
    #Функция обрабатывает асинхронное создание нового
типа оборудования через POST-запрос без перезагрузки
страницы.
    #Метод использует декоратор require_POST, ORM
Django для поиска дубликатов и JsonResponse для
мгновенной отдачи данных.
    #Нужна для динамического пополнения справочника
типов оборудования прямо из формы добавления
техники.
    @require_POST
    def create_equipment_type(request):
        if 'user_id' not in request.session: #Проверяем наличие
идентификатора пользователя в сессии
            return JsonResponse({'error': 'Нет доступа'},
status=403) #Возвращаем JSON-ответ с кодом ошибки
доступа
            if request.session.get('user_role') not in
['Руководитель', 'Администратор']: #Проверяем
соответствие роли списку разрешённых
```

```
return JsonResponse({'error': 'Нет доступа'},
status=403) #Блокируем доступ для неавторизованных
ролей
try:
    name = request.POST.get('name', '').strip()
#Извлекаем и очищаем название типа из POST-данных
    if not name: #Проверяем, что название не пустое
        return JsonResponse({'error': 'Название
обязательно'}, status=400) #Возвращаем ошибку
валидации на клиент
    existing =
EquipmentType.objects.filter(name__iexact=name,
delete_date__isnull=True).first() #Ищем существующий тип
без учёта регистра
    if existing: #Если тип уже найден в базе данных
        return JsonResponse({'id': existing.id, 'name':
existing.name, 'created': False}) #Возвращаем ID
существующего типа без создания нового
        new_type = EquipmentType(name=name) #Создаём
новый экземпляр модели типа оборудования
        new_type.save() #Сохраняем новый тип в базе
данных
    return JsonResponse({'id': new_type.id, 'name':
new_type.name, 'created': True}) #Возвращаем данные
созданного типа с флагом успеха
except Exception as e: #Повим любые
непредвиденные исключения при выполнении
    return JsonResponse({'error': str(e)}, status=500)
#Возвращаем текст ошибки с кодом серверной ошибки

#Функция регистрирует новую модель оборудования и
привязывает её к выбранному производителю через
AJAX-запрос.
#Метод опирается на проверку сессии, ORM-запросы к
модели EquipmentModel и формирование JSON-ответа.
#Нужна для быстрого добавления моделей в
справочник во время работы с формой создания
оборудования.
@require_POST
def create_equipment_model(request):
    if 'user_id' not in request.session: #Проверяем
авторизацию пользователя в системе
        return JsonResponse({'error': 'Нет доступа'},
status=403) #Отклоняем запрос кодом 403
        if request.session.get('user_role') not in
['Руководитель', 'Администратор']: #Проверяем наличие
прав руководителя или администратора
            return JsonResponse({'error': 'Нет доступа'},
status=403) #Запрещаем выполнение операции
        try:
```

Продолжение листинга В.2

```
name = request.POST.get('name', '').strip()
#Получаем название модели из формы и убираем
лишние пробелы

manufacturer_id = request.POST.get('manufacturer')
#Извлекаем идентификатор производителя, если он
выбран

if not name: #Проверяем обязательность
заполнения названия

    return JsonResponse({'error': 'Название
обязательно'}, status=400) #Возвращаем ошибку
валидации на клиент

existing =
EquipmentModel.objects.filter(name__iexact=name,
delete_date__isnull=True).first() #Выполняем поиск
дубликата модели в активных записях

if existing: #Если модель уже существует

    return JsonResponse({'id': existing.id, 'name':
existing.name, 'created': False}) #Возвращаем данные
существующей записи

new_model = EquipmentModel(name=name)
#Инициализируем новый объект модели оборудования

if manufacturer_id: #Если производитель был
указан в запросе

    new_model.manufacturer_id = manufacturer_id
#Привязываем модель к выбранному производителю

    new_model.save() #Сохраняем новую запись
модели в базе данных

    return JsonResponse({'id': new_model.id, 'name':
new_model.name, 'created': True}) #Отправляем успешный
ответ с ID и названием

except Exception as e: #Обрабатываем критические
ошибки выполнения

    return JsonResponse({'error': str(e)}, status=500)
#Возвращаем подробное сообщение об ошибке

#Функция добавляет нового производителя
оборудования в справочник через асинхронный POST-
запрос.

#Метод использует сессионную авторизацию, ORM-
фильтрацию модели Manufacturer и генерацию JSON-
ответа.

#Нужна для расширения списка производителей на
лету, обеспечивая целостность данных и предотвращая
дубликаты.

@require_POST
def create_manufacturer(request):

    if 'user_id' not in request.session: #Проверяем наличие
активного сеанса пользователя

        return JsonResponse({'error': 'Нет доступа'},
status=403) #Блокируем запрос без авторизации
```

```
if request.session.get('user_role') not in
['Руководитель', 'Администратор']: #Проверяем
соответствие роли требованиям

    return JsonResponse({'error': 'Нет доступа'},
status=403) #Отказываем в выполнении операции

try:

    name = request.POST.get('name', '').strip()
#Извлекаем название производителя из POST-данных

    if not name: #Проверяем, что поле не пустое

        return JsonResponse({'error': 'Название
обязательно'}, status=400) #Возвращаем ошибку
валидации формы

    existing =
Manufacturer.objects.filter(name__iexact=name,
delete_date__isnull=True).first() #Ищем существующего
производителя с учётом регистра

    if existing: #Если производитель уже записан в базе

        return JsonResponse({'id': existing.id, 'name':
existing.name, 'created': False}) #Возвращаем ID
существующего объекта без создания нового

    new_manufacturer = Manufacturer(name=name)
#Создаём экземпляр нового производителя

    new_manufacturer.save() #Фиксируем запись в базе
данных

    return JsonResponse({ #Формируем JSON-ответ с
результатами операции

        'id': new_manufacturer.id, #Передаём уникальный
идентификатор созданного производителя

        'name': new_manufacturer.name, #Передаём
название для отображения на клиенте

        'created': True #Указываем флаг успешного
создания

    })

except Exception as e: #Ловим исключения на уровне
сервера

    return JsonResponse({'error': str(e)}, status=500)
#Возвращаем текст ошибки с соответствующим HTTP-
кодом

#Функция обрабатывает асинхронное создание нового
кабинета через AJAX-запрос без перезагрузки страницы.

#Метод использует декоратор require_POST, ORM
Django для поиска дубликатов и JsonResponse для
мгновенной передачи данных.

#Нужна для оперативного пополнения справочника
кабинетов прямо из форм добавления оборудования или
сотрудников.

@require_POST
def create_office(request):

    if 'user_id' not in request.session: #Проверяем наличие
активного сеанса пользователя

        return JsonResponse({'error': 'Нет доступа'},
status=403) #Возвращаем JSON с кодом ошибки доступа
```

Продолжение листинга В.2

```
if request.session.get('user_role') not in
['Руководитель', 'Администратор']: #Проверяем
соответствие роли списку разрешённых
    return JsonResponse({'error': 'Нет доступа'},
status=403) #Блокируем выполнение для
неавторизованных ролей
try:
    number = request.POST.get('number', '').strip()
#Извлекаем номер кабинета из POST-данных и убираем
пробелы
    building_id = request.POST.get('building')
#Получаем идентификатор корпуса из формы
    if not number: #Если номер не указан или пустой
        number = None #Присваиваем пустое значение
для корректного сохранения в БД
    if number: #Если номер был передан
        existing = Office.objects.filter(number=number,
delete_date__isnull=True).first() #Ищем существующий
активный кабинет с таким же номером
        if existing: #Если дубликат найден
            return JsonResponse({'id': existing.id, 'number':
existing.number, 'created': False}) #Возвращаем данные
существующего кабинета без создания нового
        new_office = Office(number=number) #Создаём
новый экземпляр модели кабинета
        if building_id: #Если корпус был выбран
            new_office.building_id = building_id
#Привязываем новый кабинет к выбранному корпусу
        new_office.save() #Сохраняем запись в базе
данных
        return JsonResponse({'id': new_office.id, 'number':
new_office.number or 'Не распределён', 'created': True})
#Возвращаем JSON с ID, номером и флагом успешного
создания
    except Exception as e: #Повим непредвиденные
исключения
        return JsonResponse({'error': str(e)}, status=500)
#Возвращаем текст ошибки с кодом серверной ошибки

#Функция регистрирует новый гарантийный срок для
оборудования через асинхронный POST-запрос.
#Метод использует проверку сессии, модель Warranty
ORM Django и возврат структурированного JSON-ответа.
#Нужна для быстрого добавления гарантийных
периодов без перезагрузки страницы, предотвращая ввод
пустых записей.
@require_POST
def create_warranty(request):
    if 'user_id' not in request.session: #Проверяем
авторизацию пользователя
```

```
        return JsonResponse({'error': 'Нет доступа'},
status=403) #Отклоняем запрос при отсутствии сессии
    if request.session.get('user_role') not in ['Техник',
'Руководитель', 'Администратор']: #Проверяем наличие
прав у техника или руководителя
        return JsonResponse({'error': 'Нет доступа'},
status=403) #Запрещаем выполнение операции
    try:
        start_date = request.POST.get('start_date')
#Извлекаем дату начала гарантии из формы
        end_date = request.POST.get('end_date')
#Извлекаем дату окончания гарантии
        if not start_date and not end_date: #Проверяем, что
хотя бы одна дата указана
            return JsonResponse({'id': None, 'created': False})
#Возвращаем отрицательный ответ при пустых датах
        new_warranty = Warranty() #Создаём пустой
экземпляр модели гарантии
        if start_date: #Если дата начала передана
            new_warranty.start_date = start_date
#Присваиваем значение полю начала срока
        if end_date: #Если дата окончания передана
            new_warranty.end_date = end_date
#Присваиваем значение полю окончания срока
        new_warranty.save() #Фиксируем новую запись в
базе данных
        return JsonResponse({ #Формируем успешный
JSON-ответ
            'id': new_warranty.id, #Передаём уникальный
идентификатор созданной гарантии
            'end_date': str(new_warranty.end_date) if
new_warranty.end_date else 'Без срока', #Возвращаем
дату окончания или текст-заглушку
            'created': True #Указываем флаг успешного
создания записи
        })
    except Exception as e: #Обрабатываем критические
ошибки сервера
        return JsonResponse({'error': str(e)}, status=500)
#Возвращаем описание ошибки с кодом 500
#Функция обрабатывает создание нового оборудования
и динамическое пополнение связанных справочников.
#Метод использует ORM-модели для работы с БД,
стандартные библиотеки для сохранения файлов и
систему сообщений.
#Нужна для централизованной регистрации техники и
быстрого расширения справочников без перезагрузки
страницы.
def create_equipment(request):
    if 'user_id' not in request.session: #Проверяем
идентификатора пользователя в сессии
```

Продолжение листинга В.2

```
messages.error(request, 'Нет доступа') #Выводим
сообщение об отказе в доступе
return redirect('login_view') #Перенаправляем на
страницу авторизации
if request.session.get('user_role') not in ['Техник',
'Администратор']: #Проверяем соответствие роли списку
разрешённых
messages.error(request, 'Нет доступа') #Блокируем
доступ для неавторизованных ролей
return redirect('login_view') #Возвращаем
пользователя на страницу входа
try:
    max_equipment = Equipment.objects.all().order_by('-
inventory_number').first() #Находим оборудование с
максимальным инвентарным номером
    next_inventory_number =
(max_equipment.inventory_number + 1) if max_equipment
else 10001 #Вычисляем следующий номер или начинаем
с 10001
    if request.method == 'POST': #Если данные были
отправлены формой
        action = request.POST.get('action', '') #Получаем
выбранное действие из формы
        if action == 'create_warranty': #Если выбрано
создание гарантии
            new_start_date =
request.POST.get('new_warranty_start') #Извлекаем дату
начала гарантии
            new_end_date =
request.POST.get('new_warranty_end') #Извлекаем дату
окончания гарантии
            if new_start_date or new_end_date: #Если хотя
бы одна дата указана
                new_warranty = Warranty() #Создаём пустой
экземпляр модели гарантии
                if new_start_date: #Если дата начала
передана
                    new_warranty.start_date = new_start_date
#Присваиваем значение полю
                if new_end_date: #Если дата окончания
передана
                    new_warranty.end_date = new_end_date
#Присваиваем значение полю
                new_warranty.save() #Сохраняем новую
гарантию в базе данных
                messages.success(request, 'Гарантия
создана!') #Выводим сообщение об успехе
            return
        redirect('create_equipment?show_new_warranty=1')
#Возвращаем на страницу с параметром для показа
формы
```

```
elif action == 'create_type': #Если выбрано
создание типа оборудования
    if request.session.get('user_role') not in
['Администратор']: #Проверяем права администратора
        messages.error(request, 'Нет прав')
#Выводим сообщение о недостатке прав
        return redirect('create_equipment')
#Возвращаем на страницу создания
        new_type_name =
request.POST.get('new_type_name', '').strip() #Извлекаем и
очищаем название типа
        if new_type_name: #Если название не пустое
            new_type =
EquipmentType(name=new_type_name) #Создаём
экземпляр нового типа
            new_type.save() #Сохраняем тип в базе
данных
            messages.success(request, f'Тип
"{new_type_name}" создан!') #Выводим сообщение об
успехе
        return
    redirect('create_equipment?show_new_type=1')
#Возвращаем на страницу с флагом показа формы
    elif action == 'create_model': #Если выбрано
создание модели
        if request.session.get('user_role') not in
['Администратор']: #Проверяем права администратора
            messages.error(request, 'Нет прав')
#Выводим сообщение о недостатке прав
            return redirect('create_equipment')
#Возвращаем на страницу создания
            new_model_name =
request.POST.get('new_model_name', '').strip() #Извлекаем
и очищаем название модели
            new_manufacturer_id =
request.POST.get('new_model_manufacturer') #Извлекаем
ID производителя
            if new_model_name: #Если название модели
указано
                new_model =
EquipmentModel(name=new_model_name) #Создаём
экземпляр новой модели
                if new_manufacturer_id: #Если
производитель выбран
                    new_model.manufacturer_id =
new_manufacturer_id #Привязываем модель к
производителю
                new_model.save() #Сохраняем модель в
базе данных
                messages.success(request, f'Модель
"{new_model_name}" создана!') #Выводим сообщение об
успехе
```


Продолжение листинга В.2

```
        return
    redirect('create_equipment?show_new_model=1')
    #Возвращаем на страницу с флагом показа формы
    elif action == 'create_office': #Если выбрано
    создание кабинета
        if request.session.get('user_role') not in
        ['Администратор']: #Проверяем права администратора
            messages.error(request, 'Нет прав')
        #Выводим сообщение о недостатке прав
        return redirect('create_equipment')
    #Возвращаем на страницу создания
        new_office_number =
        request.POST.get('new_office_number', '').strip()
    #Извлекаем номер кабинета
        new_building_id =
        request.POST.get('new_office_building') #Извлекаем ID
        корпуса
        if new_building_id: #Если корпус выбран
            new_office =
            Office(number=new_office_number if new_office_number
            else None) #Создаём кабинет с номером или None
            new_office.building_id = new_building_id
    #Привязываем кабинет к корпусу
            new_office.save() #Сохраняем кабинет в
            базе данных
            messages.success(request, 'Кабинет
            создан!') #Выводим сообщение об успехе
            return
    redirect('create_equipment?show_new_office=1')
    #Возвращаем на страницу с флагом показа формы
    elif action == 'create_equipment': #Если выбрано
    создание основного оборудования
        inventory_number =
        request.POST.get('inventory_number') #Получаем
        инвентарный номер
        model_id = request.POST.get('model')
    #Получаем ID модели
        type_id = request.POST.get('type') #Получаем
        ID типа
        status_id = request.POST.get('status')
    #Получаем ID статуса
        configuration = request.POST.get('configuration',
        '') #Получаем комплектацию
        purchase_date =
        request.POST.get('purchase_date') or None #Получаем
        дату покупки
        assigned_office_id =
        request.POST.get('assigned_office') or None #Получаем ID
        кабинета
        warranty_id = request.POST.get('warranty') or
        None #Получаем ID гарантии
```

```
        if
        Equipment.objects.filter(inventory_number=inventory_number,
        delete_date__isnull=True).exists(): #Проверяем
        уникальность номера
            messages.error(request, f'Оборудование с
            инвентарным номером {inventory_number} уже
            существует!') #Выводим ошибку дубликата
            return redirect('create_equipment')
    #Возвращаем на страницу формы
        photo_file = request.FILES.get('photo')
    #Получаем загруженный файл фотографии
        photo = None #Инициализируем переменную
        для объекта фото
        if photo_file: #Если файл был загружен
            ext = os.path.splitext(photo_file.name)[1]
    #Извлекаем расширение файла
            unique_id =
            datetime.now().strftime('%Y%m%d_%H%M%S_%f')
    #Генерируем уникальный ID на основе времени
            filename =
            f"equipment_{inventory_number}_{unique_id}{ext}"
    #Формируем имя файла
            file_path = os.path.join('equipment', filename)
    #Формируем относительный путь
            full_path =
            os.path.join(settings.MEDIA_ROOT, file_path) #Формируем
            абсолютный путь на сервере
            os.makedirs(os.path.dirname(full_path),
            exist_ok=True) #Создаём директорию, если её нет
            with open(full_path, 'wb+') as destination:
    #Открываем файл для бинарной записи
                for chunk in photo_file.chunks(): #Читаем
                файл частями
                    destination.write(chunk) #Записываем
                данные на диск
                photo =
                Photos.objects.create(name=filename) #Создаём запись о
                фото в базе данных
                equipment = Equipment( #Создаём экземпляр
                нового оборудования
                    inventory_number=inventory_number,
    #Присваиваем инвентарный номер
                    model_id=model_id, #Привязываем модель
                    type_id=type_id, #Привязываем тип
                    status_id=status_id, #Присваиваем статус
                    configuration=configuration or "",
    #Присваиваем комплектацию
                    purchase_date=purchase_date,
    #Присваиваем дату покупки
                    assigned_office_id=assigned_office_id,
    #Привязываем кабинет
```

Продолжение листинга В.2

```
warranty_id=warranty_id,      #Привязываем
гарантию
    photo=photo #Привязываем фотографию
)
equipment.save() #Сохраняем оборудование в
базе данных
    messages.success(request, f'Оборудование
#{inventory_number} создано!') #Выводим сообщение об
успехе
    return redirect('show_equipment')
#Перенаправляем на страницу списка оборудования
    else: #Если действие не распознано
        messages.error(request, 'Неизвестное
действие') #Выводим сообщение об ошибке
        return redirect('create_equipment')
#Возвращаем на страницу формы
    context = { #Формируем контекст для передачи
данных в шаблон
        'models':
EquipmentModel.objects.filter(delete_date__isnull=True),
#Передаём активные модели
        'types':
EquipmentType.objects.filter(delete_date__isnull=True),
#Передаём активные типы
        'statuses':
EquipmentStatus.objects.filter(delete_date__isnull=True),
#Передаём активные статусы
        'offices':
Office.objects.filter(delete_date__isnull=True), #Передаём
активные кабинеты
        'buildings':
Building.objects.filter(delete_date__isnull=True), #Передаём
активные корпуса
        'warranties':
Warranty.objects.filter(delete_date__isnull=True),
#Передаём активные гарантии
        'manufacturers':
Manufacturer.objects.filter(delete_date__isnull=True),
#Передаём активных производителей
        'next_inventory_number': next_inventory_number,
#Передаём рассчитанный следующий номер
        'user_name': request.session.get('user_name', ''),
#Передаём имя пользователя
        'user_role': request.session.get('user_role', ''),
#Передаём роль пользователя
        'show_new_type':
request.GET.get('show_new_type') == '1', #Передаём флаг
показа формы типа
        'show_new_model':
request.GET.get('show_new_model') == '1', #Передаём
флаг показа формы модели
```

```
        'show_new_office':
request.GET.get('show_new_office') == '1', #Передаём флаг
показа формы кабинета
        'show_new_warranty':
request.GET.get('show_new_warranty') == '1', #Передаём
флаг показа формы гарантии
    }
    return render(request, 'create_equipment.html',
context) #Рендерим страницу создания оборудования
    except Exception as e: #Ловим критические ошибки
        messages.error(request, f'Ошибка: {str(e)}')
#Выводим сообщение об ошибке
    return redirect('show_equipment') #Перенаправляем
на список оборудования
    #Функция обновляет характеристики существующего
оборудования и заменяет его фотографию при
необходимости.
    #Метод использует ORM для поиска записи, файловую
систему для загрузки изображений и валидацию сессий.
    #Нужна для поддержания актуальности данных парка
техники и своевременного обновления визуальных
материалов.
    def edit_equipment(request, inventory_number):
        if 'user_id' not in request.session: #Проверяем наличие
идентификатора пользователя в сессии
            messages.error(request, 'Нет доступа') #Выводим
сообщение об отказе в доступе
            return redirect('login_view') #Перенаправляем на
страницу авторизации
            if request.session.get('user_role') not in ['Техник',
'Администратор']: #Проверяем соответствие роли списку
разрешённых
                messages.error(request, 'Нет доступа') #Блокируем
доступ для неавторизованных ролей
                return redirect('login_view') #Возвращаем
пользователя на страницу входа
            try:
                equipment = get_object_or_404(Equipment,
inventory_number=inventory_number,
delete_date__isnull=True) #Находим активное
оборудование по номеру или возвращаем 404
                if request.method == 'POST': #Если форма была
отправлена
                    try:
                        equipment.model_id =
request.POST.get('model') #Обновляем привязку модели
                        equipment.type_id = request.POST.get('type')
#Обновляем привязку типа
                        equipment.status_id =
request.POST.get('status') #Обновляем привязку статуса
```

Продолжение листинга В.2

```
equipment.configuration =
request.POST.get('configuration', '') #Обновляем
комплектацию

equipment.assigned_office_id =
request.POST.get('assigned_office') #Обновляем привязку
кабинета

photo_file = request.FILES.get('photo')
#Получаем новый загруженный файл фотографии
if photo_file: #Если файл был загружен
    ext = os.path.splitext(photo_file.name)[1]
#Извлекаем расширение файла
    unique_id =
datetime.now().strftime('%Y%m%d_%H%M%S%f')
#Генерируем уникальный ID на основе времени
    filename =
f"equipment_{equipment.inventory_number}_{unique_id}{ext}"
#Формируем новое имя файла
    file_path = os.path.join('equipment', filename)
#Формируем относительный путь
    full_path =
os.path.join(settings.MEDIA_ROOT, file_path) #Формируем
абсолютный путь
    os.makedirs(os.path.dirname(full_path),
exist_ok=True) #Создаём директорию при необходимости
    with open(full_path, 'wb+') as destination:
#Открываем файл для записи
        for chunk in photo_file.chunks(): #Читаем
данные частями
            destination.write(chunk) #Записываем
на диск

photo =
Photos.objects.create(name=filename) #Создаём новую
запись фотографии в БД
    if equipment.photo and equipment.photo.id !=
photo.id: #Если старое фото существует и отличается от
нового
        equipment.photo.delete_date =
timezone.now() #Устанавливаем дату удаления для
старой записи
        equipment.photo.save() #Сохраняем
архивацию старого фото
    equipment.photo = photo #Привязываем
новую фотографию к оборудованию
    equipment.save() #Сохраняем обновлённые
данные оборудования
    messages.success(request, 'Оборудование
обновлено!') #Выводим сообщение об успехе
    return redirect('show_equipment')
#Перенаправляем на список оборудования
except Exception as e: #Ловим ошибки
сохранения
```

```
messages.error(request, f'Ошибка при
обновлении: {str(e)}') #Выводим сообщение об ошибке
context = { #Формируем контекст для шаблона
    'equipment': equipment, #Передаём объект
редактируемого оборудования
    'models':
EquipmentModel.objects.filter(delete_date__isnull=True),
#Передаём активные модели
    'types':
EquipmentType.objects.filter(delete_date__isnull=True),
#Передаём активные типы
    'statuses':
EquipmentStatus.objects.filter(delete_date__isnull=True),
#Передаём активные статусы
    'offices':
Office.objects.filter(delete_date__isnull=True), #Передаём
активные кабинеты
    'user_name': request.session.get('user_name', ''),
#Передаём имя пользователя
}
return render(request, 'edit_equipment.html', context)
#Рендерим страницу редактирования
except Exception as e: #Ловим ошибки поиска
оборудования
    messages.error(request, f'Ошибка: {str(e)}')
#Выводим сообщение об ошибке
    return redirect('show_equipment') #Перенаправляем
на список оборудования
    #Функция выполняет мягкое архивирование или
безвозвратное удаление записи оборудования в
зависимости от роли.
    #Метод использует ORM для изменения статуса записи,
проверку прав доступа и модуль timezone.
    #Нужна для безопасного управления жизненным
циклом техники с возможностью восстановления или
полной очистки.
    def delete_equipment(request, inventory_number):
        if 'user_id' not in request.session: #Проверяем наличие
идентификатора пользователя в сессии
            messages.error(request, 'Нет доступа') #Выводим
сообщение об отказе в доступе
            return redirect('login_view') #Перенаправляем на
страницу авторизации
            user_role = request.session.get('user_role', '')
#Извлекаем роль текущего пользователя
            if user_role not in ['Техник', 'Администратор']:
#Проверяем соответствие роли списку разрешённых
            messages.error(request, 'Нет доступа') #Блокируем
доступ для неавторизованных ролей
            return redirect('login_view') #Возвращаем
пользователя на страницу входа
```

Продолжение листинга В.2

```
if request.method == 'POST': #Разрешаем удаление
    только через POST-запрос
    try:
        equipment =
Equipment.objects.get(inventory_number=inventory_number
, delete_date__isnull=True) #Находим активное
оборудование по номеру
        if user_role == 'Администратор': #Если
запрашивает администратор
            permanent = request.POST.get('permanent',
'false') #Получаем флаг безвозвратного удаления
            if permanent == 'true': #Если выбрано полное
удаление
                equipment.delete() #Физически стираем
запись из базы данных
                messages.success(request, f'Оборудование
#{inventory_number} удалено навсегда!') #Выводим
сообщение об успехе
                return redirect('show_deleted_equipment')
#Перенаправляем в архив
            equipment.delete_date = timezone.now()
#Устанавливаем текущую дату для мягкого удаления
            equipment.save() #Сохраняем изменения в базе
данных
            messages.success(request, f'Оборудование
#{inventory_number} удалено (архивировано)!') #Выводим
сообщение об архивации
            except Exception as e: #Ловим ошибки выполнения
                messages.error(request, f'Ошибка: {str(e)}')
#Выводим сообщение об ошибке
            return redirect('show_equipment') #Возвращаем на
страницу списка оборудования
            return redirect('show_equipment') #Возвращаем на
страницу списка при GET-запросе
        #Функция обрабатывает мягкое архивирование или
безвозвратное удаление записи оборудования в
зависимости от роли пользователя.
        #Метод использует проверку сессии, ORM-запросы к
модели Equipment, модуль timezone и систему сообщений
Django.
        #Нужна для безопасного управления жизненным
циклом техники, позволяя скрывать записи из общего
доступа или очищать базу данных.
        def delete_equipment(request, inventory_number):
            if 'user_id' not in request.session: #Проверяем наличие
идентификатора пользователя в сессии
                messages.error(request, 'Нет доступа') #Выводим
сообщение об отказе в доступе
                return redirect('login_view') #Перенаправляем на
страницу входа
```

```
        user_role = request.session.get('user_role', '')
#Извлекаем роль текущего пользователя из данных
сессии
        if user_role not in ['Техник', 'Администратор']:
#Проверяем соответствие роли списку разрешённых
            messages.error(request, 'Нет доступа') #Блокируем
доступ для неавторизованных ролей
            return redirect('login_view') #Возвращаем
пользователя на страницу входа
        if request.method == 'POST': #Разрешаем удаление
только через безопасный POST-запрос
            try:
                equipment =
Equipment.objects.get(inventory_number=inventory_number
, delete_date__isnull=True) #Находим активное
оборудование по инвентарному номеру
                if user_role == 'Администратор': #Проверяем,
имеет ли пользователь права администратора
                    permanent = request.POST.get('permanent',
'false') #Получаем флаг полного удаления из формы
                    if permanent == 'true': #Если администратор
выбрал безвозвратное удаление
                        equipment.delete() #Физически стираем
запись из базы данных
                        messages.success(request, f'Оборудование
#{inventory_number} удалено навсегда!') #Выводим
сообщение об успехе
                        return redirect('show_deleted_equipment')
#Перенаправляем в раздел архивного оборудования
                    equipment.delete_date = timezone.now()
#Устанавливаем текущую дату и время в поле мягкого
удаления
                    equipment.save() #Применяем изменения к
записи в базе данных
                    messages.success(request, f'Оборудование
#{inventory_number} удалено (архивировано)!')
#Информируем об успешной архивации
                    except Exception as e: #Ловим непредвиденные
ошибки выполнения запроса
                        messages.error(request, f'Ошибка: {str(e)}')
#Выводим сообщение с текстом возникшего исключения
                    return redirect('show_equipment') #Возвращаем
пользователя на страницу списка оборудования
                    return redirect('show_equipment') #Перенаправляем
на страницу списка при получении GET-запроса
                #Функция формирует и отображает список
архивированных единиц оборудования для
пользователей с расширенными правами.
                #Метод использует ORM-фильтрацию по дате
удаления, оптимизацию запросов через select_related и
рендеринг HTML-шаблона.
```

Продолжение листинга В.2

```
#Нужна для предоставления техники и
администраторам возможности просматривать скрытые
записи для восстановления или полной очистки.
def show_deleted_equipment(request):
    if 'user_id' not in request.session: #Проверяем наличие
идентификатора пользователя в сессии
        messages.error(request, 'Нет доступа') #Выводим
сообщение об отказе в доступе
        return redirect('login_view') #Перенаправляем на
страницу входа
    user_role = request.session.get('user_role', '')
#Извлекаем роль текущего пользователя из сессии
    if user_role not in ['Техник', 'Администратор']:
#Проверяем соответствие роли списку разрешённых
        messages.error(request, 'Нет доступа') #Блокируем
доступ для неавторизованных ролей
        return redirect('login_view') #Возвращаем
пользователя на страницу входа
    try:
        equipment_list =
Equipment.objects.filter(delete_date__isnull=False).select_r
elated( #Получаем архивированное оборудование и
подгружаем связанные таблицы одним запросом
            'model', 'model__manufacturer', 'type', 'status',
#Подгружаем данные о модели, производителе, типе и
статусе
            'assigned_office', 'assigned_office__building',
'warranty', 'photo' #Добавляем кабинет, корпус, гарантию и
фотографию
        ).order_by('-delete_date') #Сортируем список по
дате удаления от новых к старым
        context = { #Формируем словарь данных для
передачи в HTML-шаблон
            'equipment_list': equipment_list, #Передаём
отфильтрованный список архивного оборудования
            'user_name': request.session.get('user_name', ''),
#Передаём имя текущего пользователя для шапки сайта
            'user_role': user_role, #Передаём роль
пользователя для управления видимостью кнопок
            'page_title': 'Удалённое оборудование',
#Указываем заголовок текущей страницы
        }
        return render(request,
'show_deleted_equipment.html', context) #Рендерим
страницу архива с подготовленными данными
    except Exception as e: #Ловим ошибки при
выполнении запроса к базе данных
        messages.error(request, f'Ошибка: {str(e)}')
#Выводим сообщение с текстом возникшего исключения
        return redirect('show_equipment') #Возвращаем
пользователя на страницу списка оборудования
```

```
#Функция восстанавливает архивированную запись
оборудования, возвращая её в активный реестр системы.
#Метод использует проверку авторизации, ORM-поиск
записи по инвентарному номеру и очистку поля даты
удаления.
#Нужна для отмены ошибочной архивации техники и
быстрого возврата данных в рабочий оборот без потери
истории.
def restore_equipment(request, inventory_number):
    if 'user_id' not in request.session: #Проверяем наличие
идентификатора пользователя в сессии
        messages.error(request, 'Нет доступа') #Выводим
сообщение об отказе в доступе
        return redirect('login_view') #Перенаправляем на
страницу входа
    user_role = request.session.get('user_role', '')
#Извлекаем роль текущего пользователя из данных
сессии
    if user_role not in ['Техник', 'Администратор']:
#Проверяем соответствие роли списку разрешённых
        messages.error(request, 'Нет доступа') #Блокируем
доступ для неавторизованных ролей
        return redirect('login_view') #Возвращаем
пользователя на страницу входа
    if request.method == 'POST': #Разрешаем
восстановление только через POST-запрос
        try:
            equipment =
Equipment.objects.get(inventory_number=inventory_number
, delete_date__isnull=False) #Находим архивную запись по
инвентарному номеру
            equipment.delete_date = None #Очищаем поле
даты удаления, возвращая запись активный статус
            equipment.save() #Применяем изменения к базе
данных
            messages.success(request, f'Оборудование
#{inventory_number} восстановлено!') #Информируем об
успешном восстановлении
        except Exception as e: #Ловим ошибки при поиске
или сохранении записи
            messages.error(request, f'Ошибка: {str(e)}')
#Выводим сообщение с текстом возникшего исключения
            return redirect('show_deleted_equipment')
#Возвращаем пользователя на страницу архивного
оборудования
#Функция выполняет полное физическое удаление
архивной записи оборудования из базы данных.
#Метод использует строгую проверку роли
администратора, ORM-запрос к архивным записям и
метод delete().
```

Продолжение листинга В.2

#Нужна для окончательной очистки реестра от списанной или неактуальной техники с целью освобождения хранилища.

```
def delete_equipment_permanent(request, inventory_number):
    if 'user_id' not in request.session: #Проверяем наличие
        идентификатора пользователя в сессии
        messages.error(request, 'Нет доступа') #Выводим
        сообщение об отказе в доступе
        return redirect('login_view') #Перенаправляем на
        страницу входа
    user_role = request.session.get('user_role', '')
    #Извлекаем роль текущего пользователя из сессии
    if user_role not in ['Администратор']: #Проверяем,
        является ли пользователь администратором
        messages.error(request, 'Нет доступа') #Блокируем
        доступ для всех остальных ролей
        return redirect('login_view') #Возвращаем
        пользователя на страницу входа
    if request.method == 'POST': #Разрешаем удаление
        только через POST-запрос
        try:
            equipment = Equipment.objects.get(inventory_number=inventory_number
            , delete_date__isnull=False) #Находим архивную запись
            оборудования по номеру
            equipment.delete() #Полностью стираем запись
            из базы данных
            messages.success(request, f'Оборудование
            #{inventory_number} удалено навсегда!') #Информируем
            об успешном выполнении операции
            except Exception as e: #Полним непредвиденные
            ошибки выполнения запроса
            messages.error(request, f'Ошибка: {str(e)}')
            #Выводим сообщение с текстом возникшего исключения
            return redirect('show_deleted_equipment')
            #Возвращаем пользователя на страницу архивного
            оборудования
            #Функция рассчитывает финансовые затраты на ремонт
            оборудования, позволяя фильтровать данные по
            конкретной технике или периоду.
            #Метод использует ORM для выборки заявок с
            предварительной загрузкой связанных услуг и запчастей,
            а также математические операции для подсчета сумм.
            #Нужна для анализа бюджета отдела ИТ, контроля
            расходов на запчасти и оценки стоимости обслуживания
            конкретных единиц техники.
            def equipment_costs(request):
                if 'user_id' not in request.session: #Проверяем наличие
                идентификатора пользователя в сессии
```

```
                messages.error(request, 'Нет доступа') #Выводим
                сообщение об отказе в доступе
                return redirect('login_view') #Перенаправляем на
                страницу входа
                if request.session.get('user_role') not in ['Техник',
                'Руководитель', 'Администратор']: #Проверяем
                соответствие роли списку разрешённых
                messages.error(request, 'Нет доступа') #Блокируем
                доступ для неавторизованных ролей
                return redirect('login_view') #Возвращаем
                пользователя на страницу входа
                try:
                    equipment_id = request.GET.get('equipment_id')
                    #Извлекаем ID оборудования из параметров URL
                    equipment = None #Инициализируем переменную
                    для объекта оборудования
                    if equipment_id: #Если ID оборудования передан в
                    запросе
                        equipment = get_object_or_404(Equipment,
                        inventory_number=int(equipment_id),
                        delete_date__isnull=True) #Находим оборудование по
                        номеру или возвращаем ошибку 404
                        requests = RequestFix.objects.filter(equipment=equipment,
                        delete_date__isnull=True).prefetch_related(
                        #Получаем
                        заявки только для этого оборудования и предзагружаем
                        связанные таблицы
                        'used_spare_parts', 'used_services'
                        )
                        else: #Если ID не передан
                        requests = RequestFix.objects.filter(delete_date__isnull=True).prefetch
                        _related(
                        #Получаем все активные заявки и
                        предзагружаем данные о запчастях и услугах
                        'used_spare_parts', 'used_services'
                        )
                        date_from = request.GET.get('date_from', '')
                        #Извлекаем начальную дату фильтрации из параметров
                        URL
                        date_to = request.GET.get('date_to', '') #Извлекаем
                        конечную дату фильтрации из параметров URL
                        if date_from: #Если начальная дата указана
                        requests = requests.filter(registration_date__gte=date_from)
                        #Фильтруем заявки, зарегистрированные после указанной
                        даты
                        if date_to: #Если конечная дата указана
                        requests = requests.filter(registration_date__lte=date_to) #Фильтруем
                        заявки, зарегистрированные до указанной даты
                        total_cost = 0 #Инициализируем переменную для
                        накопления общей суммы затрат
```

Продолжение листинга В.2

```
for req in requests: #Проходимся по каждой заявке
    в выборке
        spare_parts_cost = sum(item.total_cost for item in
req.used_spare_parts.filter(delete_date__isnull=True))
#Суммируем стоимость запчастей по каждой заявке
        services_cost = sum(item.total_cost for item in
req.used_services.filter(delete_date__isnull=True))
#Суммируем стоимость услуг по каждой заявке
        req.total_cost = spare_parts_cost + services_cost
#Записываем итоговую стоимость ремонта в объект
заявки
        req.spare_parts_cost = spare_parts_cost
#Сохраняем отдельную сумму затрат на запчасти
        req.services_cost = services_cost #Сохраняем
отдельную сумму затрат на услуги
        total_cost += req.total_cost #Прибавляем
стоимость текущей заявки к общей сумме
        avg_cost = total_cost / requests.count() if
requests.count() > 0 else 0 #Вычисляем среднюю
стоимость одной заявки или ноль при отсутствии заявок
        context = { #Формируем словарь данных для
передачи в HTML-шаблон
            'equipment': equipment, #Передаём объект
оборудования (или None)
            'requests': requests, #Передаём
отфильтрованный список заявок с рассчитанными
затратами
            'total_cost': total_cost, #Передаём общую сумму
затрат за период
            'avg_cost': round(avg_cost, 2), #Передаём
округлённую среднюю стоимость заявки
            'date_from': date_from, #Передаём начальную
дату для сохранения в форме фильтра
            'date_to': date_to, #Передаём конечную дату для
сохранения в форме фильтра
            'user_name': request.session.get('user_name', ''),
#Передаём имя пользователя для отображения в шапке
        }
        return render(request, 'equipment_costs.html',
context) #Рендерим страницу затрат с подготовленным
контекстом
    except Exception as e: #Ловим любые
непредвиденные ошибки выполнения
        messages.error(request, f'Ошибка: {str(e)}')
#Выводим сообщение с текстом ошибки
        return redirect('show_equipment') #Перенаправляем
пользователя на страницу списка оборудования
    #Функция формирует аналитическую панель
руководителя со статистикой заявок, графиками и
расчётом финансовых затрат.
```

```
#Метод использует библиотеку Matplotlib для генерации
изображений графиков, ORM для агрегации данных и
стандартные средства работы с файловой системой.
#Нужна для принятия управленческих решений на
основе объективных данных о нагрузке, частоте поломок
и расходах бюджета.
def manager_dashboard(request):
    if 'user_id' not in request.session: #Проверяем наличие
идентификатора пользователя в сессии
        messages.error(request, 'Нет доступа') #Выводим
сообщение об отказе в доступе
        return redirect('login_view') #Перенаправляем на
страницу входа
    if request.session.get('user_role') not in
['Руководитель', 'Администратор']: #Проверяем
соответствие роли списку разрешённых
        messages.error(request, 'Нет доступа') #Блокируем
доступ для пользователей без нужных прав
        return redirect('login_view') #Возвращаем
пользователя на страницу входа
    try:
        total_requests =
RequestFix.objects.filter(delete_date__isnull=True).count()
#Подсчитываем общее количество активных заявок
        completed_requests = RequestFix.objects.filter(
#Подсчитываем количество заявок, у которых есть дата
выполнения
            delete_date__isnull=True,
            completion_date__isnull=False
        ).count()
        pending_requests = total_requests -
completed_requests #Вычисляем количество заявок,
находящихся в работе
        matplotlib.use('Agg') #Устанавливаем
неблокирующий бэкэнд Matplotlib для работы без
графического интерфейса
        charts_folder = os.path.join(settings.MEDIA_ROOT,
'charts') #Формируем путь к директории для сохранения
графиков
        os.makedirs(charts_folder, exist_ok=True) #Создаём
директорию, если она ещё не существует
        labels = ['Всего заявок', 'Выполнено', 'В работе']
#Задаём подписи для оси X столбчатой диаграммы
        values = [total_requests, completed_requests,
pending_requests] #Задаём числовые значения для
столбцов
        colors = ['#EB5A61', '#ab47bc', '#ff6f00'] #Задаём
цвета для каждого столбца диаграммы
        fig, ax = plt.subplots(figsize=(10, 6)) #Создаём
фигуру и оси графика с указанным размером
```

Продолжение листинга В.2

```
bars = ax.bar(labels, values, color=colors,
edgecolor='black', linewidth=1.5) #Рисуем столбчатую
диаграмму с цветовой заливкой и обводкой

for bar in bars: #Проходимся по каждому столбцу
для добавления числовых меток
    height = bar.get_height() #Получаем высоту
текущего столбца
    ax.annotate(f'{height}', #Аннотируем столбец
текстом с числовым значением
        xy=(bar.get_x() + bar.get_width() / 2,
height), #Указываем координаты центра верхней части
столбца
        xytext=(0, 3), #Смещаем текст на 3 пункта
вверх от точки привязки
        textcoords="offset points", #Указываем,
что смещение задано в пунктах
        ha='center', va='bottom', #Выравниваем
текст по центру и по низу
        fontsize=12, fontweight='bold')
#Устанавливаем размер и жирность шрифта
ax.set_title('Статистика заявок', fontsize=14,
fontweight='bold', pad=20) #Устанавливаем заголовок
графика с отступом
ax.set_ylabel('Количество', fontsize=12)
#Устанавливаем подпись для оси Y
ax.set_ylim(0, max(values) * 1.2 if max(values) > 0
else 10) #Устанавливаем предел оси Y с запасом 20% от
максимума
ax.grid(axis='y', alpha=0.3, linestyle='--') #Включаем
вертикальную сетку с прозрачностью и пунктирным
стилем

chart_filename =
f'manager_stats_{timezone.now().strftime("%Y%m%d_%H%M%S")}.png' #Генерируем уникальное имя файла с
текущим временем

chart_path = os.path.join(charts_folder,
chart_filename) #Формируем полный путь к файлу графика
plt.tight_layout() #Подстраиваем параметры макета
для предотвращения обрезки элементов
plt.savefig(chart_path, dpi=150, bbox_inches='tight')
#Сохраняем изображение графика в файл с высоким
разрешением
plt.close() #Закрываем фигуру для освобождения
памяти

equipment_stats = RequestFix.objects.filter(
#Получаем статистику поломок по оборудованию
    delete_date__isnull=True
).values(
    'equipment__inventory_number', #Группируем по
инвентарному номеру
```

```
'equipment__model__name', #Добавляем
название модели
    'equipment__type__name' #Добавляем название
типа
).annotate(
    breakdown_count=Count('act_number')
#Подсчитываем количество заявок для каждой единицы
).order_by('-breakdown_count')[:5] #Сортируем по
убыванию количества и берём топ-5
six_months_ago = timezone.now() -
timedelta(days=180) #Вычисляем дату шесть месяцев
назад
monthly_stats = RequestFix.objects.filter( #Получаем
статистику заявок по месяцам
    delete_date__isnull=True,
    registration_date__gte=six_months_ago
).extra(
    select={'month': "TO_CHAR(registration_date,
'YYYY-MM)"} #Извлекаем год и месяц из даты
регистрации
).values('month').annotate(
    count=Count('act_number') #Подсчитываем
количество заявок в каждом месяце
).order_by('month') #Сортируем результаты по
возрастанию даты
all_requests =
RequestFix.objects.filter(delete_date__isnull=True).prefetch
_related( #Получаем все активные заявки с предзагрузкой
связанных данных
    'used_spare_parts', 'used_services',
    'equipment__type'
)
total_cost = 0 #Инициализируем переменную для
общей суммы затрат
monthly_costs = {} #Создаём словарь для
накопления затрат по месяцам
type_costs = {} #Создаём словарь для накопления
затрат по типам оборудования
for req in all_requests: #Проходимся по каждой
заявке для расчёта финансов
    spare_parts_cost = sum( #Суммируем стоимость
запчастей по заявке
        float(item.total_cost or 0) for item in
req.used_spare_parts.filter(delete_date__isnull=True))
    services_cost = sum( #Суммируем стоимость
услуг по заявке
        float(item.total_cost or 0) for item in
req.used_services.filter(delete_date__isnull=True))
    req_cost = spare_parts_cost + services_cost
#Считаем итоговую стоимость ремонта
    total_cost += req_cost #Прибавляем к общей
сумме затрат
```


Продолжение листинга В.2

```
month = req.registration_date.strftime("%Y-%m")
#Формируем строковое представление месяца заявки
monthly_costs[month] = monthly_costs.get(month,
0) + req_cost #Добавляем стоимость в словарь затрат по
месяцам
equip_type = req.equipment.type.name if
req.equipment.type else 'Без типа' #Определяем тип
оборудования или заглушку
type_costs[equip_type] =
type_costs.get(equip_type, 0) + req_cost #Добавляем
стоимость в словарь затрат по типам
monthly_labels = list(monthly_costs.keys())[-6:]
#Берём последние 6 месяцев для отображения на
графике
monthly_data = [round(monthly_costs[m], 2) for m in
monthly_labels] #Формируем список числовых данных для
графика затрат
type_costs_list = [(type_name, round(cost, 2)) for
type_name, cost in type_costs.items()] #Преобразуем
словарь затрат по типам в список кортежей
context = { #Формируем словарь данных для
передачи в HTML-шаблон
'total_requests': total_requests, #Передаём общее
количество заявок
'completed_requests': completed_requests,
#Передаём количество выполненных заявок
'pending_requests': pending_requests,
#Передаём количество заявок в работе
'equipment_stats': list(equipment_stats),
#Передаём список топ-5 ломающегося оборудования
'monthly_stats': list(monthly_stats), #Передаём
статистику по месяцам
'total_cost': round(total_cost, 2), #Передаём
общую сумму затрат
'monthly_labels': monthly_labels, #Передаём
метки месяцев для графика
'monthly_data': monthly_data, #Передаём данные
затрат по месяцам
'type_costs_list': type_costs_list, #Передаём
список затрат по типам оборудования
'user_name': request.session.get('user_name', ''),
#Передаём имя пользователя
'chart_filename': chart_filename, #Передаём имя
файла сгенерированного графика
}
return render(request, 'manager_dashboard.html',
context) #Рендерим дашборд руководителя с
подготовленными данными
except Exception as e: #Ловим любые
непредвиденные ошибки выполнения
```

```
messages.error(request, f'Ошибка: {str(e)}')
#Выводим сообщение с текстом ошибки
return redirect('login_view') #Перенаправляем
пользователя на страницу входа
#Функция детализирует затраты по оборудованию на
панели руководителя, позволяя анализировать расходы в
разрезе техники и периодов.
#Метод использует ORM для выборки заявок, агрегации
данных о запчастях и услугах, а также математические
операции для расчёта средних значений.
#Нужна для углубленного финансового анализа,
выявления самых затратных единиц техники и
планирования бюджета на техническое обслуживание.
def manager_equipment_costs(request):
if 'user_id' not in request.session: #Проверяем наличие
идентификатора пользователя в сессии
messages.error(request, 'Нет доступа') #Выводим
сообщение об отказе в доступе
return redirect('login_view') #Перенаправляем на
страницу входа
if request.session.get('user_role') not in
['Руководитель', 'Администратор']: #Проверяем
соответствие роли списку разрешённых
messages.error(request, 'Нет доступа') #Блокируем
доступ для пользователей без нужных прав
return redirect('login_view') #Возвращаем
пользователя на страницу входа
try:
equipment_id = request.GET.get('equipment_id')
#Извлекаем ID оборудования из параметров URL
equipment = None #Инициализируем переменную
для объекта оборудования
if equipment_id: #Если ID передан в запросе
equipment = get_object_or_404(Equipment,
inventory_number=int(equipment_id),
delete_date__isnull=True) #Находим оборудование по
инвентарному номеру
requests =
RequestFix.objects.filter(equipment=equipment,
delete_date__isnull=True).prefetch_related( #Получаем
заявки только для этого оборудования с предзагрузкой
связанных данных
'used_spare_parts', 'used_services'
)
else: #Если ID не передан
requests =
RequestFix.objects.filter(delete_date__isnull=True).prefetch
_related( #Получаем все активные заявки с предзагрузкой
данных
'used_spare_parts', 'used_services'
)
```

Продолжение листинга В.2

```
date_from = request.GET.get('date_from', "")
#Извлекаем начальную дату фильтрации
date_to = request.GET.get('date_to', "") #Извлекаем
конечную дату фильтрации
if date_from: #Если начальная дата указана
    requests =
requests.filter(registration_date__gte=date_from)
#Фильтруем заявки по начальной дате
if date_to: #Если конечная дата указана
    requests =
requests.filter(registration_date__lte=date_to) #Фильтруем
заявки по конечной дате
total_cost = 0 #Инициализируем переменную для
общей суммы затрат
for req in requests: #Проходимся по каждой заявке
для расчёта стоимости
    spare_parts_cost = sum( #Суммируем стоимость
запчастей по заявке
        item.total_cost or 0 for item in
req.used_spare_parts.filter(delete_date__isnull=True))
    services_cost = sum( #Суммируем стоимость
услуг по заявке
        item.total_cost or 0 for item in
req.used_services.filter(delete_date__isnull=True))
    req.total_cost = spare_parts_cost + services_cost
#Записываем итоговую стоимость в объект заявки
    total_cost += req.total_cost #Прибавляем к общей
сумме затрат
avg_cost = total_cost / requests.count() if
requests.count() > 0 else 0 #Вычисляем среднюю
стоимость ремонта или ноль
context = { #Формируем словарь данных для
передачи в HTML-шаблон
    'equipment': equipment, #Передаём объект
оборудования (или None)
    'requests': requests, #Передаём
отфильтрованный список заявок с рассчитанными
суммами
    'total_cost': total_cost, #Передаём общую сумму
затрат за период
    'avg_cost': round(avg_cost, 2), #Передаём
округлённую среднюю стоимость заявки
    'date_from': date_from, #Передаём начальную
дату для сохранения в форме
    'date_to': date_to, #Передаём конечную дату для
сохранения в форме
    'user_name': request.session.get('user_name', ""),
#Передаём имя пользователя для отображения
}
```

```
return render(request,
'manager_equipment_costs.html', context) #Рендерим
страницу затрат руководителя
except Exception as e: #Повим любые
непредвиденные ошибки выполнения
    messages.error(request, f'Ошибка: {str(e)}')
#Выводим сообщение с текстом ошибки
return redirect('manager_dashboard')
#Перенаправляем пользователя на главную панель
руководителя
#Функция обрабатывает самостоятельную регистрацию
новых пользователей в системе без участия
администратора.
#Метод использует модель Employee для создания
записи, ORM для проверки дубликатов логина и модуль
сессий для автоматического входа.
#Нужна для быстрого предоставления доступа
сотрудникам с автоматическим назначением базовой
роли «Сотрудник».
def registration_view(request): #Объявляем функцию
обработки запроса регистрации
    if request.method == 'POST': #Проверяем, что форма
была отправлена методом POST
        first_name = request.POST.get('first_name', "").strip()
#Извлекаем и очищаем имя из данных формы
        last_name = request.POST.get('last_name', "").strip()
#Извлекаем и очищаем фамилию из данных формы
        middle_name = request.POST.get('middle_name',
        "").strip() #Извлекаем и очищаем отчество из данных
формы
        login = request.POST.get('login', "").strip()
#Извлекаем и очищаем логин (почту) из формы
        phone_number = request.POST.get('phone_number',
        "").strip() #Извлекаем и очищаем номер телефона
        password = request.POST.get('password', "")
#Извлекаем пароль без обрезки пробелов для точности
        password_confirm =
request.POST.get('password_confirm', "") #Извлекаем
подтверждение пароля для сверки
        position_id = request.POST.get('position')
#Получаем ID выбранной должности
        department_id = request.POST.get('department')
#Получаем ID выбранного отдела
        office_id = request.POST.get('office') #Получаем ID
выбранного кабинета
        if not first_name or not last_name or not login or not
password: #Проверяем заполненность обязательных
полей
            messages.error(request, 'Заполните все
обязательные поля (Имя, Фамилия, Логин, Пароль)')
#Выводим сообщение об ошибке валидации
```

Продолжение листинга В.2

```
        return redirect('registration_view') #Возвращаем
пользователя на страницу регистрации
        if password != password_confirm: #Сравниваем
введённый пароль с подтверждением
            messages.error(request, 'Пароли не совпадают')
#Выводим сообщение о несовпадении паролей
        return redirect('registration_view') #Возвращаем
пользователя на страницу регистрации
        if Employee.objects.filter(login=login,
delete_date__isnull=True).exists(): #Проверяем
уникальность логина среди активных пользователей
            messages.error(request, 'Пользователь с таким
логинем уже существует') #Выводим сообщение о
занятом логине
        return redirect('registration_view') #Возвращаем
пользователя на страницу регистрации
        try: #Начинаем блок обработки возможных ошибок
создания пользователя
            employee_role, created =
Role.objects.get_or_create( #Ищем существующую роль
«Сотрудник» или создаём её при отсутствии
                role_name='Сотрудник', #Указываем
системное название роли для поиска
                defaults={'role_name': 'Сотрудник'} #Задаём
параметры создания новой записи роли
            )
            new_user = Employee( #Инициализируем новый
экземпляр модели сотрудника с полученными данными
                first_name=first_name, #Присваиваем имя
пользователя
                last_name=last_name, #Присваиваем
фамилию пользователя
                middle_name=middle_name if middle_name
else "", #Присваиваем отчество или пустую строку
                login=login, #Присваиваем уникальный логин
                phone_number=phone_number if
phone_number else "", #Присваиваем телефон или пустую
строку
                password=password, #Присваиваем пароль
                role=employee_role, #Привязываем роль
«Сотрудник»
                position_id=position_id if position_id else None,
#Привязываем должность или None
                department_id=department_id if department_id
else None, #Привязываем отдел или None
                office_id=office_id if office_id else None
#Привязываем кабинет или None
            )
            new_user.save() #Сохраняем нового сотрудника
в базе данных
```

```
        messages.success(request, f'Регистрация
успешна! Добро пожаловать, {first_name} {last_name}!')
#Выводим сообщение об успехе
        request.session['user_id'] = new_user.id
#Сохраняем ID пользователя в сессии для
автоматической авторизации
        request.session['user_login'] = new_user.login
#Сохраняем логин в сессии
        request.session['user_role'] = 'Сотрудник'
#Фиксируем роль в сессии
        request.session['user_name'] = f'{last_name}
{first_name} {middle_name}'.strip() #Формируем и
сохраняем ФИО в сессии
        request.session['form_open'] = False
#Сбрасываем флаг открытой формы
        return redirect('show_customer')
#Перенаправляем в личный кабинет сотрудника
        except Exception as e: #Повим непредвиденные
ошибки при сохранении
            messages.error(request, f'Ошибка при
регистрации: {str(e)}') #Выводим текст возникшей ошибки
            return redirect('registration_view') #Возвращаем
пользователя на страницу регистрации
            context = { #Формируем словарь данных для
передачи в HTML-шаблон
                'positions':
Position.objects.filter(delete_date__isnull=True), #Передаём
список активных должностей
                'departments':
Department.objects.filter(delete_date__isnull=True),
#Передаём список активных отделов
                'offices': Office.objects.filter( #Получаем активные
кабинеты
                    delete_date__isnull=True #Исключаем
удалённые записи
                ).select_related('building').order_by('building__name',
'number'), #Подгружаем корпуса и сортируем по названию
и номеру
            }
            return render(request, 'registration.html', context)
#Рендерим страницу регистрации с подготовленным
контекстом
        #Функция позволяет пользователю просматривать и
редактировать свои личные данные, а администратору —
управлять параметрами любых аккаунтов.
        #Метод использует модель Employee, систему сессий
для идентификации и ORM для безопасного обновления
записей с проверкой уникальности логина.
        #Нужна для поддержания актуальности контактной
информации пользователей и предоставления
инструментов самообслуживания в системе.
```

Продолжение листинга В.2

```
def show_my_profile(request): #Объявляем функцию
    отображения и редактирования профиля
    if 'user_id' not in request.session: #Проверяем наличие
        идентификатора пользователя в сессии
        messages.error(request, 'Необходимо
        авторизоваться') #Выводим сообщение о необходимости
        входа
        return redirect('login_view') #Перенаправляем на
        страницу авторизации
    try: #Начинаем блок обработки возможных ошибок
        доступа к данным
        user_id = request.session['user_id'] #Извлекаем ID
        текущего пользователя из сессии
        user_role = request.session.get('user_role', '')
        #Получаем роль пользователя из сессии
        employee = get_object_or_404(Employee,
        id=user_id, delete_date__isnull=True) #Находим активную
        запись сотрудника или возвращаем ошибку 404
        if request.method == 'POST': #Проверяем, что
        форма редактирования была отправлена
            if user_role == 'Администратор': #Проверяем
            права администратора для расширенного
            редактирования
                employee.first_name =
                request.POST.get('first_name', '').strip() #Обновляем имя из
                данных формы
                employee.last_name =
                request.POST.get('last_name', '').strip() #Обновляем
                фамилию из данных формы
                employee.middle_name =
                request.POST.get('middle_name', '').strip() #Обновляем
                отчество из данных формы
                employee.login = request.POST.get('login',
                '').strip() #Обновляем логин из данных формы
                employee.phone_number =
                request.POST.get('phone_number', '').strip() #Обновляем
                номер телефона
                employee.position_id =
                request.POST.get('position') #Обновляем привязку к
                должности
                employee.department_id =
                request.POST.get('department') #Обновляем привязку к
                отделу
                employee.office_id = request.POST.get('office')
            #Обновляем привязку к кабинету
            if
            Employee.objects.filter(login=employee.login).exclude(id=em
            ployee.id).exists(): #Проверяем уникальность логина,
            исключая текущего пользователя
                messages.error(request, 'Такой логин уже
                занят!') #Выводим сообщение о занятом логине
```

```
        return redirect('show_my_profile')
    #Возвращаем на страницу профиля
    else: #Блок редактирования для обычных ролей
        (Сотрудник, Техник, Руководитель)
        employee.first_name =
        request.POST.get('first_name', '').strip() #Обновляем имя из
        данных формы
        employee.last_name =
        request.POST.get('last_name', '').strip() #Обновляем
        фамилию из данных формы
        employee.middle_name =
        request.POST.get('middle_name', '').strip() #Обновляем
        отчество из данных формы
        employee.login = request.POST.get('login',
        '').strip() #Обновляем логин из данных формы
        employee.phone_number =
        request.POST.get('phone_number', '').strip() #Обновляем
        номер телефона
        if
        Employee.objects.filter(login=employee.login).exclude(id=em
        ployee.id).exists(): #Проверяем уникальность логина,
        исключая текущего пользователя
            messages.error(request, 'Такой логин уже
            занят!') #Выводим сообщение о занятом логине
            return redirect('show_my_profile')
    #Возвращаем на страницу профиля
    new_password = request.POST.get('password', '')
    #Извлекаем новый пароль из формы
    if new_password: #Проверяем, было ли поле
    пароля заполнено
        employee.password = new_password
    #Обновляем пароль в записи сотрудника
    employee.save() #Сохраняем все изменённые
    данные в базе данных
    messages.success(request, 'Профиль
    обновлён!') #Выводим сообщение об успешном
    сохранении
    request.session['user_name'] =
    f'{employee.last_name} {employee.first_name}
    {employee.middle_name}'.strip() #Обновляем ФИО в
    текущей сессии
    return redirect('show_my_profile')
    #Перезагружаем страницу профиля с новыми данными
    context = { #Формируем словарь данных для
    передачи в HTML-шаблон
        'employee': employee, #Передаём объект
        текущего сотрудника для отображения в форме
        'user_role': user_role, #Передаём роль
        пользователя для управления видимостью полей
        'positions':
        Position.objects.filter(delete_date__isnull=True), #Передаём
        список активных должностей
```

Продолжение листинга В.2

```
'departments':
Department.objects.filter(delete_date__isnull=True),
#Передаём список активных отделов

'offices':
Office.objects.filter(delete_date__isnull=True).select_related(
'building'), #Передаём кабинеты с подгрузкой корпусов

'user_name': request.session.get('user_name', ''),
#Передаём имя пользователя для шапки сайта
}
return render(request, 'show_my_profile.html',
context) #Рендерим страницу профиля с подготовленным
контекстом

except Exception as e: #Получим непредвиденные
ошибки при выполнении запроса
messages.error(request, f'Ошибка: {str(e)}')
#Выводим текст возникшего исключения
return redirect('login_view') #Перенаправляем
пользователя на страницу входа

#Функция генерирует и сохраняет печатную форму акта
о подаче новой заявки, проверяя права доступа и наличие
дубликатов файлов.

#Метод использует ORM Django, регулярные
выражения для парсинга имён файлов и стандартные
библиотеки для работы с файловой системой.

#Нужна для документальной фиксации обращения на
этапе регистрации без возможности редактирования
после печати.

def print_new_request(request, request_id):
    if 'user_id' not in request.session: #Проверяем наличие
идентификатора пользователя в сессии
        messages.error(request, 'Нет доступа') #Выводим
сообщение об отказе в доступе
        return redirect('login_view') #Перенаправляем на
страницу входа
        user_role = request.session.get('user_role', '')
#Извлекаем роль текущего пользователя из сессии
        if user_role not in ['Техник', 'Администратор',]:
#Проверяем соответствие роли списку разрешённых
            messages.error(request, 'Нет доступа') #Блокируем
доступ для неавторизованных ролей
            return redirect('login_view') #Возвращаем
пользователя на страницу входа
        try:
            req = get_object_or_404(RequestFix,
act_number=request_id) #Находим заявку по номеру акта
или возвращаем ошибку 404
            forms_folder = os.path.join(settings.MEDIA_ROOT,
'request_forms') #Формируем абсолютный путь к папке
печатных форм
            os.makedirs(forms_folder, exist_ok=True) #Создаём
директорию, если она ещё не существует
```

```
registration_date_str =
req.registration_date.strftime('%Y%m%d') #Преобразуем
дату регистрации в строку формата YYYYMMDD

duplicate_key =
f"new_{req.act_number}_{registration_date_str}"
#Формируем уникальный ключ для поиска дубликатов
existing_file = None #Инициализируем переменную
для имени найденного файла
if os.path.exists(forms_folder): #Проверяем
физическое наличие директории
    for filename in os.listdir(forms_folder):
#Перебираем все файлы в папке
        if filename.startswith('Акт_о_подаче_заявки_')
and filename.endswith('.html'): #Фильтруем только файлы
актов новой заявки
            if duplicate_key in filename: #Проверяем
совпадение с уникальным ключом
                existing_file = filename #Сохраняем имя
найденного файла
                break #Прерываем цикл после первого
совпадения для оптимизации
            if existing_file: #Если файл уже существует
                match = re.search(r'_(\d+)\.html$', existing_file)
#Извлекаем номер акта из имени файла с помощью
регулярного выражения
                act_number = int(match.group(1)) if match else 1
#Преобразуем извлечённый номер в целое число или
ставим 1
            context = { #Формируем контекст для передачи
данных в шаблон
                'request': req, #Передаём объект заявки
                'organization_name': 'ГБУЗ РХ «Абаканская
межрайонная клиническая больница», #Добавляем
название организации
                'print_date':
timezone.now().strftime('%d.%m.%Y %H:%M'), #Добавляем
текущую дату и время печати
                'act_number': act_number, #Добавляем
извлечённый номер акта
                'existing_file': True, #Указываем, что файл уже
существует
            }
            return render(request, 'print_new_request.html',
context) #Рендерим шаблон с данными существующего
файла

max_act_number = 0 #Инициализируем
переменную для поиска максимального номера акта
if os.path.exists(forms_folder): #Проверяем наличие
директории
    for filename in os.listdir(forms_folder):
#Перебираем файлы в папке
```

Продолжение листинга В.2

```
if filename.startswith('Акт_о_подаче_заявки_')
and filename.endswith('.html'): #Фильтруем файлы по
имени и расширению
    match = re.search(r'_(\d+)\.html$', filename)
#Извлекаем номер акта из имени файла
    if match: #Если номер успешно извлечён
        num = int(match.group(1)) #Преобразуем
строку в число
        if num > max_act_number: #Сравниваем с
текущим максимумом
            max_act_number = num #Обновляем
максимальное значение
        new_act_number = max_act_number + 1
#Вычисляем новый уникальный номер акта
        context = { #Формируем новый контекст для
генерации документа
            'request': req, #Передаём объект заявки
            'organization_name': 'ГБУЗ РХ «Абаканская
межрайонная клиническая больница», #Добавляем
название организации
            'print_date': timezone.now().strftime('%d.%m.%Y'),
#Добавляем текущую дату
            'act_number': new_act_number, #Добавляем
новый номер акта
            'existing_file': False, #Указываем, что файл
создаётся впервые
        }
        from django.template.loader import render_to_string
#Импортируем функцию рендеринга шаблона в строку
        html_string =
render_to_string('print_new_request.html', context)
#Генерируем HTML-код акта из шаблона
        filename =
f"Акт_о_подаче_заявки_{new_act_number}_{req.act_numb
er}_{registration_date_str}.html" #Формируем уникальное
имя файла
        file_path = os.path.join(forms_folder, filename)
#Объединяем путь и имя для получения полного пути
        with open(file_path, 'w', encoding='utf-8') as f:
#Открываем файл в режиме записи с кодировкой UTF-8
            f.write(html_string) #Записываем
сгенерированный HTML в файл
        return render(request, 'print_new_request.html',
context) #Открываем созданный акт в браузере для
печати
    except Exception as e: #Обрабатываем любые
непредвиденные ошибки
        messages.error(request, f'Ошибка: {str(e)}')
#Выводим сообщение с текстом ошибки
        return redirect('show_technician') #Возвращаем
пользователя на страницу техника
```

```
#Функция формирует акт о выполнении заявки после
перевода её в статус «Завершена», предотвращая
создание дубликатов документов.
#Метод использует ORM для проверки этапа ремонта,
файловые операции для поиска и сохранения актов, а
также рендеринг Django-шаблонов.
#Нужна для официального закрытия заявки и
предоставления заказчику документа с перечнем
выполненных работ и затрат.
def print_completed_request(request, request_id):
    if 'user_id' not in request.session: #Проверяем наличие
идентификатора пользователя в сессии
        messages.error(request, 'Нет доступа') #Выводим
сообщение об отказе в доступе
        return redirect('login_view') #Перенаправляем на
страницу входа
        user_role = request.session.get('user_role', '')
#Извлекаем роль текущего пользователя из сессии
        if user_role not in ['Техник', 'Администратор']:
#Проверяем соответствие роли списку разрешённых
        messages.error(request, 'Нет доступа') #Блокируем
доступ для неавторизованных ролей
        return redirect('login_view') #Возвращаем
пользователя на страницу входа
    try:
        req = get_object_or_404(RequestFix,
act_number=request_id) #Находим заявку по номеру акта
или возвращаем ошибку 404
        if req.repair_stage.name != 'Завершена':
#Проверяем, находится ли заявка на этапе «Завершена»
        messages.error(request, 'Печать доступна только
для заявок с этапом ремонта "Завершена"') #Выводим
сообщение об ограничении
        return redirect('show_technician') #Возвращаем
пользователя на страницу списка заявок
        forms_folder = os.path.join(settings.MEDIA_ROOT,
'request_forms') #Формируем путь к директории хранения
актов
        os.makedirs(forms_folder, exist_ok=True) #Создаём
папку при её отсутствии
        completion_date_str =
req.completion_date.strftime('%Y%m%d') if
req.completion_date else 'no_date' #Преобразуем дату
выполнения или ставим заглушку
        duplicate_key =
f"completed_{req.act_number}_{completion_date_str}"
#Формируем уникальный ключ для поиска дубликатов
акта выполнения
        existing_file = None #Инициализируем переменную
для найденного файла
        if os.path.exists(forms_folder): #Проверяем
существование директории
```

Продолжение листинга В.2

```
for filename in os.listdir(forms_folder):
#Перебираем файлы в папке
    if
filename.startswith('Акт_об_выполнении_Заявки_') and
filename.endswith('.html'): #Фильтруем файлы актов
выполнения
        if duplicate_key in filename: #Ищем
совпадение по уникальному ключу
            existing_file = filename #Сохраняем имя
найденного файла
            break #Прерываем цикл после первого
совпадения
        if existing_file: #Если файл уже существует
            match = re.search(r'_(\d+)\.html$', existing_file)
#Извлекаем номер акта из имени файла
            act_number = int(match.group(1)) if match else 1
#Преобразуем в число или ставим 1
            context = { #Формируем контекст для
существующего документа
                'request': req, #Передаём объект заявки
                'organization_name': 'ГБУЗ РХ «Абаканская
межрайонная клиническая больница», #Добавляем
название организации
                'print_date':
timezone.now().strftime('%d.%m.%Y %H:%M'), #Добавляем
текущую дату и время
                'act_number': act_number, #Добавляем номер
акта
                'existing_file': True, #Указываем на
существование файла
            }
            return render(request,
'print_completed_request.html', context) #Рендерим шаблон
с готовым актом
            max_act_number = 0 #Инициализируем счётчик
максимального номера
            if os.path.exists(forms_folder): #Проверяем наличие
папки
                for filename in os.listdir(forms_folder):
#Перебираем файлы
                    if
filename.startswith('Акт_об_выполнении_Заявки_') and
filename.endswith('.html'): #Фильтруем по имени и
расширению
                        match = re.search(r'_(\d+)\.html$', filename)
#Извлекаем номер акта
                        if match: #Если номер найден
                            num = int(match.group(1)) #Преобразуем в
целое число
                            if num > max_act_number: #Сравниваем с
текущим максимумом
```

```
max_act_number = num #Обновляем
максимум
            new_act_number = max_act_number + 1
#Вычисляем следующий уникальный номер
            context = { #Формируем контекст для нового акта
                'request': req, #Передаём объект заявки
                'organization_name': 'ГБУЗ РХ «Абаканская
межрайонная клиническая больница», #Добавляем
название организации
                'act_number': new_act_number, #Добавляем
новый номер акта
                'existing_file': False, #Указываем на создание
нового файла
            }
            from django.template.loader import render_to_string
#Импортируем функцию рендеринга
            html_string =
render_to_string('print_completed_request.html', context)
#Генерируем HTML-код акта
            filename =
f"Акт_об_выполнении_Заявки_{new_act_number}_{req.act
_number}_{completion_date_str}.html" #Формируем имя
файла
            file_path = os.path.join(forms_folder, filename)
#Получаем полный путь сохранения
            with open(file_path, 'w', encoding='utf-8') as f:
#Открываем файл для записи
                f.write(html_string) #Сохраняем HTML-
содержимое на диск
            return render(request,
'print_completed_request.html', context) #Открываем
готовый акт для печати
        except Exception as e: #Обрабатываем ошибки
выполнения
            messages.error(request, f'Ошибка: {str(e)}')
#Выводим текст ошибки
            return redirect('show_technician') #Возвращаем на
страницу техника
        #Функция асинхронно сохраняет акт выполнения заявки
в папку через AJAX-запрос, возвращая путь к файлу в
формате JSON.
        #Метод использует ORM с оптимизированными
запросами, файловые операции для проверки дубликатов
и генерации HTML.
        #Нужна для автоматической архивации документов без
перезагрузки страницы и обеспечения быстрого доступа к
файлам через интерфейс.
        def save_request_form_to_folder(request, request_id):
            if 'user_id' not in request.session: #Проверяем наличие
сессии пользователя
```

Продолжение листинга В.2

```
        return JsonResponse({'error': 'Необходимо
авторизоваться'}, status=403) #Возвращаем ошибку
авторизации

        if request.session.get('user_role') not in ['Техник',
'Администратор']: #Проверяем роль пользователя
            return JsonResponse({'error': 'Нет доступа'},
status=403) #Возвращаем ошибку доступа

        try:
            req =
get_object_or_404(RequestFix.objects.select_related(
#Находим активную заявку с подгрузкой связанных
данных

            'requester', 'assigned_technician', 'equipment'
            ).prefetch_related('used_spare_parts',
'used_services'), act_number=request_id,
delete_date__isnull=True)

            forms_folder = os.path.join(settings.MEDIA_ROOT,
'request_forms') #Формируем путь к папке актов
            os.makedirs(forms_folder, exist_ok=True) #Создаём
директорию при необходимости

            completion_date_str =
req.completion_date.strftime('%Y%m%d') if
req.completion_date else 'no_date' #Получаем дату
выполнения или заглушку

            duplicate_key =
f"completed_{req.act_number}_{completion_date_str}"
#Формируем ключ для поиска дубликата

            existing_file = None #Инициализируем переменную
для файла

            if os.path.exists(forms_folder): #Проверяем наличие
папки

                for filename in os.listdir(forms_folder):
#Перебираем файлы

                    if
filename.startswith('Акт_об_выполнении_Заявки_') and
filename.endswith('.pdf'): #Ищем файлы PDF актов
выполнения

                        if duplicate_key in filename: #Проверяем
совпадение ключа

                            existing_file = filename #Сохраняем имя
файла

                                break #Прерываем цикл

                            if existing_file: #Если файл уже существует

                                return JsonResponse({ #Возвращаем JSON с
путём к существующему файлу

                                    'success': True, #Указываем на успешный
поиск

                                    'file_path': f'/media/request_forms/{existing_file}',
#Передаём относительный путь

                                    'existing': True #Фиксируем факт
существования
```

```
                                })

                                max_act_number = 0 #Инициализируем максимум
номера акта

                                if os.path.exists(forms_folder): #Проверяем папку

                                    for filename in os.listdir(forms_folder):
#Перебираем файлы

                                        if
filename.startswith('Акт_об_выполнении_Заявки_') and
filename.endswith('.pdf'): #Фильтруем PDF файлы

                                            match = re.search(r'_(\d+)\.html$', filename)
#Извлекаем номер из имени (логика парсинга)

                                                if match: #Если номер найден

                                                    num = int(match.group(1)) #Преобразуем в
число

                                                        if num > max_act_number: #Сравниваем с
максимумом

                                                            max_act_number = num #Обновляем
значение

                                                                new_act_number = max_act_number + 1
#Вычисляем новый номер

                                                                    html_string =
render_to_string('print_completed_request.html', {
#Генерируем HTML из шаблона

                                                                        'request': req, #Передаём заявку

                                                                        'organization_name': 'ГБУЗ РХ «Абаканская
межрайонная клиническая больница», #Название
организации

                                                                        'print_date': timezone.now().strftime('%d.%m.%Y
%H:%M'), #Текущая дата и время

                                                                        'act_number': new_act_number, #Новый номер
акта

                                                                            })

                                                                                filename =
f"Акт_об_выполнении_Заявки_{new_act_number}_{req.act
_number}_{completion_date_str}.pdf" #Формируем имя PDF
файла

                                                                                    file_path = os.path.join(forms_folder, filename)
#Получаем полный путь

                                                                                        with open(file_path, 'w', encoding='utf-8') as f:
#Открываем файл для записи

                                                                                            f.write(html_string) #Сохраняем HTML-код в файл

                                                                                                return JsonResponse({ #Возвращаем успешный
JSON ответ

                                                                                                    'success': True, #Подтверждаем успешное
сохранение

                                                                                                    'file_path': f'/media/request_forms/{filename}',
#Передаём путь к новому файлу

                                                                                                    'existing': False, #Указываем на создание нового
файла

                                                                                                    'act_number': new_act_number #Возвращаем
присвоенный номер акта

                                                                                                        })
```


Продолжение листинга В.2

```
except Exception as e: #Обрабатываем ошибки
сервера
    return JsonResponse({'error': str(e)}, status=500)
#Возвращаем текст ошибки с кодом 500
```

ПРИЛОЖЕНИЕ Г

РУКОВОДСТВО ОПЕРАТОРА

Настоящее руководство предназначено для пользователей автоматизированной системы учёта заявок по неисправностям техники ГБУЗ РХ «АМКБ». Документ содержит пошаговые инструкции по работе с интерфейсом системы, описание функциональных возможностей для каждой роли пользователя, правила формирования отчётной документации, а также рекомендации по устранению типовых ошибок при эксплуатации.

Система рассчитана на четыре основные роли:

- сотрудник;
- техник;
- руководитель;
- администратор.

Доступ к функциям строго регламентирован политикой информационной безопасности - неавторизованные пользователи направляются на страницу входа, а попытки доступа к закрытым разделам блокируются с выводом соответствующего уведомления.

1. Начало работы с системой

1.1. Запуск и вход в систему

1. откройте веб-браузер и перейдите по адресу локального сервера;
2. на главной странице откроется форма «Вход в систему»;
3. введите зарегистрированный логин и пароль;
4. нажмите кнопку «Войти».

При успешной аутентификации система автоматически перенаправит вас на персональную панель в соответствии с назначенной ролью.

При ошибке ввода отобразится сообщение: «Неверный пароль» или «Пользователь с таким логином не найден». Записи, помеченные как удалённые, не допускаются к входу.

1.2. Самостоятельная регистрация

1. на странице входа перейдите по ссылке «Регистрация»;
2. заполните обязательные поля: Фамилия, Имя, Логин, Пароль. По желанию укажите телефон, должность, отдел и кабине;

3. нажмите «Зарегистрироваться».

Система автоматически присвоит роль «Сотрудник». При совпадении паролей и отсутствии дубликата логина произойдёт автоматический вход в личный кабинет.

1.3. Выход из системы

Для завершения сеанса нажмите кнопку «Выйти» в верхнем меню. Сессия будет полностью очищена, данные формы сброшены, пользователь перенаправлен на страницу входа.

2. Работа по ролям пользователей

2.1. Роль «Сотрудник»

Назначение: регистрация неисправностей закреплённого оборудования и отслеживание статуса обращений. Просмотреть действия можно в таблице Г.1.

Таблица Г.1 – Действия сотрудника

Действие	Инструкция
Просмотр заявок	В личном кабинете отображается таблица с вашими заявками. Используйте строку поиска для фильтрации по описанию, номеру акта или инвентарному номеру. Применяйте фильтр по статусу оборудования и сортировку по дате/номеру.
Создание заявки	Нажать «Создать заявку». В форме автоматически доступен только список оборудования, закреплённого за вашим кабинетом. Заполнить описание неисправности, выберите категорию и приоритет. Нажмите «Создать». Система защитит от дублирования при обновлении страницы.
Архивирование заявки	Напротив выполненных заявок доступна кнопка «Удалить». Нажатие переводит заявку в архив. Удалить незавершённые или чужие заявки невозможно.

2.2. Роль «Техник»

Назначение: полный цикл обработки заявок, управление оборудованием, формирование актов и расчёт затрат. Просмотреть действия можно в таблице Г.2.

Таблица Г.2 – Действия техника

Действие	Инструкция
Панель заявок	Главный экран разделён на блоки: «Полностью новые», «Требуется техника», «В работе». Отображается счётчик необработанных обращений. Доступны расширенный поиск, фильтрация по приоритету и статусу, сортировка по дате, номеру акта, фамилии заказчика
Редактирование заявки	Откройте заявку, нажмите «Редактировать». Доступно изменение описания, исполнителя, оборудования, категории, этапа и статуса. В блоках «Запчасти» и «Услуги» добавляйте позиции, указывайте количество и цену на момент ремонта. При необходимости загружайте сканы чеков

Продолжение таблицы Г.2

Печать актов	Доступны два типа документов: «Акт о подаче заявки» и «Акт об выполнении заявки». Печать акта выполнения доступна только при этапе ремонта «Завершена». Система проверяет наличие ранее сгенерированного файла, предотвращая дублирование
Управление оборудованием	Раздел «Оборудование» позволяет добавлять, редактировать и архивировать технику. При создании инвентарный номер формируется автоматически. Поддерживается загрузка фотографий, указание комплектации, даты приобретения и гарантийных сроков
История и затраты	Перейдите в карточку оборудования на «История ремонтов». Отображается список всех обращений, общее количество поломок, суммарная и средняя стоимость обслуживания. Финансовые данные видны только «Руководителю» и «Администратору»

2.3. Роль «Руководитель»

Назначение: Мониторинг эффективности работы отдела, анализ затрат, управление справочниками. Просмотреть действия можно в таблице Г.3.

Таблица Г.3 – Действия руководителя

Действие	Инструкция
Страница аналитики	На главной странице отображаются: общая статистика заявок, столбчатая диаграмма, оборудование по частоте поломок, графики затрат по месяцам и типам техники. Данные генерируются автоматически и сохраняются в директории media/charts/
Затраты по оборудованию	Выберите конкретную единицу техники или просмотрите общую сводку. Задайте период фильтрации. Система отобразит детализацию по запчастям и услугам с расчётом итоговой суммы
Быстрое создание справочников	В формах добавления оборудования доступны кнопки быстрого создания: тип, модель, производитель, кабинет, гарантия. Запись создаётся без перезагрузки страницы, при совпадении названия система вернёт существующий ID

2.4. Роль «Администратор»

Назначение: администрирование учётных записей, полный контроль над системой, резервное управление данными. Просмотреть действия можно в таблице Г.4.

Таблица Г.4 – Действия администратора

Действие	Инструкция
Управление сотрудниками	В разделе «Сотрудники» доступна таблица со всеми пользователями. Доступны: поиск по ФИО/логину, фильтр по ролям, создание, редактирование, архивирование и полное удаление. Администратор не может удалить собственную учётную запись. При редактировании пароль обновляется только при явном заполнении поля
Архив и восстановление	Раздел «Удалённые сотрудники» и «Удалённое оборудование» содержат записи с установленной delete_date. Нажмите «Восстановить» для возврата в активный реестр. Кнопка «Удалить навсегда» доступна только Администратору и удаляет запись физически
Расширенные права	Администратор имеет полный доступ ко всем функциям Техника и Руководителя, включая печать актов, редактирование заявок, управление оборудованием и просмотр аналитики

3. Общие операции и элементы интерфейса

3.1. Поиск, фильтрация и сортировка

Поиск: введите текст в поле «Поиск». Система ищет по описанию проблемы, номеру акта, инвентарному номеру, ФИО заказчика.

Фильтрация: выберите значение в выпадающих списках «Статус», «Приоритет», «Роль».

Сортировка: кликайте по заголовкам столбцов или используйте кнопки. Доступна сортировка по дате регистрации, номеру акта, фамилии, статусу оборудования.

3.2. Личный профиль

1. перейдите в раздел «Мой профиль»;
2. отредактируйте ФИО, логин, телефон, должность, отдел, кабинет;
3. при изменении логина система проверит его на уникальность;
4. нажмите «Сохранить». Обновлённое имя отобразится в сессии и шапке сайта немедленно.

4. Формирование и печать документов

1. откройте необходимую заявку или раздел оборудования;
2. нажмите кнопку «Печать акта» (подача или выполнение);
3. система проверит этап ремонта и наличие дубликатов в папке `media/request_forms/`;
4. откроется страница с заполненным шаблоном, содержащим: наименование организации, данные заявителя и исполнителя, перечень запчастей/услуг, итоговую стоимость, дату формирования.

Используйте стандартную функцию печати браузера «Сохранить как PDF» или отправка на принтер.

Файлы сохраняются на сервере в формате HTML/PDF и доступны для последующего скачивания или аудита.

ПРИЛОЖЕНИЕ Д
ПРОГРАММНЫЙ ПРОДУКТ НА ДИСКЕ